

药材热风烘干过程的热-质耦合模型与收缩域烘干时长预测

摘 要

药材热风烘干的时长取决于内部传热传质阻力。本文以圆柱形药材径向截面为对象，建立温度场与含水率场耦合的一维非线性抛物方程组，按预热、变物性、达标时长与失水收缩四个层次递进求解。

针对问题一，按傅里叶导热与菲克扩散定律写出柱坐标守恒关系，表面取**第三类边界** ($h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$)，轴心取对称零通量；毕奥数 $Bi_h = 1.389$ 、 $Bi_m = 3.240$ 均为 $O(1)$ ，故集中参数与第一类边界都不适用。常物性下温度可独立求解，扩散系数仍随含水率逐点更新。采用**全隐式有限体积法**离散、Thomas 消元推进至 1800 s，得中心 33.577°C 与 2.5500 kg/kg 、表面 36.786°C 与 1.511 kg/kg 。热扩散率与扩散系数之比约 34，「热先于质」得到定量刻画；以 Bessel 级数叠加 Duhamel 卷积独立对照，末时刻温度偏差仅 0.0012°C 、全程不超过 0.0020°C 。

针对问题二，四条经验式使密度、比热与导热系数随含水率变化，扩散系数同时含 $e^{-0.45/C}$ 与活化能因子 e^{-3850/T_K} ，两场经物性双向耦合。采用**保序算子分裂**并以 Picard 迭代复核：先由旧场算热物性解温度，再用新温度算扩散系数解含水率，最多 3 次收敛。3 h 时中心已升至 49.85°C 接近烘房温度，中心含水率仍为 1.766 kg/kg 、表面 1.008 kg/kg ，表明温度场先达准稳态，其后由内部扩散阻力控制。

针对问题三，定义达标事件 $t_{\text{end}} = \min\{t : \max_r C(r, t) < 0.15\}$ ，取逐点最大值而非体平均（后者会在芯部尚未达标时提前跨过阈值），控制点为含水率最高的轴心，并在输出亚步间线性插值定位。求得**烘干时长** 56.706 h ，达标时各位置含水率由中心 0.1500 递降至表面 0.0525 ；干燥速率自 $0.6899 \text{ kg}/(\text{kg} \cdot \text{h})$ 起单调衰减，全程无恒速段。

针对问题四，外边界随失水内移。附件 2 全部 145 个半径数据经单调化与 PCHIP 保形插值给出 $R(t)$ 。干基含水率跟随固体骨架，故以物质坐标 $\eta = (r/R(t))^2$ 为主实现，收缩进入 $4/R^2$ 几何因子，固相速度由均匀收缩唯一确定为 $v_s = r\dot{R}/R$ 。Eulerian Landau 变换作为交叉验证，二者相差 2.78%。求得**时长** 50.780 h 、末态半径 1.20 cm ，较同物性尺寸冻结情形缩短 60.6%。补齐「原物性 + 收缩」后的 2×2 因子设计给出交互项 -46.240 h ，与主效应同量级，故物性贡献与收缩贡献不能作唯一加性分解；Shapley 平均分别为 $+48.812 \text{ h}$ 与 -55.006 h 。反演的体积收缩系数变异系数 22%，线性收缩律不成立。

四问离散水分方程收支残差均不超过 7×10^{-14} ；扩散系数前因子是最强不确定度来源，忽略蒸发潜热使时长预测偏短约一成。

关键字： 热-质耦合 有限体积法 移动边界 物质坐标 烘干时长预测

一、问题重述

1.1 问题背景

中药材采收后含水量高，必须及时干燥到安全水分以下才能贮藏与流通。热风烘干是产地初加工中最常用的方式：药材置于烘房，热空气以对流方式向表面供热并带走表面析出的水分，水分则依靠内部扩散由芯部向表面迁移。这一过程的困难在于两种输运的时间尺度并不相同。热量在药材内部传播得快，水分迁移得慢，因而芯部往往在表面已经明显干燥之后很久才达到安全水分。干燥不足会留下霉变风险，过度延长烘干时间则既增加能耗，又可能损失热敏性有效成分^[1]。因此，能否事先预测内部温度与水分的分布演化、并据此给出可靠的烘干时长，直接决定烘干工艺的参数设定与设备排产。

真实药材在失水过程中还会收缩。体积缩小一方面缩短了水分向表面扩散的路径，一方面改变了表面积与体积之比，对干燥快慢的影响并非单向^[2]。同时，药材的密度、比热容、导热系数和水分扩散系数都随含水率变化，扩散系数还强烈依赖温度。这些依赖关系把原本可以分开处理的传热与传质问题耦合成一个整体，必须同时求解温度场与水分场才能得到自洽的结果。

1.2 问题的提出

本题以圆柱形药材为对象。药材初始半径 2 cm、长度 25 cm，初始温度 28 °C、初始干基含水率 2.55 kg/kg，被送入烘房干燥。烘房内热空气的温度与含湿量随时间变化，由附件给出的实测时序描述；药材表面与空气之间的对流换热系数和对流传质系数为已知常数。四个子问题沿同一物理过程逐步放开限制条件：前两问要求刻画给定时段内的场分布，后两问要求给出达到安全水分所需的烘干时长，其中最后一问还需考虑药材的实际收缩。

问题一考察预热阶段。此时物性可按常数处理，水分扩散系数只随局部含水率变化。需要建立药材内部的温度与水分输运关系，求出前 1800 s 内若干时刻沿半径若干位置的温度与含水率，并给出全过程的完整场数据。

问题二把物性的真实依赖关系全部放开。密度、比热容与导热系数随含水率变化，水分扩散系数同时随含水率和温度变化，温度场与水分场因此互相牵动。需要在这一耦合条件下重新刻画整个烘干过程，并给出前 3 h 内的温度与含水率分布。

问题三在问题二的模型上追加干燥完成的判定。当药材内部含水率降到 0.15 kg/kg 以下即视为烘干完成，需要给出达到该状态所需的时长，并给出全过程中若干时刻的含水率分布。

问题四进一步考虑药材的径向收缩。半径随时间的实测数据由附件给出，物性关系也相应更换。求解域不再固定，需要在随时间缩小的区域上重建模型，重新给出烘干时长与含水率分布，并说明收缩这一因素究竟把烘干时长改变了多少。

四问之间是逐层承接的关系：问题一提供常物性条件下的基准与可用于独立校核的简化情形，问题二在此之上建立双向耦合，问题三为耦合模型补上终止条件，问题四把固定求解域换成动域。每一问只新增一项机制，后一问相对前一问的差异因而可以归因到该项机制上。

二、问题分析

四问共享同一个物理对象与同一套输运机理，差异只在物性关系、是否需要终止判据、求解域是否随时间变化。据此把公共部分前置为统一的连续模型与离散方法，各问只陈述本问新增的内容，其间的依赖次序见图 1。

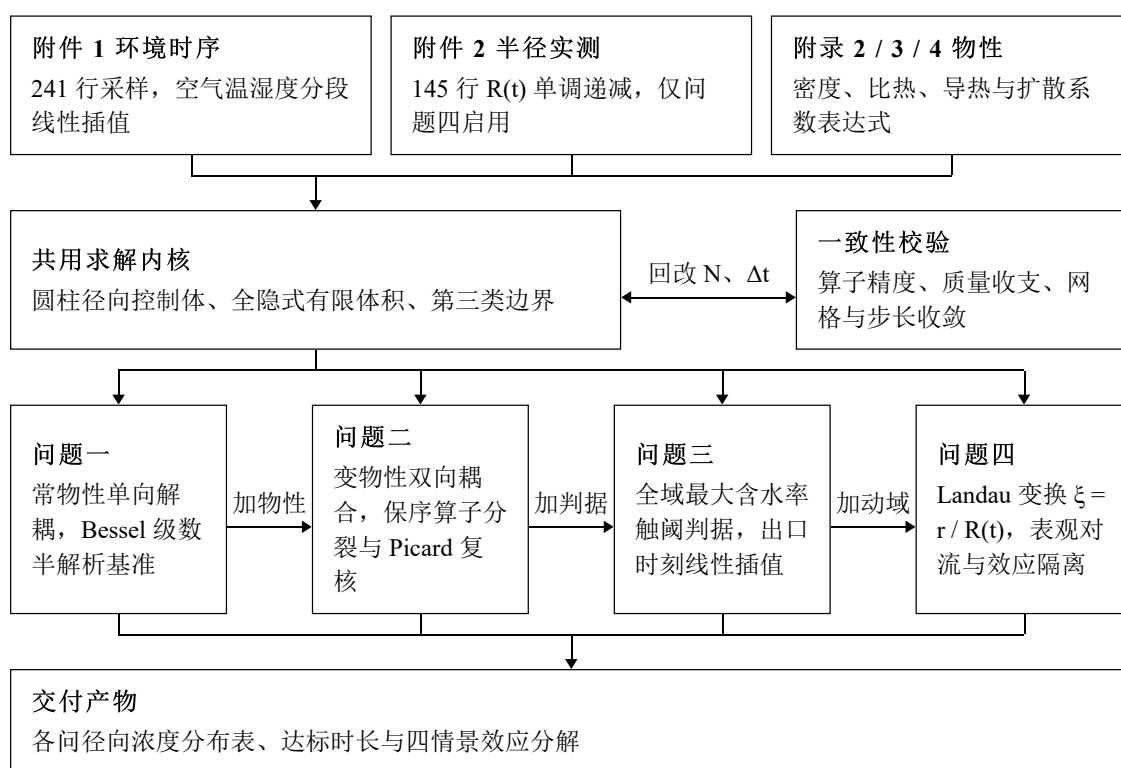


图 1 全文建模路线

图中三份题给材料汇入同一求解内核，内核与一致性校验之间是双向边：算子精度、质量收支和网格与步长收敛一旦不达标，回改的是控制体数与时间步长，而不是物性参数。四问在图中呈链式而非并列排布，正是因为每一问只新增一项机制。

2.1 问题一的分析

本问是给定环境驱动下的瞬态输运正问题。难点不在于方程本身，而在于两个场的时间尺度相差悬殊：按附录 2 的常物性计算，热扩散率 $\alpha = k/(\rho c_p) = 1.689 \times 10^{-7} \text{ m}^2/\text{s}$,

而初始含水率下的水分扩散系数约为 $4.94 \times 10^{-9} \text{ m}^2/\text{s}$ ，两者相差约 34 倍。这意味着 1800 s 内温度已能穿透整个半径，水分的可见变化却仍局限在表层。若用集中参数（把药材视为温度均匀的一个整体）处理，这一空间差异会被完全抹去，因此必须保留径向分布。以表面对流系数估计的毕奥数 $Bi_h = hR/k = 1.389$ 、 $Bi_m = h_m R/D = 3.240$ 均为 $O(1)$ 量级，既不允许简化为表面温度等于空气温度的第一类边界，也不允许忽略内部阻力，第三类边界与分布参数模型是本题唯一自洽的选择。

本问的另一价值在于可校核性。常物性下能量方程与水分方程解除耦合，温度场退化为线性问题，可以用 Bessel 函数级数配合 Duhamel 叠加得到半解析解。这是全题唯一可以与数值解独立比对的外部数学基准，因此把它安排在本问完成，为后三问的同一套离散格式提供信心。

2.2 问题二的分析

本问在问题一基础上放开物性依赖，任务类型不变而数学类型改变。密度、比热容、导热系数随含水率变化，使能量方程的系数由水分场决定；水分扩散系数中含绝对温度的指数因子，使水分方程的系数由温度场决定。两个方向的依赖同时存在，温度场与水分场构成一个双向耦合环，不能先算完整条温度时程再算水分。图 2 把这一结构与问题一的解耦情形并置。

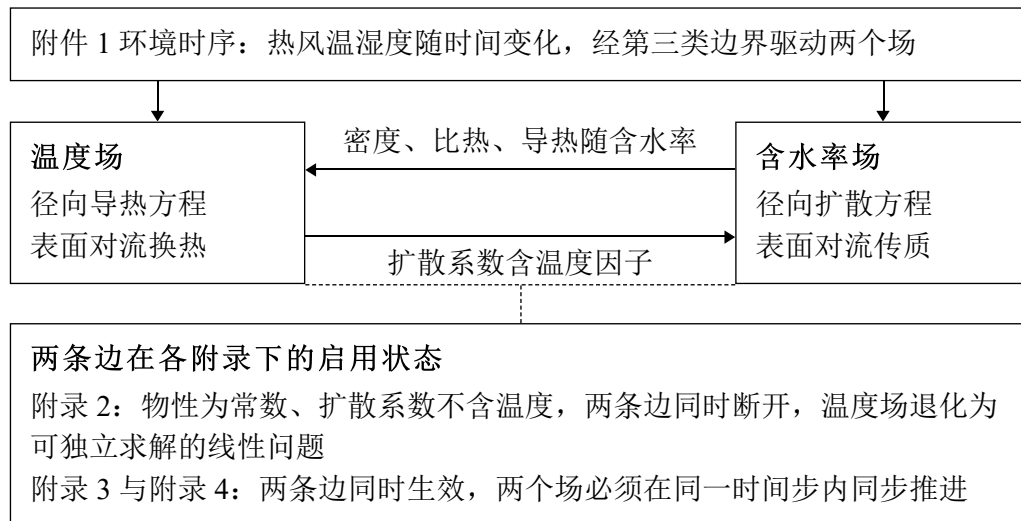


图 2 温度场与含水率场的耦合结构

图中向左的边是物性对含水率的依赖，向右的边是扩散系数中的温度因子。两条边在附录 2 下同时断开，温度场因而可以独立求解；附录 3 与附录 4 下两条边同时接通，两场必须在同一时间步内同步推进。求解上由此引出两个必须处理的问题：一是同一时间步内物性用哪一层的场计算，二是耦合是否已经收敛。本问采用保序的算子分裂配合

迭代复核解决，具体次序在公共模型一章给出。

2.3 问题三的分析

本问是在问题二的模型上追加一个事件型终止条件，模型结构不需改动。关键是判据本身如何定义。题目要求药材内部含水率降到阈值以下，而干燥过程中含水率沿半径始终不均匀，中心最高、表面最低。若以体平均含水率判定，会在芯部尚未达标时提前宣布完成；只有以逐点最大值判定，才能保证药材内所有位置都已达标。因此本问把达标时刻定义为逐点最大含水率首次低于阈值的时刻，并需要在输出步之间做插值定位，否则时长精度受输出间隔限制。达标时刻的量化差别有多大，在结果部分用同一次求解的两种判据读数直接对照。

2.4 问题四的分析

本问同时改变了两件事：物性关系由附录 3 换成附录 4，求解域由固定半径换成随时间收缩。几何域随时间变化带来的本质困难，是网格结点的物理位置不再固定。干基含水率以干物质为基准，收缩时固体骨架携带水分向内运动，因此宜把 C 放在物质点上描述。本问以 $\eta = (r/R(t))^2$ 为物质坐标，均匀收缩给出唯一的仿射固相速度 $v_s = r\dot{R}/R$ ，把动域化为定域且不再需要额外的对流开关；Eulerian Landau 变换保留表观对流项，用作交叉验证。

更需要警惕的是归因。两件事同时改变，直接比较问题四与问题三的时长并把差异说成「收缩使干燥加快」是混淆归因。因此本问补算缺失的「原物性 + 收缩半径」一格，形成完整的 2×2 因子设计，报告交互项，并以两条路径的平均给出物性贡献与收缩贡献。此外，附录 4 的密度关系与附件 2 的收缩数据在严格干物质守恒下并不完全相容，这一不相容度只作为题给数据的诊断量报告，不反过来修正实测半径。

三、模型假设

(1) 药材内部的温度与水分只沿径向变化，忽略轴向与周向的不均匀。药材长度 25 cm 与直径 4 cm 之比为 6.25，两端面面积仅占总换热面积的 7.4%；按扩散时间 L^2/D 估计，轴向输运所需时间约为径向的 39 倍，在本题关心的时间范围内轴向梯度不足以改变径向结果。

(2) 药材视为均质连续介质，水分以有效扩散的方式迁移，不区分液态水与水蒸气的输运路径。题目直接给出有效水分扩散系数的表达式，即已把毛细流动与气相扩散归并到同一个系数中，这是干燥文献中处理多孔物料的常用简化^[3]。

(3) 能量方程不含水分蒸发的潜热汇。题目未给出汽化潜热与相变发生位置，无法确定该汇项的大小与分布。忽略吸热会使药材升温偏快、扩散系数偏大，因此本文预测的烘干时长是偏短的乐观估计，结果讨论中会明确这一偏向。

(4) 烘房空气温度与含湿量在实测时序覆盖范围之外保持末端平台值，不作线性外推。附件 1 的温度在约 2.95 h 后已进入平台，继续线性外推会得到无物理依据的持续升温。

(5) 药材收缩只发生在径向，长度不变，且收缩不可逆；内部骨架按仿射方式随边界运动，即 $v_s = r\dot{R}/R$ 。题目仅给出半径随时间的实测数据，未给出长度变化与内部位移；失水造成的骨架塌缩在恒温干燥中不会自行回复，故半径取为单调不增。

(6) 表面对流换热系数与对流传质系数在四个子问题中取同一组给定值，不随温度、含水率或表面收缩而变化。题目在给出这两个系数时未附带修正关系，按题面条件统一使用。

四、符号说明

下表收录贯穿全文、直接进入核心方程与判据的符号。

表 1 主要符号说明

符号	含义	单位	取值或范围
r	到药材轴心的径向距离	m	$[0, R]$
t	时间	s	$[0, t_{\text{end}}]$
$T(r, t)$	药材内部温度	°C	$[28, 50.2]$
$C(r, t)$	药材内部干基含水率	kg/kg	$[0.0525, 2.55]$
T_K	绝对温度 $T + 273.15$	K	$[301.15, 323.32]$
w	湿基水分分率 $C/(C + 1)$	—	$[0.050, 0.718]$
ρ	药材密度	kg·m ⁻³	随物性关系而定

符号	含义	单位	取值或范围
c_p	定压比热容	$\text{J}\cdot\text{kg}^{-1}\cdot\text{K}^{-1}$	随物性关系而定
k	导热系数	$\text{W}\cdot\text{m}^{-1}\cdot\text{K}^{-1}$	随物性关系而定
D	水分有效扩散系数	$\text{m}^2\cdot\text{s}^{-1}$	$1.6 \times 10^{-10} - 1.4 \times 10^{-8}$
h	表面对流换热系数	$\text{W}\cdot\text{m}^{-2}\cdot\text{K}^{-1}$	25.0
h_m	表面对流传质系数	$\text{m}\cdot\text{s}^{-1}$	8×10^{-7}
$T_{\text{air}}(t)$	烘房空气温度	$^{\circ}\text{C}$	[28.000, 50.246]
$C_{\text{air}}(t)$	烘房空气含湿量	kg/kg	[0.01963, 0.05025]
$R(t)$	药材半径	m	$0.0200 \rightarrow 0.01198$
$\dot{R}(t)$	半径收缩速率	$\text{m}\cdot\text{s}^{-1}$	≤ 0
ξ	Landau 归一化坐标 $r/R(t)$	—	[0, 1]
η	物质坐标 $(r/R(t))^2$	—	[0, 1]
v_s	固相径向速度	$\text{m}\cdot\text{s}^{-1}$	$r\dot{R}/R$
C_{th}	达标含水率阈值	kg/kg	0.15
t_{end}	烘干结束时间	h	见各问结果
α	热扩散率 $k/(\rho c_p)$	$\text{m}^2\cdot\text{s}^{-1}$	1.689×10^{-7}
N	径向控制体数	—	40
Δt	时间步长	s	1–4

五、热-质耦合运输的公共模型与离散方法

本章建立四问共用的连续模型、边界条件、环境驱动与数值方法。各问只在物性关系、终止条件和求解域三处与本章不同，后续章节只陈述这些差异，不重复整套推导。

5.1 系统边界与守恒关系

取药材横截面的径向一维区域 $r \in [0, R]$ 为研究对象， R 在前三问为常数、在问题四随时间变化。以单位轴向长度的圆环控制体为对象写守恒关系。热量方面，控制体内焓的变化率等于经内外两个圆柱面净流入的导热量；水分方面，干基含水率的变化率等于净流入的扩散水量。圆柱几何使两个圆环面的面积不等（内侧面积小、外侧面积大），这一差异在微分形式中表现为 $1/r$ 因子。按傅里叶导热定律与菲克扩散定律闭合通量^[4]，得到

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right), \quad (1)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right). \quad (2)$$

式 (1) 中 $T(r, t)$ 为药材内部温度, ρ 、 c_p 、 k 分别为密度、比热容与导热系数; 式 (2) 中 $C(r, t)$ 为干基含水率, D 为水分有效扩散系数。干基含水率以干物质质量为基准, 在干物质不迁移的前提下其守恒形式不含密度因子, 这与以湿基质量分数为变量的写法不同, 后者会额外引入密度的时间导数。两式的系数都可能依赖 C 与 T , 因此这是一组非线性抛物型方程, 其耦合强度完全由所用物性关系决定。

5.2 边界条件与初始条件

药材表面与热空气之间既有对流换热又有对流传质, 两者都属第三类边界: 表面通量与药材表面值和空气侧值之差成正比。表面处

$$-k(C) \frac{\partial T}{\partial r} \Big|_{r=R} = h [T(R, t) - T_{\text{air}}(t)], \quad -D(C, T) \frac{\partial C}{\partial r} \Big|_{r=R} = h_m [C(R, t) - C_{\text{air}}(t)], \quad (3)$$

其中 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 为题给的对流换热与对流传质系数。按第三类边界建模而不用第一类, 理由已在问题分析中给出: 两个毕奥数均为 $O(1)$, 表面既非温度受控也非阻力可忽略, 实测条件下的表面值必须由内外阻力共同决定^[5]。轴心处由几何对称性给出零通量条件

$$\frac{\partial T}{\partial r} \Big|_{r=0} = 0, \quad \frac{\partial C}{\partial r} \Big|_{r=0} = 0. \quad (4)$$

式 (4) 并非附加假设, 而是径向对称场在轴心可微的必然结果。初始时刻药材温度与含水率均匀

$$T(r, 0) = T_0 = 28^\circ \text{C}, \quad C(r, 0) = C_0 = 2.55 \text{ kg/kg}, \quad R(0) = R_0 = 2 \text{ cm}. \quad (5)$$

5.3 环境驱动的处理

式 (3) 中的 $T_{\text{air}}(t)$ 与 $C_{\text{air}}(t)$ 来自附件 1 的实测时序, 全部 241 个采样点参与插值, 不作两点线性化。相邻采样点之间按线性插值取值; 超出实测覆盖范围的时刻取末端平台值 $T_{\text{air}} = 50.165^\circ \text{C}$ 、 $C_{\text{air}} = 0.04986 \text{ kg/kg}$, 即假设 4 所述的处理。两条驱动曲线的实测段与平台段如何衔接, 见图 3。

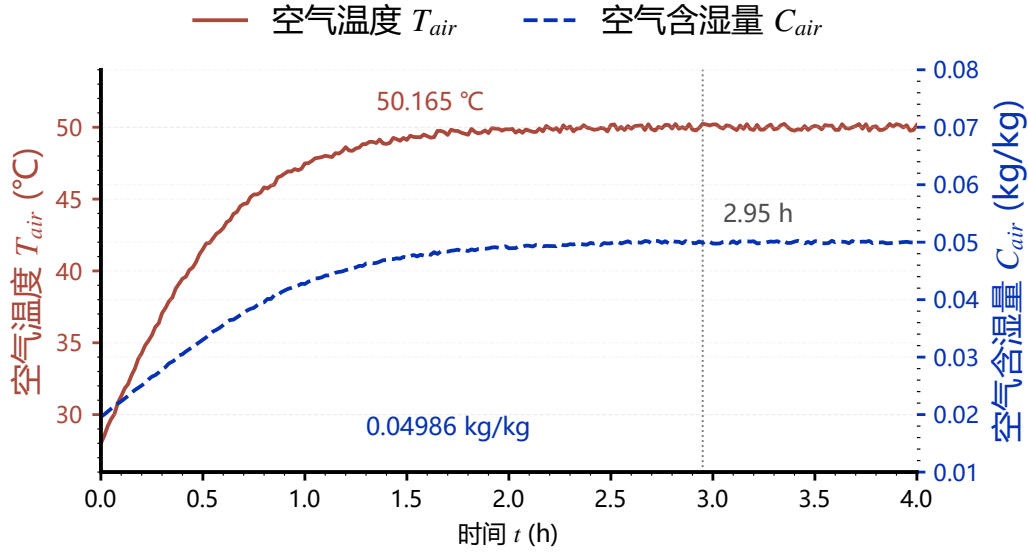


图3 附件1干燥室工况的双轴时程

空气温度自 28 °C 快速抬升, 1.37 h 已达 49 °C, 峰值 50.246 °C 出现在 2.95 h, 此后基本不再变化; 含湿量峰值为 0.05025 kg/kg。平台段的存在使问题二至问题四的长时程求解具有明确的边界驱动: 进入平台后药材内部的继续变化只由内部扩散阻力决定, 而不再由环境变化驱动。这一点在后文解释干燥速率为何全程单调衰减时会用到。

5.4 物性关系与由此决定的数学类型

三个附录给出三套物性关系, 分别用于问题一、问题二至三、问题四。记湿基水分分率 $w = C/(C + 1)$ 、绝对温度 $T_K = T + 273.15$ 。问题一采用常物性

$$\rho = 820, \quad c_p = 2600, \quad k = 0.36, \quad D(C) = 7 \times 10^{-9} e^{-0.89/C}, \quad (6)$$

问题二与问题三采用

$$\begin{aligned} \rho &= 650 + 128C, \quad c_p = 1450 + 2736w, \quad k = 0.21 + 0.38w, \\ D &= 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T_K}, \end{aligned} \quad (7)$$

问题四采用

$$\begin{aligned} \rho &= 760 + 90C, \quad c_p = 1850 + 2150w, \quad k = 0.12 + 0.20w, \\ D &= 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T_K}. \end{aligned} \quad (8)$$

三式中扩散系数的指数因子 $e^{-a/C}$ 表示含水率越低、水分迁移越困难, e^{-3850/T_K} 是典型的活化能形式, 表示升温促进扩散^[6]。指数中的温度必须取绝对温度: 若误用摄氏读数, 在 28 °C 附近该因子会相差十余个数量级。

式 (6) 与后两式的区别决定了问题的数学类型。常物性下式 (1) 的系数与 C 无关, 且 D 不含温度, 两个方程完全解耦: 温度场可以独立求解, 水分场只共享同一几何域与环境时间轴。式 (7) 与式 (8) 下两条依赖同时接通, 两场必须联立推进。图 4 比较后两套关系中扩散系数在实际工作区间上的分布。

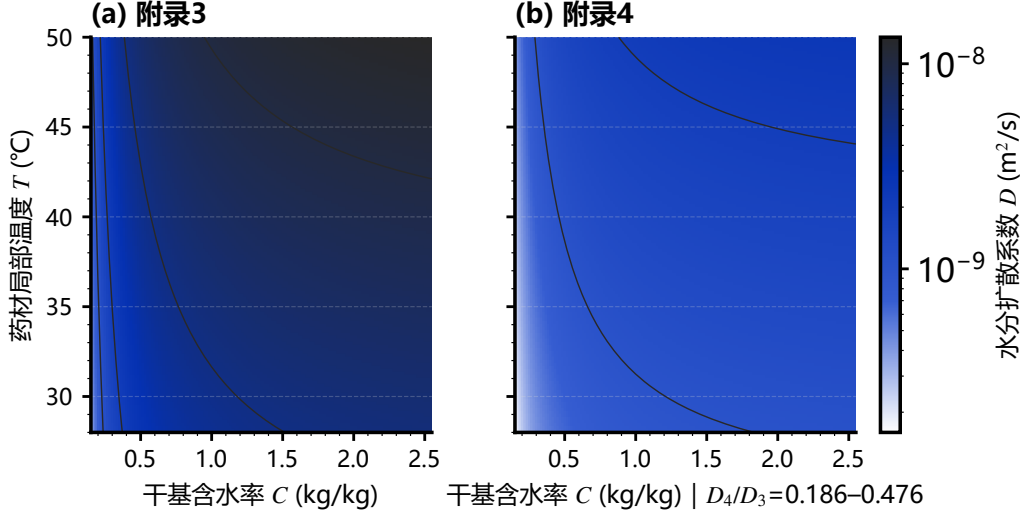


图 4 附录 3 与附录 4 的水分扩散系数分布对比

在 $C \in [0.15, 2.55]$ 、 $T \in [28, 50]$ °C 的工作区间上, 附录 3 的扩散系数覆盖 3.35×10^{-10} – 1.35×10^{-8} m²/s, 附录 4 覆盖 1.59×10^{-10} – 2.50×10^{-9} m²/s, 两者之比在 0.186–0.476 之间。也就是说, 附录 4 的水分迁移能力在整个工作区间上系统地低于附录 3, 最多低约五倍。这一量级差异是问题四结果解释的关键前提: 若只更换物性而不引入收缩, 烘干时长必然显著延长。两图共用对数色标, 故色阶的水平位移即为倍数关系, 可直接读出上述比值。

5.5 有限体积离散

式 (1) 与式 (2) 的系数随解变化, 不存在通用的解析解, 须数值求解。本文采用有限体积法而非有限差分: 有限体积法从控制体积分形式出发, 相邻控制体共用界面通量, 离散层面自动满足守恒, 圆柱几何的 $1/r$ 因子由界面面积 $2\pi r_{i\pm 1/2}$ 自然承担, 不需要额外处理奇点 [7,8]。这一性质在本题尤其重要, 因为烘干时长由累积失水量决定, 守恒误差会直接积累成时长误差。

把 $[0, R]$ 划分为 N 个控制体, 结点位于控制体中心, 界面位于 $r_{i\pm 1/2}$ 。轴心与表面处的控制体是半宽的, 三类控制体的单位长度体积分别为

$$V_0 = \pi \left(\frac{\Delta r}{2} \right)^2, \quad V_i = 2\pi r_i \Delta r \quad (1 \leq i \leq N-1), \quad V_N = \pi (R - r_{N-1/2}) (R + r_{N-1/2}). \quad (9)$$

式 (9) 的三种写法互不相同：轴心控制体是实心小圆盘，内部控制体含向心汇聚因子 r_i ，表面控制体是半宽圆环。若照搬直角坐标下的等体积划分，中心区域的体积会被系统性高估，从而低估中心的升温与失水速率。三类控制体在柱截面上的实际几何见图 5。

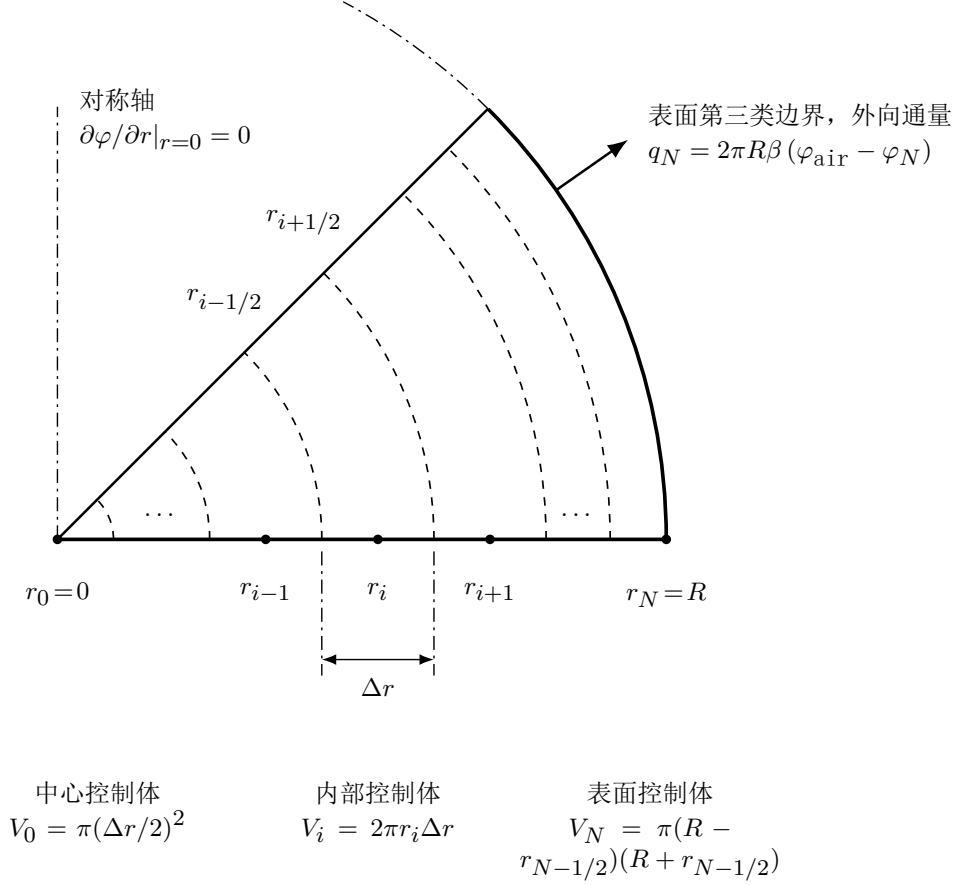


图 5 圆柱径向有限体积的控制体划分

图中虚线弧为控制体界面、实心圆点为结点。表面控制体的外向通量由第三类边界 $q_N = 2\pi R\beta(\varphi_{\text{air}} - \varphi_N)$ 给出，其中 φ 泛指 T 或 C 、 β 泛指 $h/(\rho c_p)$ 或 h_m ；轴心控制体则由式 (4) 的对称条件关闭，不需要虚拟结点。

5.6 时间推进、界面物性与耦合处理

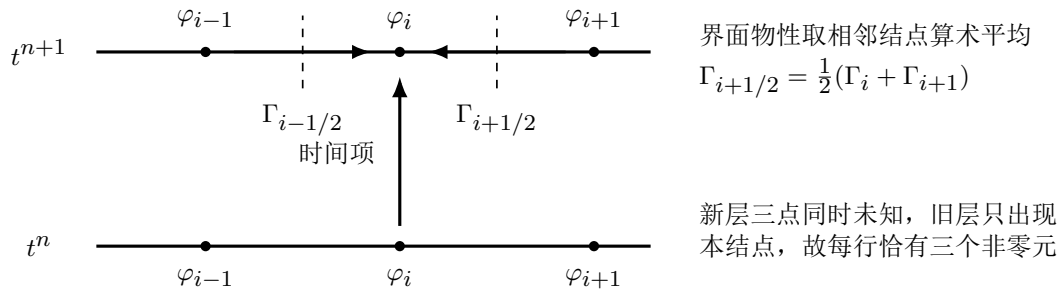
时间方向采用全隐式后向 Euler 格式。对通用标量 φ ，在控制体 i 上积分后得到

$$V_i \frac{\varphi_i^{n+1} - \varphi_i^n}{\Delta t} = 2\pi r_{i+1/2} \Gamma_{i+1/2} \frac{\varphi_{i+1}^{n+1} - \varphi_i^{n+1}}{\Delta r} - 2\pi r_{i-1/2} \Gamma_{i-1/2} \frac{\varphi_i^{n+1} - \varphi_{i-1}^{n+1}}{\Delta r}, \quad (10)$$

其中 Γ 为对应的输运系数。选择全隐式而非显式格式的理由是稳定性：显式格式受 $\Delta t \leq \Delta r^2/(2\alpha)$ 一类限制，本题热扩散率量级下这一上限约为 0.7 s，而烘干全程长达数

十小时，显式推进需要数十万至上百万步且步长不可放宽。全隐式格式无条件稳定，代价是每步需解一个线性系统。

式 (10) 的每一行只涉及 $i-1$ 、 i 、 $i+1$ 三个未知量，逐行装配即得三对角系统，可用 Thomas 消元在 $O(N)$ 内求解^[9]。界面输运系数取相邻结点的算术平均 $\Gamma_{i+1/2} = \frac{1}{2}(\Gamma_i + \Gamma_{i+1})$ 。调和平均更符合串联阻力，但在本题扩散系数空间差异可达两个数量级时，交付网格 $N = 40$ 尚未收敛；算术平均在 $N = 40$ 到 $N = 80$ 的时长漂移已低于 1%，故正式方案用算术平均。对照图 6 中的两时间层模板可以看清，新层的三个结点同时进入同一条方程，这正是全隐式格式无条件稳定的来源。



逐行装配后得三对角系统 $A\varphi = d$ ，
第 i 行的三个系数依次为 a_i, b_i, c_i ，
其中 $a_i, c_i \leq 0$ 且 $b_i \geq |a_i| + |c_i|$ ，对角占优

轴心控制体无左侧界面，该行只保留右侧通量；
表面控制体的右侧通量由第三类边界给出

图 6 全隐式有限体积的两时间层耦合模板

图中新时间层三个结点同时未知、旧层只有同一结点进入右端项，故系数矩阵每行恰有三个非零元。装配所得系数满足 $a_i, c_i \leq 0$ 且 $b_i \geq |a_i| + |c_i|$ ，即严格对角占优。这是该格式不产生非物理振荡的代数依据，也是后文含水率剖面始终保持单调有序的原因。

两个场的耦合按固定次序的算子分裂处理。一个时间步内依次完成：由旧层的 (C^n, T^n) 计算 ρ 、 c_p 、 k ；隐式求解温度得 T^{n+1} ；由 (C^n, T^{n+1}) 计算扩散系数；隐式求解含水率得 C^{n+1} ；对新含水率场施加下界以防非物理负值。这一次序不可交换：若在求解温度之前计算扩散系数，该系数就退化为由旧温度决定，温度对传质的促进作用被人为削弱。随后以 Picard 迭代复核——用新层场重算物性并重解，直至相邻两次迭代的场变化按绝对值低于 $10^{-6} \text{ }^\circ\text{C}$ （温度）与 10^{-8} kg/kg （含水率）——以消除分裂本身引入的滞后^[10]。上述次序与迭代关系的数据流向汇总于图 7。

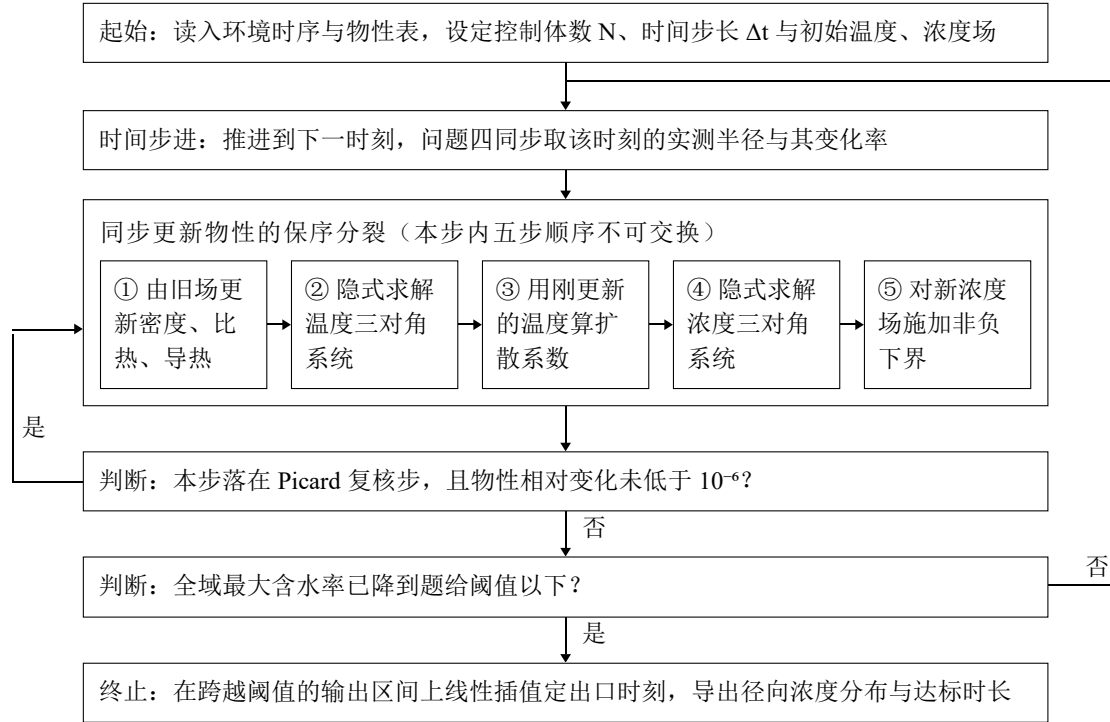


图 7 单个时间步内的求解流程

图中左侧回路为 Picard 复核、右侧回路为时间推进，两者分工不同。终止判据只在 Picard 收敛之后检查，避免用尚未收敛的中间场触发停机判定。四问的实际计算中 Picard 迭代最多 3 次即达到上述收敛标准，说明所选时间步长下分裂误差很小；下界截断在四问中均未被触发，即含水率场自始至终保持为正。

交付计算统一取 $N = 40$ 个控制体，时间步长按各问的时间跨度取 1–4 s：问题一至问题三取 1 s，问题四按时段取 2–4 s。这一组网格与步长的选取依据是收敛性检验，具体证据在数值可信度一章给出。

六、问题一：常物性下预热阶段的热-质输运模型与半解析校核

6.1 本问模型

本问沿用公共模型一章的式 (1)–(5)，物性取式 (6)。由于 ρ 、 c_p 、 k 均为常数，式 (1) 化为线性热传导方程

$$\frac{\partial T}{\partial t} = \alpha \frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial T}{\partial r} \right), \quad \alpha = \frac{k}{\rho c_p} = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}, \quad (11)$$

而水分方程保留非线性

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right), \quad D(C) = 7 \times 10^{-9} e^{-0.89/C}. \quad (12)$$

式 (11) 不含 C 、式 (12) 不含 T ，两者在题给物性下完全解耦，仅共享几何域和环境时间轴。这一解耦不是人为简化，而是附录 2 物性关系的直接后果，也正是本问可以引入独立数学基准的原因。初始含水率下 $D(C_0) = 4.94 \times 10^{-9} \text{ m}^2/\text{s}$ ，与 α 相差 34 倍，预示两个场在同一时间窗内的推进深度将有量级差别。

时间步长取 $\Delta t = 1 \text{ s}$ ，推进至 1800 s。

6.2 温度场与含水率场的结果

按题目要求的时刻与位置整理结果，见表 2 与表 3。

表 2 预热阶段药材内部温度 ($^{\circ}\text{C}$)

时间 (s) \ 距中心距离 (cm)	0	0.5	1	1.5	2
100	28.0001	28.0004	28.0043	28.0332	28.1803
300	28.0416	28.0643	28.1523	28.3690	28.8493
600	28.4550	28.5377	28.8054	29.3170	30.1659
900	29.3263	29.4601	29.8770	30.6173	31.7311
1200	30.5446	30.7115	31.2239	32.1138	33.4285
1500	31.9974	32.1883	32.7673	33.7472	35.1209
1800	33.5765	33.7731	34.3652	35.3630	36.7862

表 3 预热阶段药材内部水分浓度 (kg/kg)

时间 (s) \ 距中心距离 (cm)	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2555
300	2.5500	2.5500	2.5500	2.5490	2.0545
600	2.5500	2.5500	2.5500	2.5349	1.8789
900	2.5500	2.5500	2.5496	2.5044	1.7560
1200	2.5500	2.5500	2.5481	2.4646	1.6595
1500	2.5500	2.5499	2.5444	2.4208	1.5794
1800	2.5500	2.5497	2.5381	2.3757	1.5108

两表呈现的格局截然不同。温度在 1800 s 内各位置都有明显上升，中心由 28 °C 升至 33.5765 °C、表面升至 36.7862 °C，且同一时刻始终满足表面高于中心的单调序；含水率则只有表面有大幅下降（由 2.55 降至 1.5108 kg/kg）， $r = 1.5$ cm 处降幅 0.174 kg/kg，而中心处四位小数下仍显示为 2.5500 kg/kg。这一对照即式 (11) 与式 (12) 中输运系数相差 34 倍的直接体现。表面含水率在前 100 s 内即降至 2.2555 kg/kg，说明表面失水几乎从加热一开始就发生，但它并未立即传递到内部。

温度剖面与含水率剖面的空间形态见图 8。

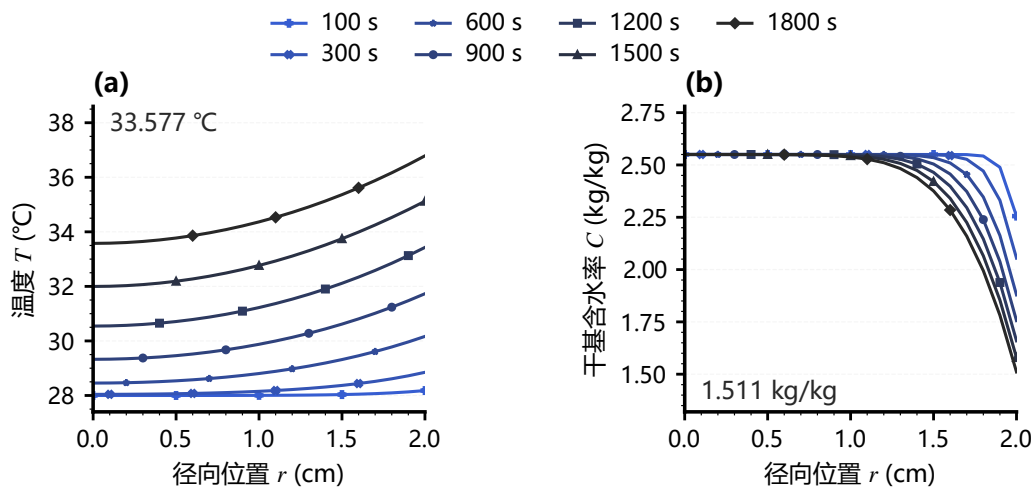


图 8 问题一温度与含水率的径向剖面

图中两个面板共用一条时刻色带。温度剖面自初值起整体抬升，各时刻都保持表面

高、中心低的单调形状，且梯度在整个半径上都不为零，说明热扰动已穿透全域。含水率剖面则始终贴合初值线，仅在 $r \gtrsim 1.5 \text{ cm}$ 的外层出现可见弯折，中心处相对初值仅降低 $1.0 \times 10^{-5} \text{ kg/kg}$ 。若把两个面板的可见变化区厚度相比，水分的影响深度不足半径的四分之一。

中心与表面两个测点的时程对照进一步给出量化的层级关系，见图 9。

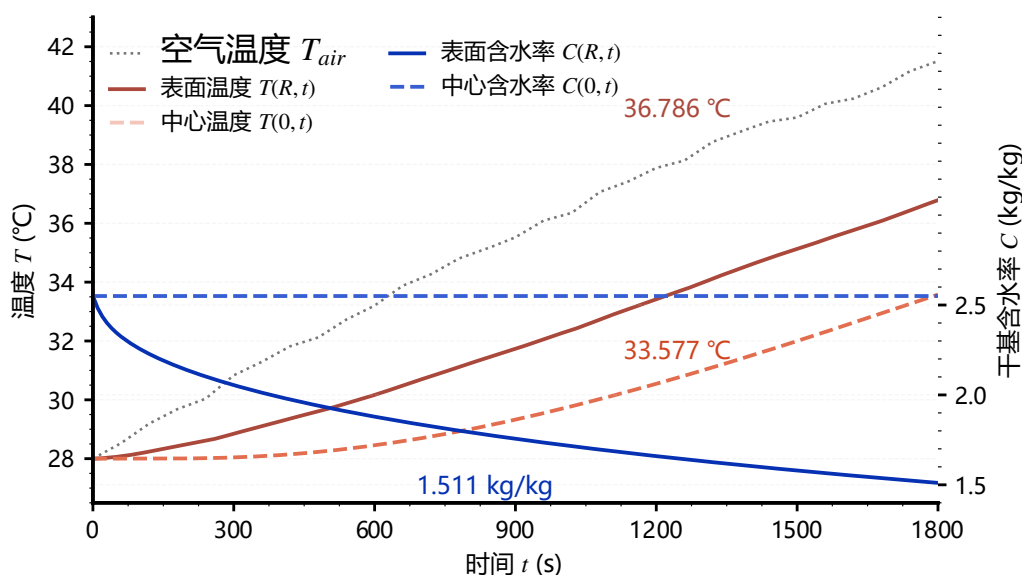


图 9 问题一中心与表面测点的时程对照

左轴三条温度曲线全程保持空气高于表面、表面高于中心的次序，这与第三类边界的方向一致：热流始终由空气流向药材，且表面必须低于空气温度才能维持正向换热。若表面温度追平空气温度，则说明表面阻力被忽略，与 $Bi_h = 1.389$ 的估计矛盾。右轴的中心含水率曲线在 1800 s 内近乎水平（终值 2.54999 kg/kg ），可以读出中心水分在预热阶段基本未被动用。

两个场的渗透深度差异在时空分布上更为直观，见图 10。

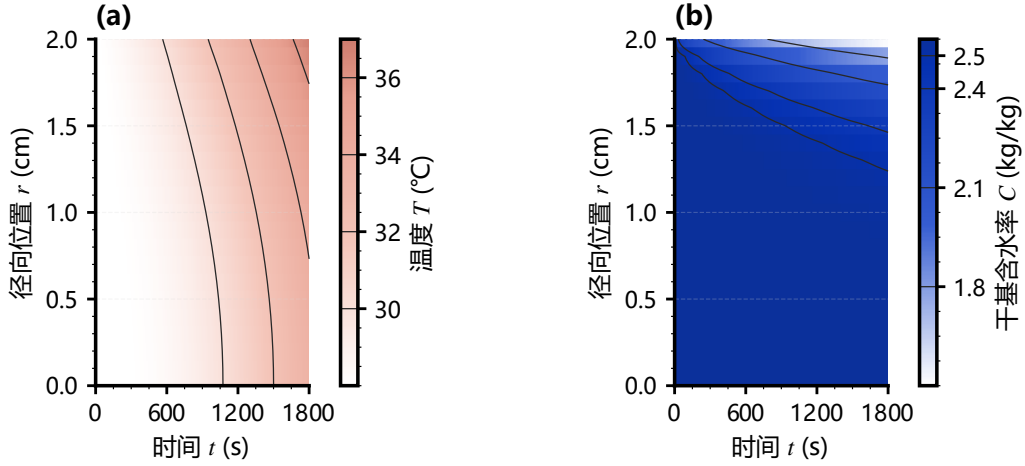


图 10 问题一温度场与含水率场的时空分布

温度场的色阶变化在 1800 s 内已铺满整个纵向坐标，含水率场的可见变化则被压在图幅顶部一条窄带内： $r = 1.5 \text{ cm}$ 处降幅 0.174 kg/kg ，到 $r = 0.7 \text{ cm}$ 处已不足 $1.5 \times 10^{-3} \text{ kg/kg}$ ，两个位置相距仅 0.8 cm 而降幅差两个数量级。由此可以给出预热阶段的物理图景：热量先行铺满全域并把药材整体加热，水分迁移则从表面开始逐层向内推进，中心水分要到进入恒温干燥阶段之后才会明显下降。这一时标分离也解释了为何题目把预热阶段与整个烘干过程分开设问——两个阶段的控制机制并不相同。

6.3 半解析级数解的独立校核

式 (11) 是常系数线性方程，配合第三类边界与均匀初值，可用分离变量法求解。对阶跃型环境温度，特征值 λ_n 由

$$\lambda_n J_1(\lambda_n) = Bi_h J_0(\lambda_n), \quad Bi_h = \frac{hR}{k} = 1.389 \quad (13)$$

确定， J_0 、 J_1 为零阶与一阶第一类 Bessel 函数，该本征关系是无限长圆柱对流换热问题的经典结果^[1]。实际环境温度并非阶跃而是随时间变化，故按 Duhamel 原理把 $T_{\text{air}}(t)$ 的分段线性时程分解为一系列阶跃响应的叠加，得到

$$T(r, t) = T_{\text{air}}(0) + \int_0^t \frac{dT_{\text{air}}(\tau)}{d\tau} \Phi(r, t - \tau) d\tau, \quad \Phi(r, s) = 1 - \sum_{n=1}^{\infty} A_n J_0\left(\lambda_n \frac{r}{R}\right) e^{-\alpha \lambda_n^2 s / R^2}, \quad (14)$$

其中 A_n 为由初值与正交性确定的展开系数， Φ 为单位阶跃响应。式 (14) 的价值不在于它比数值解更精确，而在于它与公共模型一章的离散格式完全无关：一条路径来自 Bessel 函数正交展开，另一条来自控制体通量平衡，两者若吻合，则离散实现中的系数

装配、边界处理与时间推进都得到了独立确认。本文取前 30 个特征根，并对分段线性的环境时程逐段精确叠加。校核结果与网格、步长收敛证据一并给出，见图 11。

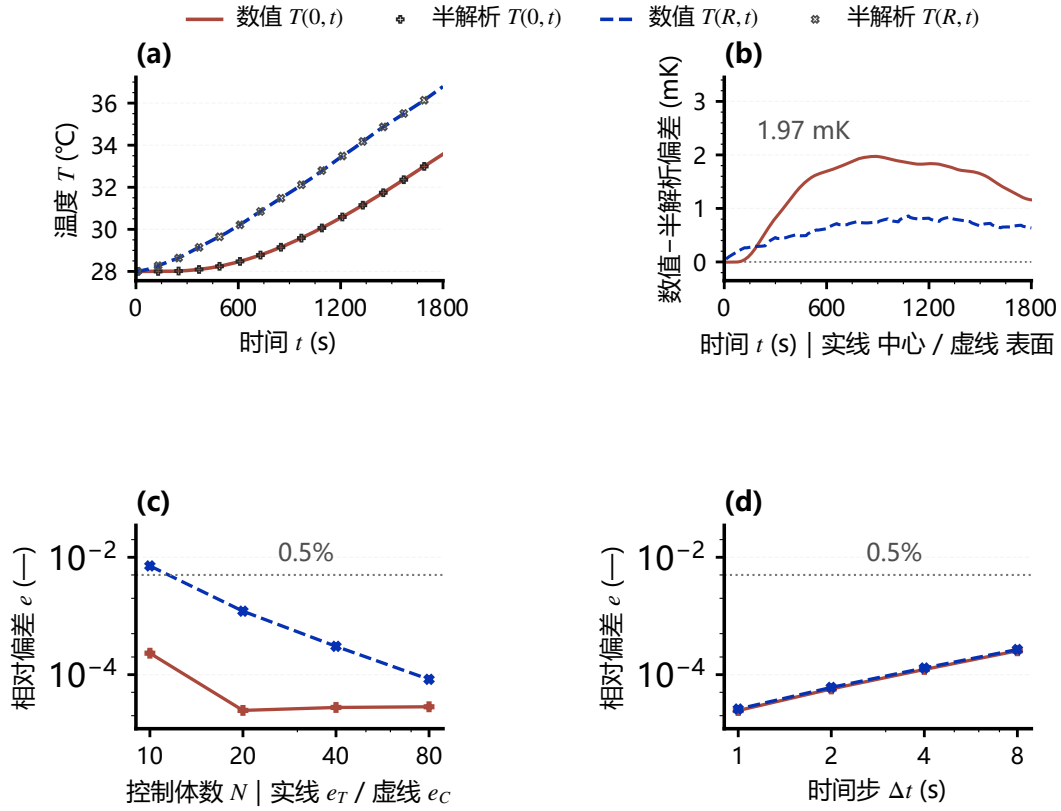


图 11 问题一数值解的四项校核证据

面板 (a) 中数值解与级数解的曲线逐点重合，面板 (b) 给出两者之差：中心与表面两条测点曲线在 1800 s 全程内的最大偏差为 1.97 mK，相对于表面 8.786 °C 的温升约为 0.022%；若只看末时刻沿半径的偏差，最大值为 1.16 mK 即 0.0012 °C，约为温升的 0.013%。两个量级都与时间离散的截断误差相当，说明偏差来自 Δt 而非模型实现有误。面板 (c)(d) 为网格与时间步的加密收敛：交付网格 $N = 40$ 相对 $N = 160$ 参照解的偏差为 $e_T = 2.76 \times 10^{-5}$ 、 $e_C = 3.04 \times 10^{-4}$ ， $\Delta t = 1$ s 时 $e_T = 2.45 \times 10^{-5}$ ，均远低于图中标出的 0.5% 判据线。

需要说明的是，这一校核只覆盖温度场，因为水分方程含非线性扩散系数，不存在同类级数解。水分场的可信度依靠守恒残差与网格收敛证据支撑，详见数值可信度一章。本问的质量收支相对残差为 7.41×10^{-16} ，已接近双精度浮点的机器精度，即离散格式在本问未引入可测的守恒破坏。

6.4 完整场数据的交付

按题目要求，1800 s 内每隔 1 s、距中心每隔 0.1 cm 的温度与含水率完整结果以双表形式保存于结果文件中，数值保留四位小数。表格中的位置坐标由 $N = 40$ 交付网格的结点值按线性插值给出至题目要求的 0.1 cm 间隔。

七、问题二：变物性双向耦合的全过程烘干模型

7.1 相对问题一的改动

本问保留公共模型一章的控制方程、边界条件与初始条件，也保留问题一的固定求解域 $R = 2\text{ cm}$ ，只把物性关系由式 (6) 换成式 (7)。这一处改动使式 (1) 与式 (2) 由完全解耦变为双向耦合： ρ 、 c_p 、 k 随含水率变化，扩散系数同时随含水率与温度变化。改动虽只在系数，数学类型却由「线性温度场 + 非线性水分场」变成「两个互相牵动的非线性方程」，求解上必须使用公共模型一章所述的算子分裂与迭代复核，不能沿用问题一的先算温度再算水分的做法。

耦合还改变了物性本身的量程。将式 (7) 代入本问实际出现的含水率区间，密度、比热容与导热系数都随失水而下降，同时扩散系数受两个方向的相反影响：含水率下降使其减小、温度上升使其增大。哪一方占优决定干燥的快慢，这一点在结果讨论中用实际计算的物性联动来回答。

时间步长取 $\Delta t = 1\text{ s}$ ，报表窗口为题目要求的 0–3 h。需要强调的是 3 h 只是报表要求，并非过程终止：同一次求解一直推进到含水率达标为止，其终止时刻由下一章的判断给出。

7.2 前 3 小时的温度与含水率分布

按题目要求的时刻与位置整理，见表 4 与表 5。

表 4 全过程模型下药材内部温度 (°C)

时间 (h) \ 距中心距离 (cm)	0	0.5	1	1.5	2
0.5	32.1908	32.3836	32.9673	33.9620	35.4141
1.0	40.3817	40.5536	41.0606	41.8785	42.9980
1.5	45.8462	45.9343	46.1925	46.6047	47.1398
2.0	48.4498	48.4876	48.5980	48.7732	49.0031
2.5	49.4667	49.4789	49.5134	49.5653	49.6609
3.0	49.8495	49.8553	49.8745	49.9101	49.9664

表 5 全过程模型下药材内部水分浓度 (kg/kg)

时间 (h) \ 距中心距离 (cm)	0	0.5	1	1.5	2
0.5	2.5499	2.5488	2.5254	2.3260	1.6486
1.0	2.5255	2.4946	2.3578	2.0231	1.4710
1.5	2.3859	2.3255	2.1344	1.8020	1.3475
2.0	2.1708	2.1084	1.9236	1.6259	1.2310
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7166	1.5702	1.3332	1.0080

温度方面, 中心由 0.5 h 的 32.1908 °C 升至 3 h 的 49.8495 °C, 已逼近烘房平台温度 50.165 °C; 同一时刻中心与表面之差由 3.22 °C 收窄到 0.12 °C。也就是说药材在约 3 h 后基本进入等温状态, 此后温度不再是限制因素。含水率方面, 3 h 时中心仍有 1.7662 kg/kg, 距 0.15 kg/kg 的目标相去甚远, 而表面已降至 1.0080 kg/kg。两者对比给出一个重要判断: 干燥的限制环节不是供热, 而是水分从芯部迁移到表面的内部阻力。这也解释了题目所述烘干需持续两三天的量级。

与问题一相比, 中心含水率在 3 h 内下降了 0.78 kg/kg, 而问题一在 0.5 h 内几乎未动。图 12 给出本问剖面的演化形态。

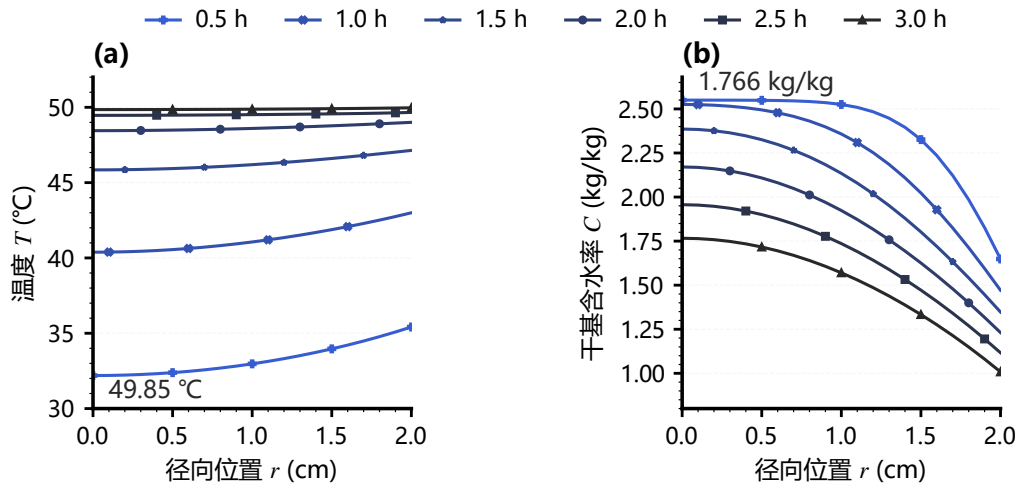


图 12 问题二温度与含水率的径向剖面

温度剖面在 1.5 h 之后已趋于平坦，各位置之差不足 1°C ；含水率剖面则持续下移并逐渐由「仅表层弯折」变为「整体倾斜」，说明水分下降已推进到可观深度而非局限于表层。两个面板的时间尺度差异再次印证前段的判断：温度场先达到准稳态，水分场在此之后才是主角。

把两个场在整个时间窗上铺开，可与问题一作同一标尺的对照，见图 13。

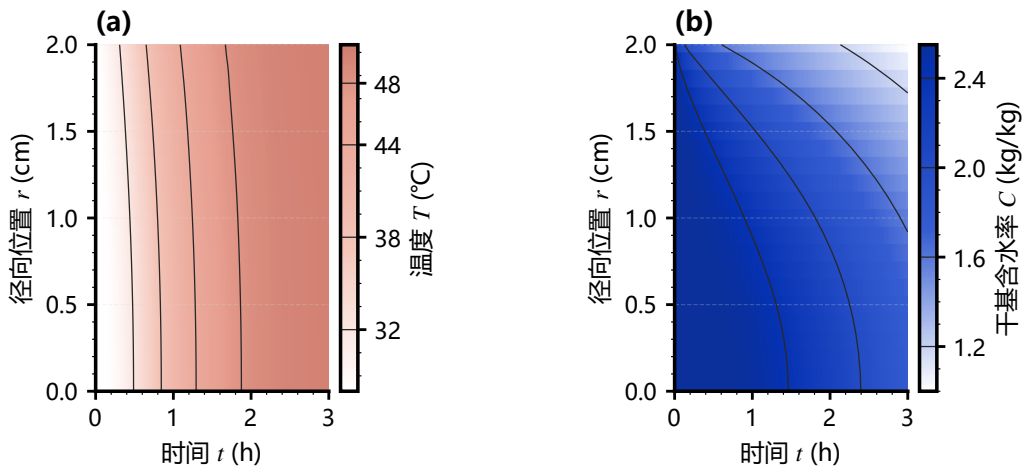


图 13 问题二温度场与含水率场的时空分布

温度场在约 1 h 内基本追平空气平台温度，其后色阶不再变化；含水率场呈现一条由表面向中心推进的干燥前沿，前沿位置随时间内移。与图 10 相比，含水率的可见变化区已由「表层窄带」扩展到「全域」，量级的改变来自物性关系的更换而非时间窗的延长。这一点需要在同一时刻上比较才能确认，见图 14。

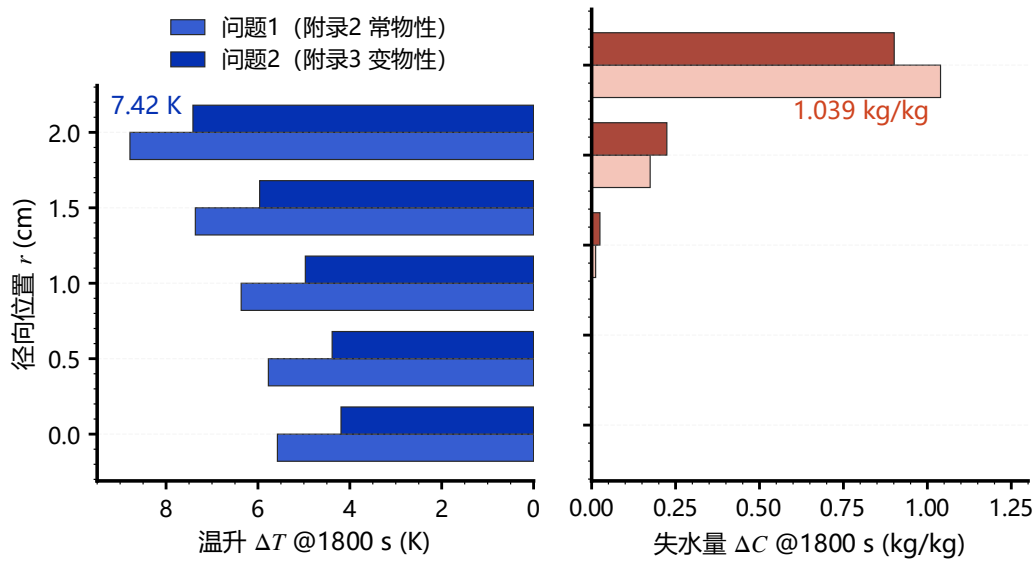


图 14 问题一与问题二在 $t = 1800$ s 的增量对比

图中取同一时刻 $t = 1800$ s 背靠背比较两问的温升与含水率降幅。温升的差异始终在数摄氏度量级,而含水率降幅在中心处相差近一个量级 (1.0×10^{-5} 对 7.9×10^{-5} kg/kg); 表面处则出现符号反转: 问题一的表面降幅 1.039 kg/kg 反而大于问题二的 0.901 kg/kg。这一「内部放大、表面反转」的格局说明物性关系的更换主要改写了水分方程的内部输运能力,而不是表面的传质条件——表面通量由 h_m 与表面浓度差决定,两问的 h_m 相同,问题二表面失水略少恰恰因为内部补水更快、表面浓度维持得更高。

7.3 耦合关系的实际表现

前文的判断需要落到物性本身。图 15 把全程节点场按含水率分箱,给出四项物性的联动。

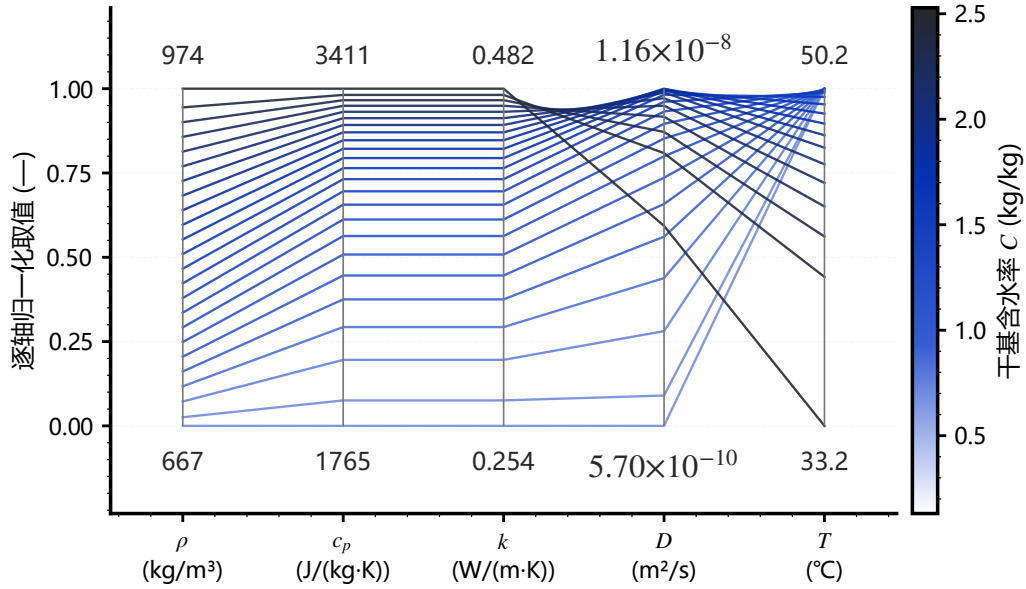


图 15 附录 3 四项物性随含水率的联动

随含水率由 2.53 降到 0.13 kg/kg，密度由 973.7 降到 666.8 kg/m³、比热容由 3411 降到 1765 J/(kg·K)、导热系数由 0.482 降到 0.254 W/(m·K)，三者同向下降。扩散系数的最低值 $5.70 \times 10^{-10} \text{ m}^2/\text{s}$ 出现在最低含水率箱，说明在含水率与温度的两个相反影响中，含水率的影响占优：即使后期温度已升至平台值，扩散系数仍随失水而下降。这正是干燥后期速率持续减慢、达标时间被显著拉长的机制来源。热风干燥试验中由失水曲线反演有效扩散系数与活化能，所得量级与本文所用经验式一致^[12]。

耦合的另一侧面是求解轨迹在状态平面上的分布，见图 16。

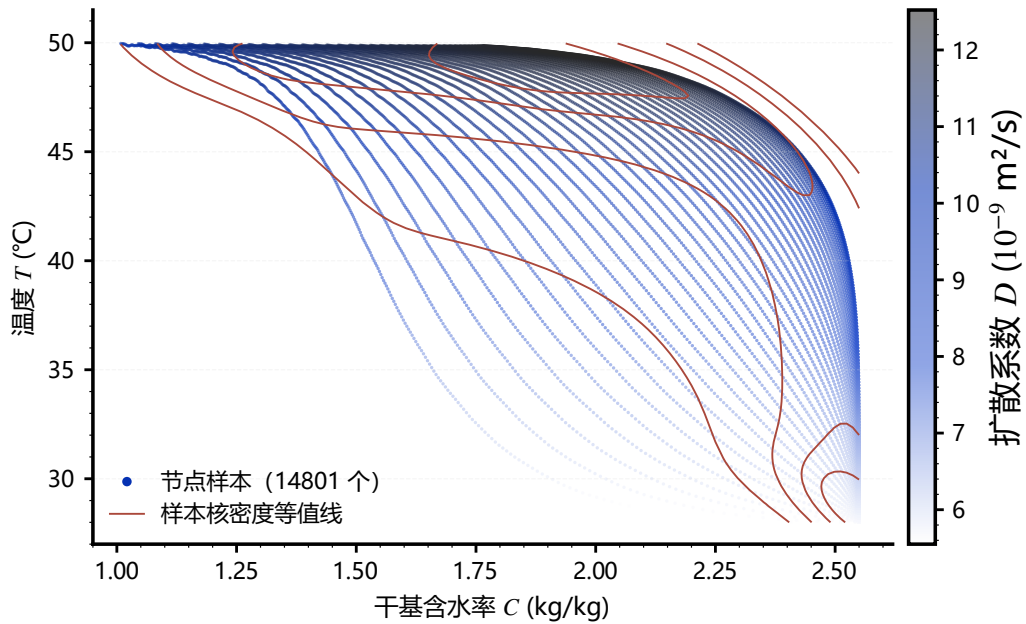


图 16 问题二时空节点在含水率-温度平面上的分布

14801 个节点样本聚成两条带：升温初期的高含水率带，以及准平台温度下的降湿带。两带交界处正是扩散系数变化最剧烈的区域，也是算子分裂次序最敏感的阶段。若此处用旧层温度计算扩散系数，误差会直接进入后续全部时程。实际计算中 Picard 迭代最多 3 次即收敛，质量收支相对残差 1.55×10^{-14} ，说明该阶段的耦合已被正确处理。

按题目要求，全过程每隔 1 s、距中心每隔 0.1 cm 的温度与含水率完整结果保存于结果文件，共 204143 个时间行，覆盖从初始时刻到含水率达标的整个过程。

八、问题三：逐点达标判据下的烘干时长确定

8.1 相对问题二的改动：达标判据的数学定义

本问的模型与问题二完全相同：同样的式 (1)–(5)、同样的式 (7) 物性、同样的固定域 $R = 2 \text{ cm}$ 。唯一新增的是一个终止条件。新增内容虽只有一式，但它的定义方式直接决定答案，因此需要单独论证。

干燥过程中含水率沿半径始终不均匀。由表 5 可见同一时刻中心最高、表面最低，两者可相差数倍。要保证「药材内部含水率降到 0.15 kg/kg 以下」，必须要求最难干的位置也达标，故把达标时刻定义为逐点最大含水率首次低于阈值的时刻

$$t_{\text{end}} = \min \left\{ t > 0 \mid \max_{0 \leq r \leq R} C(r, t) < C_{\text{th}} \right\}, \quad C_{\text{th}} = 0.15 \text{ kg/kg}. \quad (15)$$

与式 (15) 相对的是以体平均含水率判定的做法

$$\bar{C}(t) = \frac{\int_0^R C(r, t) 2\pi r \, dr}{\pi R^2}, \quad (16)$$

式 (16) 把径向分布压成一个数，必然在芯部尚未达标时就先跨过阈值，从而给出偏短的时长。本文只把 $\bar{C}$ 作为对照量输出，不用作判据。由式 (4) 的对称条件与解的单调性，含水率最大值总出现在轴心，故式 (15) 的控制点即 $r = 0$ 。

数值实现上， $C_{\text{max}}(t)$ 只在离散输出步上可得，而阈值一般落在两个输出步之间。为了不使时长精度受输出间隔限制，在跨越阈值的相邻两步之间对 C_{max} 作线性插值

$$t_{\text{end}} \approx t^n + \Delta t \frac{C_{\text{max}}(t^n) - C_{\text{th}}}{C_{\text{max}}(t^n) - C_{\text{max}}(t^{n+1})}, \quad (17)$$

式 (17) 要求 $C_{\text{max}}(t^n) \geq C_{\text{th}} > C_{\text{max}}(t^{n+1})$ ，即定位的是首次跨越而非任意一次低于阈值的时刻。

8.2 烘干时长与含水率分布

以 $\Delta t = 1 \text{ s}$ 推进求解，得到烘干时长

$$t_{\text{end}} = 56.706 \text{ h} \approx 2.36 \text{ d}, \quad (18)$$

该时刻的逐点最大含水率为 0.150000 kg/kg ，恰好落在阈值上，控制点位于轴心。式 (18) 与题目所述「烘干过程一般持续 2–3 天」的量级一致。各时刻的含水率分布见表 6。

表 6 烘干过程中药材内部水分浓度 (kg/kg)

时间 (h) \ 距中心距离 (cm)	0	0.5	1	1.5	2
6	1.0156	0.9869	0.9006	0.7546	0.5334
12	0.4548	0.4419	0.4020	0.3290	0.1641
18	0.2979	0.2906	0.2678	0.2244	0.0847
24	0.2369	0.2318	0.2158	0.1844	0.0665
30	0.2049	0.2009	0.1882	0.1628	0.0597
36	0.1850	0.1816	0.1708	0.1491	0.0565
42	0.1712	0.1682	0.1587	0.1395	0.0547
48	0.1610	0.1583	0.1497	0.1323	0.0535
54	0.1530	0.1506	0.1427	0.1266	0.0528
烘干结束时间 56.706 h	0.1500	0.1476	0.1400	0.1244	0.0525

表中每一行都保持中心含水率最高、表面最低的次序，与式 (15) 取轴心为控制点相符。前 12 h 中心含水率由 2.55 降至 0.4548 kg/kg ，减少了八成；而从 12 h 到 56.706 h 的四十余小时只再减少 0.30 kg/kg 。也就是说约五分之四的水分在前四分之一的时间内被除去，剩下的时间几乎全部花在把中心含水率从 0.45 压到 0.15 kg/kg 上。末行还显示表面含水率已降至 0.0525 kg/kg ，远低于阈值，中心与表面相差近三倍——这正是不能用平均量判定的直接证据。

判据本身的差别有多大，可由同一次求解的两条曲线读出，见图 17。

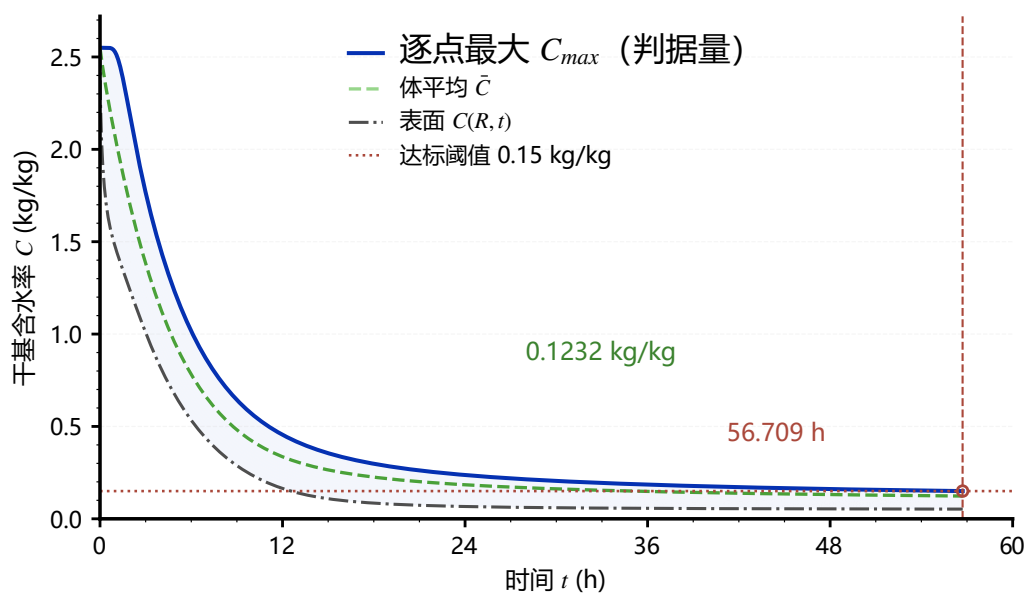


图 17 问题三干燥曲线与达标时刻定位

图中渐变带表示同一时刻药材内部由表面到芯部的含水率分布范围，带宽即内部不均匀度。 $C_{\max}$ 触及阈值时体平均含水率只有 0.1232 kg/kg，已低于阈值约 18%；若改用式 (16) 判定，会在芯部含水率尚有 0.18 kg/kg 左右时就宣布干燥完成。图中标注的时刻取自效应隔离基线情景（时间步长较粗），为 56.709 h，与式 (18) 的交付值相差 0.003 h，即 0.005%——两个不同时间步长下的达标时刻一致到三位有效数字，这本身也构成时间步无关性的一项旁证。

剖面形态随时间的演化见图 18。

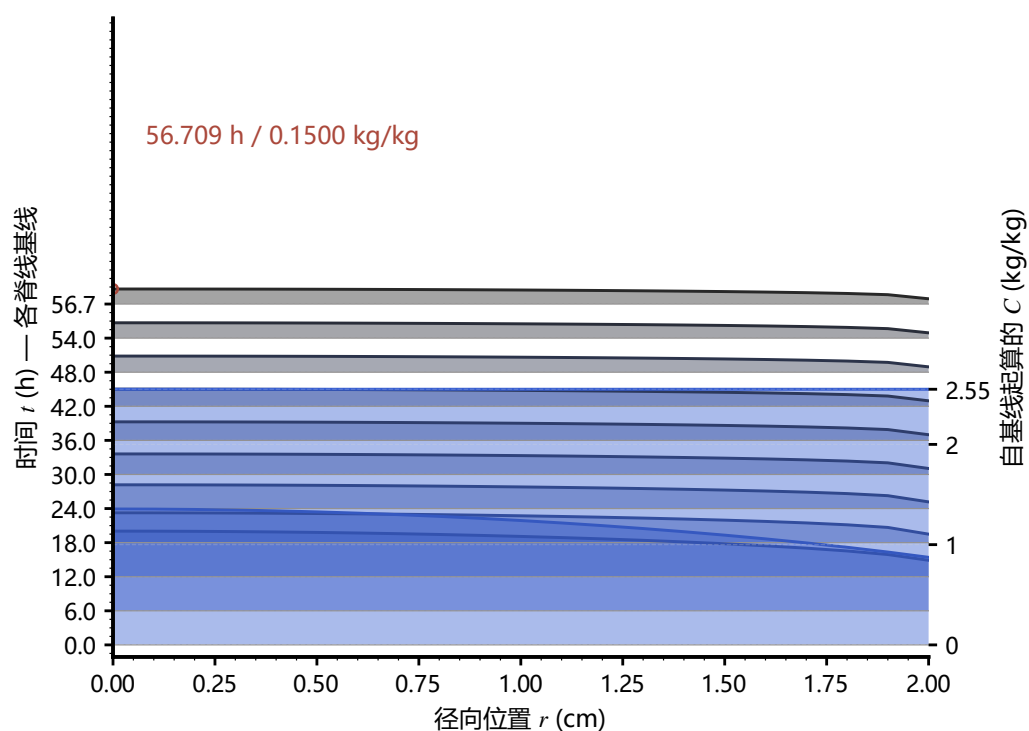


图 18 问题三含水率剖面的时序脊线

各条脊线按 6 h 取样、基线逐条下移。剖面形态由初期的近平坦逐步转为中心高、表面低的抛物型，末条脊线的峰值 0.1500 kg/kg 即式 (15) 的临界状态。脊线间距在后期明显收窄，与表 6 中后四十小时降幅有限的读数一致。

8.3 干燥速率的阶段特征

干燥文献常把过程分为升速段、恒速段与降速段，其中恒速段由表面蒸发控制、降速段由内部扩散控制。本题属于哪一类需要用实际速率曲线回答，见图 19。

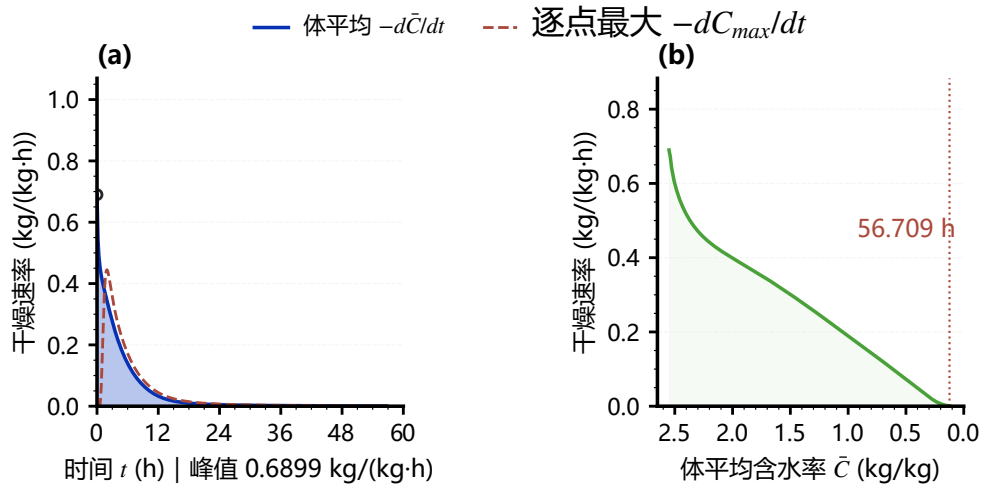


图 19 问题三干燥速率的两种读法

面板 (a) 显示速率峰值 $0.6899 \text{ kg}/(\text{kg}\cdot\text{h})$ 出现在 $t = 0$ 而非某个中间时刻，此后单调衰减，说明本题不存在升速段。面板 (b) 按 Krischer 的方式把速率对体平均含水率作图，横轴反向以顺应干燥进程；图中没有出现水平段，即不存在恒速段。两个面板合起来给出结论：本题自始至终由内部扩散控制，表面蒸发从未成为限制环节。这与式 (7) 中扩散系数强烈依赖含水率的形式一致，也与前文「限制环节是内部阻力而非供热」的判断相互印证 [6]。

按题目要求，全过程每隔 60 s 、距中心每隔 0.1 cm 的含水率完整结果保存于结果文件，覆盖从初始时刻到 t_{end} 的整个过程。

九、问题四：收缩移动边界的物质坐标模型

9.1 相对问题三的两处改动

本问相对问题三改变了两件事：物性关系由式 (7) 换成式 (8)，求解域由固定半径换成随时间收缩的 $r \in [0, R(t)]$ 。达标判据式 (15) 与插值定位式 (17) 保持不变，边界条件式 (3) 的形式不变但作用位置随 $R(t)$ 移动。两处改动中，物性只是系数替换，域的变化才是本质困难。

干基含水率 C 以干物质质量为基准。药材收缩时，固体骨架携带结合水一起向内运动，因此 C 首先是物质点上的场，而不是实验室固定半径上的场。若仍在随时间缩小的物理网格上离散，每步都要重新生成结点并把旧解插值过来，插值既破坏离散守恒，又使误差来源难以区分。坐标变换的目的，就是把物质点变成固定的计算坐标。

9.2 物质坐标与归一化坐标

定义归一化物质坐标

$$\eta = (r/R(t))^2 \in [0, 1]. \quad (19)$$

均匀收缩假设下，固相速度为仿射场

$$v_s(r, t) = r \dot{R}(t)/R(t), \quad (20)$$

它是使 η 对每个干物质质点保持不变的唯一径向速度，也使全局干物质 $S = \bar{\rho}_d(t)R(t)^2/2$ 在 $\bar{\rho}_d \propto R^{-2}$ 时守恒。在定 η 的物质导数下，前三问同一套经验扩散方程变为

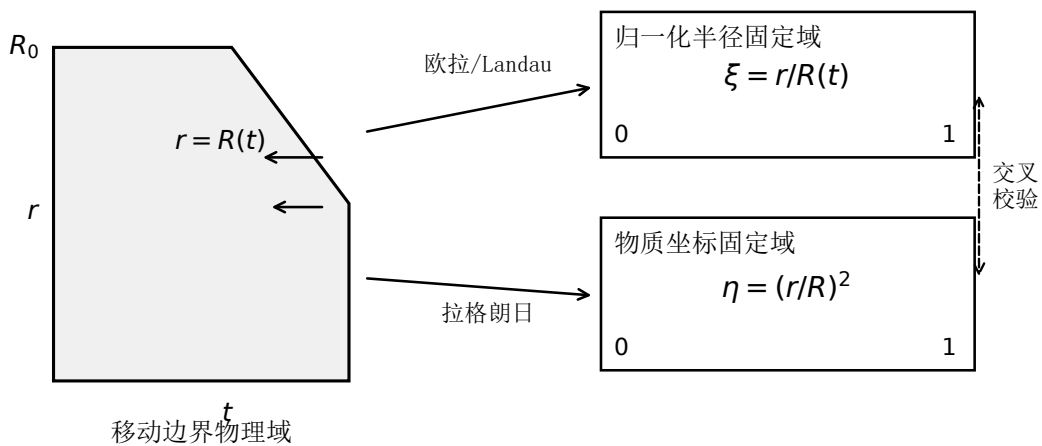
$$\frac{\partial C}{\partial t} = \frac{4}{R(t)^2} \frac{\partial}{\partial \eta} \left(\eta D(C, T) \frac{\partial C}{\partial \eta} \right), \quad (21)$$

能量方程同理，只把 D 换成 k 、左端乘 ρc_p 。收缩进入系数 $4/R^2$ ：半径缩小使同一无量纲梯度对应更陡的物理梯度，扩散路径变短。方程中不再出现表观对流项，也无需再引入开关参数 χ 。表面边界条件化为

$$-\frac{2D}{R} \frac{\partial C}{\partial \eta} \Big|_{\eta=1} = h_m [C(1, t) - C_{\text{air}}], \quad (22)$$

温度侧把 D, h_m 换成 k, h 。

为交叉验证，把同一经验方程写在实验室参考系后再作 Landau 变换 $\xi = r/R(t)$ ，定 ξ 求导与定 r 求导相差 $-\xi \dot{R}/R \partial_\xi C$ ，计算域上多出一项表观对流。该项来自坐标变形而不是真实流动；一阶迎风用以保证对角占优。两种坐标的关系见图 20。本文以物质坐标 (21) 为交付实现，以 Eulerian Landau（保留对流项）为交叉验证。



主用：定 η 物质导数，收缩进入 $4/R^2$ ，无表观对流。交叉验证：Landau 变换后出现 $(\xi \dot{R}/R) \partial_\xi C$ 。

图 20 收缩物理域经物质坐标与 Landau 坐标映为固定计算域

9.3 半径时程

式 (21) 需要 $R(t)$ 。附件 2 的 145 个半径实测点全部参与插值：先按假设 5 作累积最小值以保证单调不增，再用保形分段三次插值构造连续曲线^[13]。半径由 2.000 cm 降至 1.198 cm，末态体积约为初始的 36%。单调化前后的最大偏差为 2.2×10^{-16} cm，说明实测本身已经单调。 $R(t)$ 与 $\dot{R}(t)$ 见图 21。

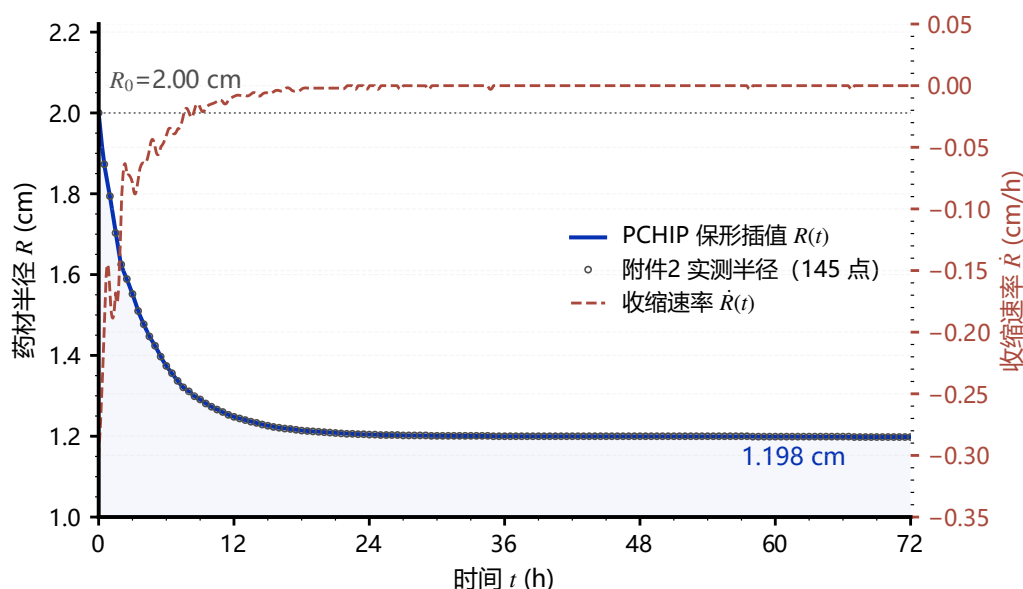


图 21 附件 2 收缩半径的实测散点、插值与收缩速率

9.4 烘干时长与含水率分布

物质坐标下求得烘干时长

$$t_{\text{end}} = 50.780 \text{ h} \approx 2.12 \text{ d}. \quad (23)$$

该时刻逐点最大含水率为 0.1500 kg/kg，末态半径 $R(t_{\text{end}}) = 1.20 \text{ cm}$ ，落在附件 2 数据覆盖之内，不涉及外推。各时刻含水率分布见表 7。

表 7 考虑收缩时药材内部水分浓度 (kg/kg)

时间 (h) \ 距中心距离 (cm)	0	0.5	1	1.5	2	药材表面
6	1.7165	1.5346	1.0212	—	—	0.4215
12	0.7343	0.6516	0.4068	—	—	0.1666
18	0.4067	0.3668	0.2387	—	—	0.0889
24	0.2838	0.2598	0.1781	—	—	0.0671
30	0.2253	0.2084	0.1485	—	—	0.0593
36	0.1920	0.1789	0.1310	—	—	0.0558
42	0.1705	0.1597	0.1194	—	—	0.0539
48	0.1555	0.1462	0.1111	—	—	0.0528
烘干结束时间 50.780 h	0.1500	0.1412	0.1079	—	—	0.0525

$t = 6$ h 时半径已缩至约 1.37 cm, 1.5 cm 与 2 cm 两列自第一个输出时刻起即为空, 这是几何事实而不是缺算。同一套附录 4 物性、但半径冻结在 2 cm 时时长为 128.906 h, 引入收缩后缩短 60.6%: 收缩是主效应, 不是小扰动。收缩域内的场演化见图 22。

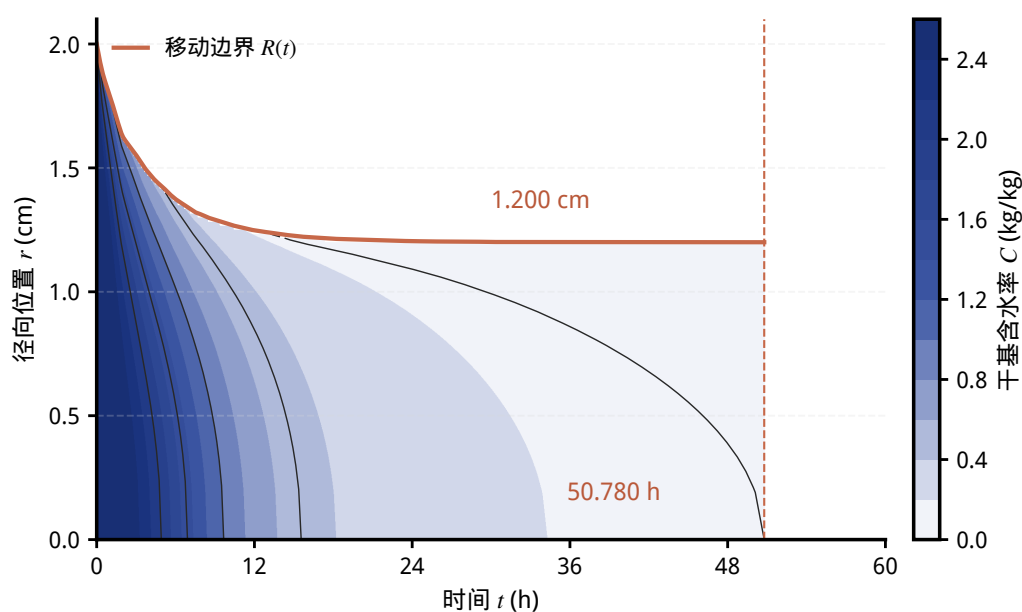


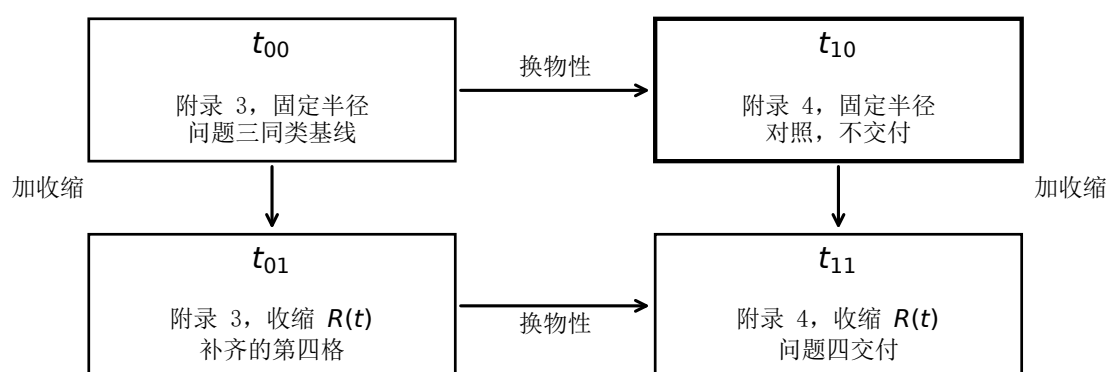
图 22 问题四收缩域内含水率的时空分布与移动边界

Eulerian Landau 交叉验证给出 52.190 h, 与物质坐标结果相对相差 2.78%。相差来

自是否让含水率随骨架一起运动，而不是来自网格重划。本文将 $[50.78, 52.19]$ h 作为坐标表述带来的模型区间，交付值取物质坐标的 50.780 h。

9.5 物性与收缩的 2×2 因子设计

式 (23) 比问题三的 56.706 h 短 5.926 h。但本问同时改了物性与求解域，把净差说成「收缩使干燥加快」是混淆归因。在完全相同的离散、环境与判据下补齐四种组合，见图 23 与表 8。



交互项 $I = t_{11} - t_{10} - t_{01} + t_{00}$ 。两条路径一般不相等；
单一贡献取 Shapley 平均，加性恒等残差不能证明分解唯一。

图 23 问题四相对问题三的 2×2 因子设计

表 8 四种情景的烘干时长（Lagrangian 核）

情景	物性关系	求解域	t_{end} (h)
t_{00}	附录 3	固定 $R = 2$ cm	56.974
t_{10}	附录 4	固定 $R = 2$ cm（对照）	128.906
t_{01}	附录 3	收缩 $R(t)$	25.088
t_{11}	附录 4	收缩 $R(t)$ （交付）	50.780

非线性系统中，两因素的贡献依赖计算路径。先换物性再加收缩，得 +71.932 h 与 -78.126 h；先加收缩再换物性，得 -31.886 h 与 +25.692 h。交互项

$$I = t_{11} - t_{10} - t_{01} + t_{00} = -46.240 \text{ h} \quad (24)$$

与主效应同量级，说明加性分解不是唯一因果分解——这正是 $t \sim R^2/D$ 的乘积结构所预期的。若仍需报两个单一贡献，取两条路径的平均（Shapley）

$$\Delta_{\text{prop}} = +48.812 \text{ h}, \quad \Delta_{\text{shrink}} = -55.006 \text{ h}, \quad \Delta_{\text{net}} = -6.194 \text{ h}. \quad (25)$$

机器精度下的加和残差只说明算术闭合，不能证明分解可靠，本文不再把它当作验证指标。四格时长的对照见图 24。

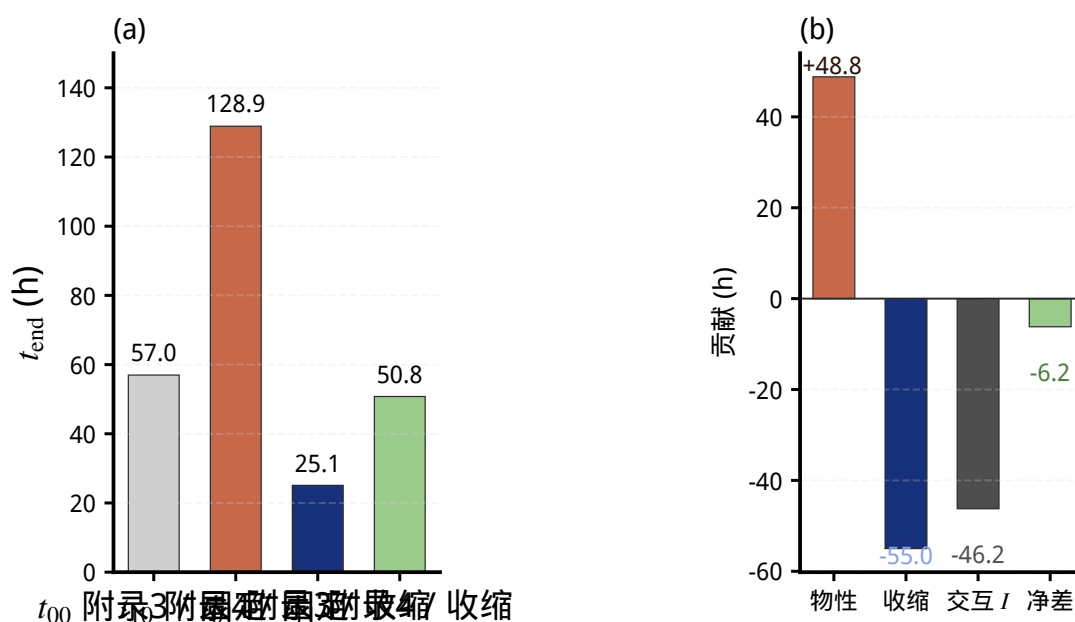


图 24 问题四 2×2 因子设计与两条路径的贡献

$t_{01} = 25.088 \text{ h}$ 说明：若仍用附录 3 的较快扩散、同时又允许收缩，时长会比问题三再短一半以上。附录 4 的低扩散把这份加速部分抵消，最终交付值落在 50.780 h。Eulerian 四格给出几乎同一交互项（ $I = -45.468 \text{ h}$ ），结论不依赖于坐标选择。

物质坐标与 Eulerian 的逐点最大含水率对照见图 25。

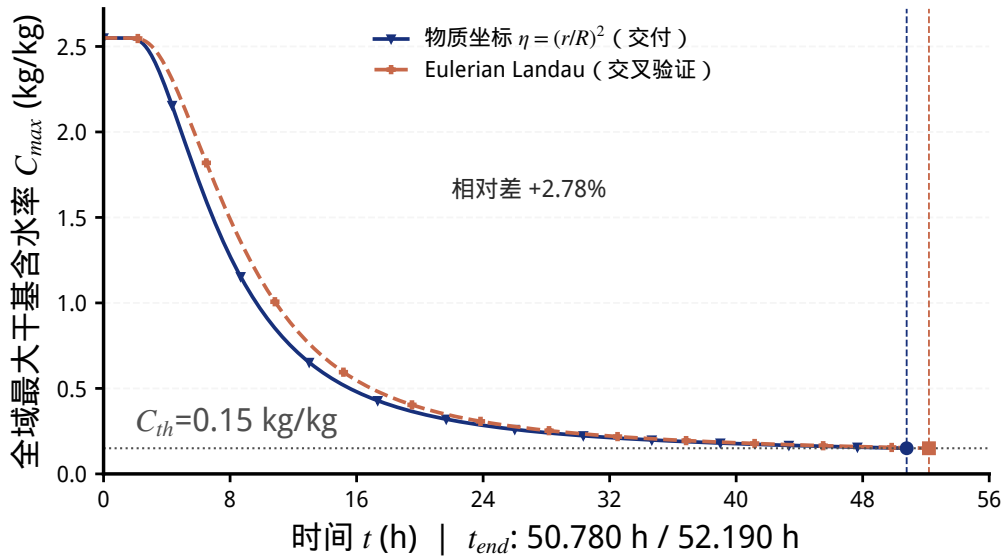


图 25 物质坐标与 Eulerian Landau 的交叉验证

9.6 收缩律与数据自洽性

由附件 2 的 $R(t)$ 与干基质量加权平均含水率 $\bar{C}(t)$ 反演体积收缩系数 $\beta(t) = [(R/R_0)^2 - 1]/(\bar{C} - C_0)$ 。若线性收缩律成立， β 应为常数；实际 β 的均值为 0.298、变异系数 22%，线性收缩律不成立。因此本文直接使用实测 $R(t)$ ，而不把体积写成 C 的仿射函数。

最后报告题给数据本身的不相容。若干物质守恒，则 $\rho_s(C_{th})/\rho_s(C_0) = (R_0/R_{end})^2$ 。按式 (8) 左端为 2.413、附件 2 右端为 2.787，相差 13.42%；由密度反演的末态半径为 1.287 cm，实测为 1.198 cm。题目要求根据附件 2 确定时长，故把实测 $R(t)$ 作为给定移动边界，不相容度只作诊断，既不修正半径，也不当作收敛判据。该不一致对时长的影响不能用坐标交叉验证的 2.78% 去代替，必须另算；本文未把这一项并入交付不确定度区间。诊断见图 26。

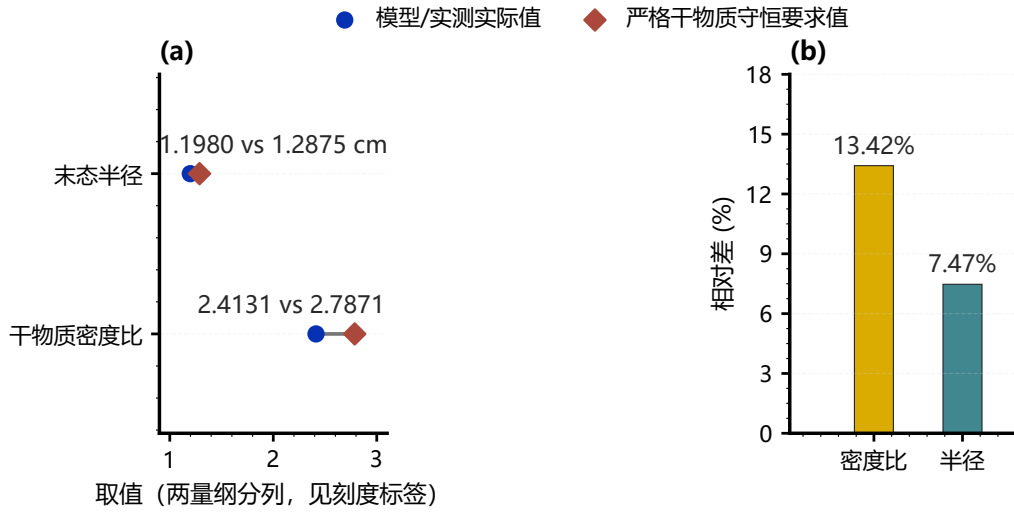


图 26 附录 4 密度关系与附件 2 收缩数据的干物质自洽性诊断

十、数值可信度与参数灵敏度

本题没有药材内部温度与含水率的实测剖面，附件给出的只是烘房环境与外轮廓半径。因此以下证据是模型的内部一致性与数学正确性，不能解释为与实测吻合；唯一具有外部性质的比对是问题一的级数解，它属于同一模型的独立解法而非独立观测^[14]。

10.1 离散守恒与收敛性

首先检验离散格式是否守恒。把控制体内含水量的累积变化与表面净流出相比，得到的是离散水分方程的收支残差，而不是实验室参考系下真实水质量的独立测量。相对残差为

$$\varepsilon = \frac{\left| \sum_i V_i [C_i(t) - C_i(0)] + \int_0^t 2\pi R h_m [C(R, \tau) - C_{\text{air}}(\tau)] d\tau \right|}{\sum_i V_i C_i(0)}. \quad (26)$$

式 (26) 在问题一为 7.41×10^{-16} 、问题三为 1.55×10^{-14} 、问题四为 4.30×10^{-14} ，均远低于 10^{-10} 的容差。这说明所选离散式在代数上收支闭合，烘干时长不会因离散漏失而系统性偏移；它不能单独证明模型与真实水质量守恒一致。

其次检验网格与时间步的无关性。以 $N = 160$ 、 $\Delta t = 0.125$ s 的细网格解为参照，交付网格 $N = 40$ 的温度与含水率相对偏差分别为 2.76×10^{-5} 与 3.04×10^{-4} ；固定 $N = 40$ 时以 $\Delta t = 0.5$ s 为参照， $\Delta t = 2$ s 与 $\Delta t = 1$ s 的偏差分别为 5.19×10^{-5} 与 1.73×10^{-5} ，均在 10^{-5} 量级。更直接的检验是达标时长本身对网格的敏感性，见图 27。

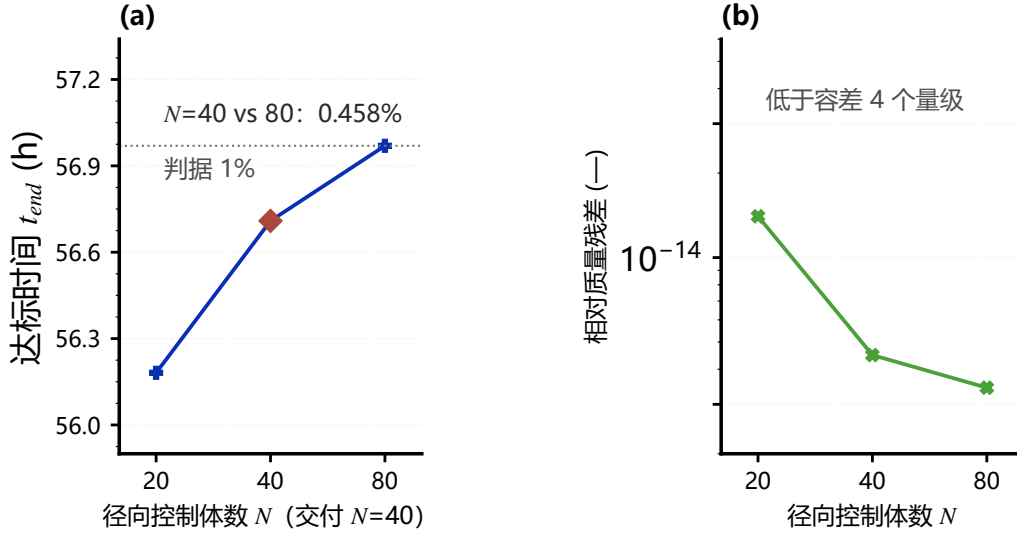


图 27 交付网络的收敛性证据

面板 (a) 给出 $N \in \{20, 40, 80\}$ 三个网格上的达标时长 56.181、56.709、56.970 h: $N = 20$ 到 $N = 40$ 相对变化 0.93%, $N = 40$ 到 $N = 80$ 已降至 0.458%, 呈单调收敛趋势, 故取 $N = 40$ 为交付网络 (图中菱形)。面板 (b) 显示三个网格上的质量残差都在 10^{-14} 量级, 即网格误差来自空间离散的截断, 而非守恒性质随网格变差。需要说明的是, 0.458% 的剩余网格误差对应约 0.26 h, 这一量级已计入后文对结论精度的表述——本文把烘干时长报到小数点后三位是为了可复核, 其物理意义上的可信位数是小时量级的整数部分与一位小数。

10.2 物理合理性与极限退化

除数值指标外, 还需检查解是否满足本题的物理次序。四问的结果一致满足: 含水率对时间单调不增; 同一时刻中心含水率不低于表面、中心温度不高于表面; 全场有界且无负值, 含水率落在 $[0.0525, 2.5500]$ 、温度落在 $[28.000, 50.165]$ 。问题一的「热先于质」也得到确认。唯一需要交代的细节是中心温度在环境温度小幅回落时出现了 $5.2 \times 10^{-5} \text{ } ^\circ\text{C}$ 的极小回落, 其幅度小于同期环境温度自身回落 $7.5 \times 10^{-3} \text{ } ^\circ\text{C}$ 的百分之一, 是边界扰动向内衰减的正常表现, 而非时间推进产生的振荡。

移动边界模型还须验证其在退化情形下是否回到固定域模型。令 $\dot{R} \equiv 0$, 物质坐标方程与 Landau 方程都退化为同一固定域扩散格式。Eulerian 路径上两个对流开关给出完全相同的 128.594 h; 物质坐标在固定域上给出 128.906 h, 相对差 0.24%, 来自 η 网格与 ξ 网格的结点分布不同, 而不是来自额外的物理项。另令 h_m 取很大值, 表面含水率应趋近空气侧值: 计算得 1 h 时表面含水率与 C_{air} 之差为 1.7×10^{-15} , 说明式 (3) 的方向与符号正确。

10.3 参数灵敏度

模型中的参数分为三类：题给的确定量、由经验公式给出而本身带有拟合不确定性的量，以及由建模选择引入的量。对第二、三类参数作一次一因子扰动，考察达标时长的响应，见图 28。

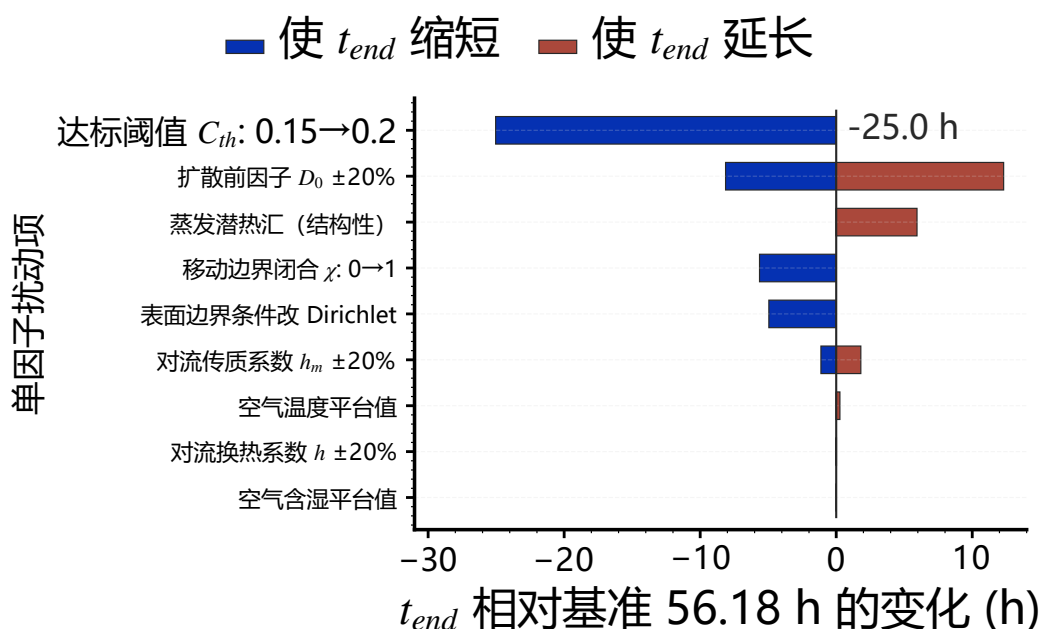


图 28 达标时长的单因子灵敏度

扩散系数前因子的 $\pm 20\%$ 扰动带来 -8.13 至 $+12.31$ h 的变化，弹性约为 -0.72 至 -1.10 ，是最强的不确定度来源；对流传质系数同幅扰动只带来 -1.12 至 $+1.81$ h，弹性约 -0.10 至 -0.16 ；对流换热系数的同幅扰动仅带来 0.02 h 量级的变化，弹性约 -0.001 。三者的排序印证了前文的判断：本题由内部扩散控制，表面换热条件几乎不影响时长，因此模型精度的关键在于扩散系数的经验公式是否可靠，而不在于对流系数的取值。

达标阈值本身不是模型参数而是工艺规定，但其影响值得说明：阈值由 0.15 改到 0.20 kg/kg 会使时长缩短 25.04 h，即近一半。这一强依赖提示在工艺上把安全水分要求放宽 0.05 kg/kg，换来的时间收益远大于任何模型改进——前提是贮藏安全性允许。

结构性选择的影响也一并探查。若把表面改为第一类边界，时长缩短至 51.23 h；若在能量方程中加入量级为 2.4×10^6 J/kg 的蒸发潜热汇，时长延长至 62.12 h。后者对应假设 3，其方向确认了前文的表述：忽略潜热使本文的时长预测偏短。两项探查都不改变交付结果，因为题面既给出了对流系数（意即第三类边界），也未给出潜热数据。

十一、模型评价与推广

11.1 模型的适用范围与优势

本文四问共用一套柱坐标径向热-质耦合方程，问题之间的差别只落在物性闭合式、求解域与终止判据等处。这种组织方式带来两个直接好处：其一，问题一的半解析对照一旦通过，其验证的是同一套有限体积离散核，因此该结论可以顺延到后三问的温度场；其二，问题四补齐 2×2 因子后可以同时看到物性路径、收缩路径及其交互，而不是把 -5.926 h 的净差说成单一原因。

分布参数描述的选择也有定量依据。表面对流与内部导热、内部扩散之比 $Bi_h = 1.389$ 、 $Bi_m = 3.240$ 均为 $O(1)$ 量级，既排除了把药材当作等温等湿集中参数体的简化，也排除了把表面直接固定为环境值的第一类边界。灵敏度探针给出的对照支持这一判断：改用第一类边界后烘干时长由 56.18 h 缩短至 51.23 h ，偏差约 8.8% ，远超其他数值设置带来的差异。移动边界采用物质坐标而非在物理坐标下重构网格，使收缩过程中网格拓扑保持不变； $\dot{R} \rightarrow 0$ 时退化为固定域，与 Eulerian 固定域时长相对差 0.24% ，来自结点分布而不是额外物理项。

11.2 模型特点

第一，问题四用物质坐标把固相速度写清楚，而不是把对流项当作可开可关的数值开关。均匀收缩给出 $v_s = r\dot{R}/R$ ，干基含水率跟随物质点，交付时长为 50.780 h ；实验室参考系下的 Landau 表述给出 52.190 h ，相对差 2.78% ，作为坐标表述的模型区间一并报告。

第二，达标判据按题目「各处」取逐点最大值。以体平均判定时， $\bar{C}$ 会在芯部远未达标时先跨过阈值： $C_{\max}$ 恰触 0.15 kg/kg 的时刻体平均仅 0.1232 kg/kg 。判据的选择在这里改变的是答案本身。

第三，物性与收缩的贡献用完整 2×2 报告，并给出交互项 $I = -46.240 \text{ h}$ 。两条路径的 Shapley 平均为物性 $+48.812 \text{ h}$ 、收缩 -55.006 h ，但交互项与主效应同量级，因此不把加性分解成唯一归因。

第四，附录 4 的 $\rho(C)$ 与附件 2 的 $R(t)$ 在严格干物质守恒下相差 13.42% 。本文依题面把实测半径作为给定边界，把不相容度降格为诊断量，并且不用坐标交叉验证的 2.78% 去代替该项对时长的影响。

11.3 模型的局限

第一项局限来自能量方程中未设蒸发潜热汇。药材内部水分汽化要吸收汽化潜热，忽略这一项会高估内部温度，进而高估水分扩散系数，使预测的烘干时长偏短。§10.3 的结构性探查量化了这一偏向：在能量方程中按 $2.4 \times 10^6 \text{ J/kg}$ 计入潜热汇后，基准时长

由 56.18 h 延长至 62.12 h，增幅约 10.6%。因此本文给出的 56.706 h 与 50.780 h 应当理解为乐观的短侧估计，工艺上若以此安排烘干批次，需要预留一成左右的时间余量。

第二项局限是均匀收缩闭合。式 (20) 假定骨架按仿射方式运动，题目没有内部位移观测来证实或否定这一点。Eulerian 交叉验证把由此带来的时长差限制在 2.78% 量级，但不能排除非均匀塌缩。

第三项局限是附录 4 密度式与附件 2 实测半径不相容。由密度反推的末态半径为 1.287 cm，实测为 1.198 cm。交付结果使用实测 $R(t)$ ，该数据冲突没有被「修正掉」，其对时长的影响也没有被单独重算。

第四项局限在于灵敏度分析采用单因素扰动，且验证以内部自洽为主。半解析解、网格收敛和离散收支残差证明的是数值实现正确，不能写成「充分验证了模型真实正确」。缺少药材内部温湿度实测剖面，模型参数未经标定。

11.4 结论与推广

按附录 2 的常物性闭合，1800 s 预热段结束时中心温度升至 33.577 °C 而中心含水率仍为 2.5500 kg/kg，表面已降至 1.511 kg/kg，热扩散与质扩散的时间尺度之比约 34，“热先于质”由此得到定量表述。按附录 3 的变物性强耦合闭合，3 h 时中心温度达 49.85 °C 已接近烘房温度，而中心含水率 1.766 kg/kg 距达标仍远，烘干时长为 56.706 h。计入附录 4 物性与附件 2 收缩规律后时长为 50.780 h，比问题三缩短 5.926 h；同物性下相对尺寸冻结缩短 60.6%。 2×2 因子的交互项为 -46.240 h，物性与收缩不能作唯一加性分解。仅凭“体积缩小所以干得快”或“扩散系数变小所以干得慢”任一单侧推理都会得到错误的量级判断。

本文的建模路线可以移用到其他伴随收缩的多孔介质干燥问题。只要给出与含水率相关的物性闭合以及可观测的外形收缩数据，把物性式与 $R(t)$ 替换即可沿用同一求解核；若干燥对象为球形或平板，仅需把控制体的面积权重由 $2\pi r$ 改为 $4\pi r^2$ 或常数，球形移动边界下的本征展开解可用作同类对照^[15]。若收缩规律本身未知而只有出口含水率可测，则问题转为由观测反求边界运动的反问题，需要另一类求解策略^[16]。对工艺参数的优化也可以在此框架内进行：由于 D 前因子的弹性接近 -1 而 h 的弹性近于零，缩短烘干时间的有效手段在于提高介质温度以抬升扩散系数，而不是加大风速强化表面换热。若进一步需要预测能耗或品质指标，需在能量方程中补入潜热汇并引入温度-时间累积的品质衰减描述，这两项都超出题给信息，属于后续工作。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，用途包括：查阅与收缩干燥相关的建模表述、代码调试、LaTeX 排版检查，以及对论文初稿的审阅（第四问守恒变量与效应分解、

Excel 单元格类型、提交规范)。使用日期为 2026-09-11。工具输出均经人工核对后才写入论文与程序；数学模型、数值结果与最终文字由参赛队负责。详细记录见支撑材料。

参考文献

- [1] Rocha R P. Influence of drying process on the quality of medicinal plants: a review[J/OL]. Journal of Medicinal Plants Research, 2011, 5 (33) . DOI:10.5897/jmprx11.001.
- [2] Park K J. Diffusional model with and without shrinkage during salted fish muscle drying [J/OL]. Drying Technology, 1998, 16 (3-5): 889-905. DOI:10.1080/07373939808917443.
- [3] Whitaker S. Simultaneous heat, mass, and momentum transfer in porous media: a theory of drying[M/OL]//Advances in heat transfer volume 13. Elsevier, 1977: 119-203. DOI:10.1016/s0065-2717(08)70223-5.
- [4] Crank J, McFarlane N R, Newby J C, et al. Introduction to the mathematics of diffusion [M/OL]//Diffusion processes in environmental systems. Macmillan Education UK, 1981: 33-60. DOI:10.1007/978-1-349-05825-9_3.
- [5] Yilbas B, Hussain M, Dincer I. Heat and moisture diffusion in slab products due to convective boundary condition[J/OL]. Heat and Mass Transfer, 2003, 39 (5): 471-476. DOI:10.1007/s00231-002-0323-x.
- [6] Sander A, Tomas S, Skansi D. The influence of air temperature on effective diffusion coefficient of moisture in the falling rate period[J/OL]. Drying Technology, 1998, 16 (7): 1487-1499. DOI:10.1080/07373939808917472.
- [7] Patankar S V. Numerical heat transfer and fluid flow[M/OL]. CRC Press, 2018. DOI:10.1201/9781482234213.
- [8] Mignone A. High-order conservative reconstruction schemes for finite volume methods in cylindrical and spherical coordinates[J/OL]. Journal of Computational Physics, 2014, 270: 784-814. DOI:10.1016/j.jcp.2014.04.001.
- [9] Dall'Osso A. An iterative back substitution algorithm for the solution of tridiagonal matrix systems with fringes[J/OL]. Journal of Computational and Applied Mathematics, 2004, 169 (1): 87-99. DOI:10.1016/j.cam.2003.11.004.
- [10] Fromme J, Golberg M. Exponential convergence of picard iteration for integrating linear and nonlinear modally coupled equations of motion[C/OL]//38th structures, structural

- dynamics, and materials conference. American Institute of Aeronautics and Astronautics, 1997. DOI:10.2514/6.1997-1280.
- [11] Guellal M, Sadat H, Prax C. A simple model for transient heat conduction in an infinite cylinder with convective boundary conditions[J/OL]. Journal of Heat Transfer, 2009, 131 (5) . DOI:10.1115/1.3082428.
- [12] Doymaz I. Air-drying characteristics, effective moisture diffusivity and activation energy of grape leaves[J/OL]. Journal of Food Processing and Preservation, 2011, 36 (2) : 161-168. DOI:10.1111/j.1745-4549.2011.00557.x.
- [13] Fritsch F N, Carlson R E. Monotone piecewise cubic interpolation[J/OL]. SIAM Journal on Numerical Analysis, 1980, 17 (2): 238-246. DOI:10.1137/0717021.
- [14] Pelletier D, Roache P J. Verification and validation of computational heat transfer[Z/OL]. Wiley, 2000: 417-442. DOI:10.1002/9780470172599.ch13.
- [15] Ali I. Eigenfunction expansion analysis of heat and mass diffusion in a spherical moving boundary problem with application to convective drying[J/OL]. Boundary Value Problems, 2026, 2026 (1) . DOI:10.1186/s13661-026-02329-2.
- [16] Lotfi Y, Parand K, Rashedi K, et al. Numerical study of temperature distribution in an inverse moving boundary problem using a meshless method[J/OL]. Engineering with Computers, 2019, 37 (1): 461-475. DOI:10.1007/s00366-019-00835-9.

附录 A 支撑材料清单

本文全部数值结果由下列 Python 源程序真实计算生成，入口为 `main.py`，运行环境为 Python 3 与 NumPy、SciPy、pandas、openpyxl。程序不含任何硬编码的结果数值，四个子问题的答案均由同一求解核在不同物性闭合与终止判据下计算得到。

表 9 支撑材料文件清单

类别	文件名	内容
程序入口	<code>main.py</code>	四问求解编排与结果写出
	<code>driver.py</code>	时间推进、算子分裂与 Picard 迭代
	<code>fvm.py</code>	柱坐标有限体积离散与三对角求解
	<code>props.py</code>	附录 2—4 物性闭合式

类别	文件名	内容
	<code>ambient.py</code>	附件 1 环境温湿度插值
	<code>geometry.py</code>	附件 2 半径单调化与 PCHIP 插值
	<code>params.py</code>	题给参数与数据读入
	<code>problem1.py</code>	问题一预热段求解与报表
	<code>problem2.py</code>	问题二全程变物性求解与报表
	<code>problem3.py</code>	问题三达标时长求解与报表
	<code>problem4.py</code>	问题四物质坐标求解、 2×2 因子与干物质核算
	<code>master_p23.py</code>	问题二、三共用的主求解流程
	<code>validate.py</code>	守恒、收敛、级数解对照与极限退化检验
	<code>io_out.py</code>	报表与工作簿写出
	<code>data_check.py</code>	附件数据完整性核对
	<code>template_check.py</code>	附件 3 模板结构核对
计算结果	<code>result1.xlsx</code>	问题一温度与水分浓度全网格结果
	<code>result2.xlsx</code>	问题二全烘干过程双表结果
	<code>result3.xlsx</code>	问题三 $60\text{ s} \rightarrow t_{\text{end}}$ 结果
	<code>result4.xlsx</code>	问题四收缩域结果
	<code>effect_isolation.csv</code>	Lagrangian/Eulerian 2×2 时长
	<code>shrinkage_beta.csv</code>	体积收缩系数反演
	<code>mass_consistency.csv</code>	干物质守恒诊断
	<code>sensitivity.csv</code>	单因子灵敏度记录
	<code>validation.json</code>	数值可信度检验记录

附录 B 程序源代码

下列代码为完整可运行源程序，按依赖层次排列：先给出参数与数据读入，再给出离散核与物性、时间推进与求解编排，最后给出四问的调用与验证。运行方式为在 `code/` 目录下执行 `python main.py`，程序自动读入附件并把全部报表与工作簿写入 `output/`。

2.1 参数与数据读入：params.py

```

1  """全局参数口径表（唯一常量来源）。
2
3  所有物理常量、附录系数、数值离散参数在此集中定义，逐项对应
4  `PROBLEM_FACTS.json` / `DATA_FACTS.json` / `MODELING_REPORT.md §12` 的口径表。
5  其余模块一律 `from params import *` 或 `import params`，不得另行硬编码物理常量。
6
7  命名约定：附录系数以 APPn_ 前缀区分；幂次用 mantissa * 10.0**power 显式书写，
8  使尾数与题面 mantissa 字段一致（题面上标  $8 \times 10^{-7}$  等）。
9  """
10 from __future__ import annotations
11
12 from pathlib import Path
13
14 # -----
15 # 数据文件路径（相对工作区根目录；run_compute 已将根目录加入 sys.path 且 cwd=root）
16 # -----
17 ROOT = Path(__file__).resolve().parent.parent
18 DATA_DIR = ROOT / "user_data" / "附件" / "附件"
19 ATTACH1 = DATA_DIR / "附件1.xlsx"          # 时间 | 温度 | 空气水分浓度 (241 行)
20 ATTACH2 = DATA_DIR / "附件2.xlsx"          # 时间 | 半径 (145 行)
21 TEMPLATE_DIR = DATA_DIR / "附件3"
22 OUTPUT_DIR = ROOT / "output"
23 FIGURE_DIR = ROOT / "figures"
24
25 # 数据行数断言 (DATA_FACTS ingest_assertion)
26 N_ROWS_ATTACH1 = 241
27 N_ROWS_ATTACH2 = 145
28
29 # -----
30 # 几何（题面正文：长 25 cm、半径 2 cm）—— 进入模型换算为 SI (m)
31 # -----
32 R0_CM = 2.0
33 L_CM = 25.0
34 CM_TO_M = 1.0 / 100.0
35 R0_M = R0_CM * CM_TO_M          # 0.02 m
36 L_M = L_CM * CM_TO_M           # 0.25 m
37
38 # -----
39 # 初始条件（题面正文）
40 # -----
41 T0_C = 28.0                      # 初始温度 °C
42 C0 = 2.55                       # 初始干基含水率 kg/kg
43
44 # -----
45 # 边界系数（附录2，规则 R6 四问共用）
46 # -----
47 H_CONV = 25.0                   # 对流换热系数 W/(m^2 · K)
48 HM_CONV = 8.0 * 10.0 ** (-7)    # 对流传质系数 m/s（题面  $8 \times 10^{-7}$ ）
49
50 # -----
51 # 达标阈值（题面问题3：各处 < 0.15）
52 # -----
53 C_TH = 0.15                     # kg/kg
54
55 # -----

```

```

56 # 绝对温度换算 (derived_formulas)
57 # -----
58 T_KELVIN = 273.15
59
60 # -----
61 # 物性系数 (三套附录严格按问题隔离, MODELING_REPORT §4)
62 # 附录2 (仅问题1): rho/cp/k 常数; D=7e-9 * exp(-0.89/C), 不含温度项
63 # -----
64 APP2_RHO = 820.0 # kg/m^3
65 APP2_CP = 2600.0 # J/(kg * K)
66 APP2_K = 0.36 # W/(m * K)
67 APP2_D_PREF = 7.0 * 10.0 ** (-9) # m^2/s
68 APP2_D_EC = 0.89 # exp(-0.89/C)
69
70 # 附录3 (问题2、3): 四条公式全变; rho 用干基 C, cp/k 用湿基 w
71 APP3_RHO_A = 650.0
72 APP3_RHO_B = 128.0 # rho = 650 + 128*C
73 APP3_CP_A = 1450.0
74 APP3_CP_B = 2736.0 # cp = 1450 + 2736*w
75 APP3_K_A = 0.21
76 APP3_K_B = 0.38 # k = 0.21 + 0.38*w
77 APP3_D_PREF = 2.4 * 10.0 ** (-3) # m^2/s
78 APP3_D_EC = 0.45 # exp(-0.45/C)
79 APP3_D_ET = 3850.0 # exp(-3850/T_K)
80
81 # 附录4 (仅问题4): 结构同附录3、系数不同
82 APP4_RHO_A = 760.0
83 APP4_RHO_B = 90.0 # rho = 760 + 90*C
84 APP4_CP_A = 1850.0
85 APP4_CP_B = 2150.0 # cp = 1850 + 2150*w
86 APP4_K_A = 0.12
87 APP4_K_B = 0.20 # k = 0.12 + 0.20*w
88 APP4_D_PREF = 4.2 * 10.0 ** (-4) # m^2/s
89 APP4_D_EC = 0.30 # exp(-0.30/C)
90 APP4_D_ET = 3850.0 # exp(-3850/T_K)
91
92 # -----
93 # 环境平台外推值 (假设5; 附件1 末值)
94 # -----
95 TA_PLATEAU = 50.165 # °C
96 CA_PLATEAU = 0.04986 # kg/kg
97 AMBIENT_T_MAX_S = 14400.0 # 附件1 时间轴上界 (含) s
98 # 灵敏度替代值 (末30点均值, SA4/SA5)
99 TA_PLATEAU_ALT = 50.0105
100 CA_PLATEAU_ALT = 0.049982
101
102 # -----
103 # 潜热汇 (假设4, 关闭; SA8 可探查)
104 # -----
105 LATENT_HEAT_SINK = 0.0 # J/kg
106 LATENT_HEAT_PROBE = 2.4 * 10.0 ** 6 # SA8 探查值
107
108 # -----
109 # 数值离散 (MODELING_REPORT §12 参数表 + §11 步长策略)
110 # -----
111 N_CV = 40 # 交付控制体数

```

```

112 C_FLOOR = 1.0 * 10.0 ** (-3) # 浓度下限保护 (§11 第5步, C_TH)
113 D_FLOOR = 1.0 * 10.0 ** (-14) # D 数值下限 (§13.3 异常预案)
114 CHI_PRIMARY = 0 # Eulerian Landau 交叉验证: 保留完整表观对流项
115 CHI_ALT = 1 # Eulerian Landau: 仿射骨架使对流项抵消 (≡ 均匀收缩 Lagrangian)
116 FRAME_PRIMARY = "lagrange" # 问题4 主用: Lagrangian 干基质量坐标
117 FRAME_CROSSCHECK = "landau" # 问题4 交叉验证: Eulerian Landau  $\xi=r/R(t)$ 
118
119 # 时间步长 (s)
120 DT_P1 = 1.0 # 问题1
121 DT_SEG_FINE = 2.0 # 问题2/3/4:  $t \leq 14400$  s
122 DT_SEG_COARSE = 4.0 # 问题2/3/4:  $t > 14400$  s
123 DT_SWITCH_S = 14400.0 # 分段切换时刻
124
125 # Picard 迭代
126 PICARD_MAX = 20
127 PICARD_TOL_T = 1.0 * 10.0 ** (-6) # °C
128 PICARD_TOL_C = 1.0 * 10.0 ** (-8) # kg/kg
129
130 # 达标事件仿真上限 ( $h \rightarrow s$ )
131 T_END_MAX_H = 120.0
132 SEC_PER_HOUR = 3600.0
133 T_END_MAX_S = T_END_MAX_H * SEC_PER_HOUR
134
135 # -----
136 # 输出网格 (output_specs)
137 # -----
138 SAVE_DR_CM = 0.1 # 落盘距离步长
139 N_DIST_COLS = 21 # 0.0 .. 2.0 cm 共 21 列
140 # 距离列 (cm): 0,0.1,...,2.0 —— 由步长生成, 避免硬编码序列
141 DIST_COLS_CM = [round(i * SAVE_DR_CM, 4) for i in range(N_DIST_COLS)]
142
143 # 问题1 报表 (表1/表2)
144 P1_SAVE_DURATION_S = 1800.0
145 P1_SAVE_DT_S = 1.0
146 P1_REPORT_TIMES_S = [100.0, 300.0, 600.0, 900.0, 1200.0, 1500.0, 1800.0]
147 P1_REPORT_RADII_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
148
149 # 问题2 报表 (表3/表4): 0.5 h 间隔至 3 h
150 P2_REPORT_TIMES_H = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
151 P2_REPORT_RADII_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
152 P2_SAVE_DT_S = 1.0 # result2.xlsx A 列步长
153
154 # 问题3/4 报表 (表5/表6): 6 h 间隔 + 末行"烘干结束时间"
155 P34_REPORT_DT_H = 6.0
156 P34_REPORT_DR_CM = 0.5
157 P34_REPORT_RADII_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
158 P34_SAVE_DT_S = 60.0 # result3/result4 A 列步长
159
160 # 输出小数位
161 OUT_DECIMALS = 4
162
163 # -----
164 # 附件2 半径末值 (S4 固定域下界参照: DATA_FACTS key_points)
165 # -----
166 R_END_CM = 1.198
167 R_END_M = R_END_CM * CM_TO_M

```

```

168
169 # -----
170 # 验证参数 (§9)
171 # -----
172 BESSEL_ROOTS = 30          # 级数解特征根数 (§9.3)
173 BESSEL_ROOTS_FALLBACK = 100 # 偏差超限时增至此数
174 # 网格无关性参考解 (§9.2, 仅问题1温度场)
175 REF_N_CV = 160
176 REF_DT_S = 0.125
177 GRID_TOL = 0.005          # e_T, e_C 判据 (K13)
178 MASS_RESIDUAL_TOL = 1.0 * 10.0 ** (-10) # K12
179
180 # 达标时长硬区间 (K9/K10; LOGIC_CONTRACT bounds)
181 T_END_HARD_LO_H = 24.0
182 T_END_HARD_HI_H = 120.0

```

2.2 环境驱动插值: ambient.py

```

1  """烘房环境驱动函数 (附件1 线性插值 + 平台外推, MODELING_REPORT §3.2 式(7)(8))。
2
3  T_air(t), C_air(t):  $t \in [0, 14400]$  线性插值附件1 全部 241 行;  $t > 14400$  取平台值 (假设5)。
4  附件1 第3列是烘房空气含湿量 (量级  $1e-2$ ), 不是药材水分浓度。
5  """
6  from __future__ import annotations
7
8  import numpy as np
9  import pandas as pd
10
11  import params as P
12
13  _CACHE = {}
14
15
16  class Ambient:
17      """时变环境边界。可注入平台替代值做灵敏度 (SA4/SA5)。"""
18
19      def __init__(self, t_plateau=None, c_plateau=None):
20          t_arr, T_arr, C_arr = _load_attach1()
21          self._t = t_arr
22          self._T = T_arr
23          self._C = C_arr
24          self.T_plateau = P.TA_PLATEAU if t_plateau is None else float(t_plateau)
25          self.C_plateau = P.CA_PLATEAU if c_plateau is None else float(c_plateau)
26          self.t_max = P.AMBIENT_T_MAX_S
27
28      def T_air(self, t):
29          """烘房空气温度 °C。t 可为标量或数组。"""
30          t = np.asarray(t, dtype=float)
31          val = np.interp(t, self._t, self._T)
32          return np.where(t <= self.t_max, val, self.T_plateau)
33
34      def C_air(self, t):
35          """烘房空气含湿量 kg/kg。t 可为标量或数组。"""
36          t = np.asarray(t, dtype=float)

```

```

37     val = np.interp(t, self._t, self._C)
38     return np.where(t <= self.t_max, val, self.C_plateau)
39
40
41 def _load_attach1():
42     if "attach1" not in _CACHE:
43         df = pd.read_excel(P.ATTACH1, sheet_name=0)
44         n = int(df.shape[0])
45         if n != P.N_ROWS_ATTACH1:
46             raise ValueError(f"附件1 应为 {P.N_ROWS_ATTACH1} 行, 实际 {n} 行 (数据未读全)")
47         t = pd.to_numeric(df.iloc[:, 0], errors="coerce").to_numpy(dtype=float)
48         T = pd.to_numeric(df.iloc[:, 1], errors="coerce").to_numpy(dtype=float)
49         C = pd.to_numeric(df.iloc[:, 2], errors="coerce").to_numpy(dtype=float)
50         if not (np.all(np.isfinite(t)) and np.all(np.isfinite(T)) and np.all(np.isfinite(C))):
51             raise ValueError("附件1 含非有限值")
52         _CACHE["attach1"] = (t, T, C)
53     return _CACHE["attach1"]

```

2.3 收缩半径插值: geometry.py

```

1  """求解域半径 R(t) 与收缩速率 Rdot(t) (MODELING_REPORT §7.4) 。
2
3  固定域: 返回常值 R 与 Rdot=0。
4  移动边界 (问题4) : 附件2 → 累积最小值单调化 → PCHIP 保形插值;
5  Rdot 取 PCHIP 解析导数; 数据用尽后冻结半径 (radius_frozen) 。
6  单调化只用累积最小值, 不用平滑滤波 (端点不变) 。
7  """
8  from __future__ import annotations
9
10 import numpy as np
11 import pandas as pd
12 from scipy.interpolate import PchipInterpolator
13
14 import params as P
15
16 _CACHE = {}
17
18
19 class FixedGeometry:
20     """固定求解域, R=R_const (问题1/2/3 与情景 S0/S1/S4) 。”””
21
22     def __init__(self, R_const_m):
23         self.R_const = float(R_const_m)
24         self.radius_frozen = False
25         self.moving = False
26
27     def R(self, t):
28         t = np.asarray(t, dtype=float)
29         return np.full(t.shape, self.R_const) if t.ndim else self.R_const
30
31     def Rdot(self, t):
32         t = np.asarray(t, dtype=float)
33         return np.zeros(t.shape) if t.ndim else 0.0
34

```

```

35
36 class MovingGeometry:
37     """附件2 移动边界（问题4，情景 S2/S3）。半径单位 cm→m。"""
38
39     def __init__(self):
40         t_arr, R_mono_m = _load_attach2_mono()
41         self._t = t_arr
42         self._R = R_mono_m
43         self._t_max = float(t_arr[-1])
44         self._R_end = float(R_mono_m[-1])
45         self._pchip = PchipInterpolator(t_arr, R_mono_m, extrapolate=False)
46         self._pchip_d = self._pchip.derivative()
47         self.radius_frozen = False
48         self.moving = True
49
50     def R(self, t):
51         t = np.asarray(t, dtype=float)
52         inside = np.clip(t, self._t[0], self._t_max)
53         val = self._pchip(inside)
54         if np.any(t > self._t_max):
55             self.radius_frozen = True
56         return np.where(t > self._t_max, self._R_end, val)
57
58     def Rdot(self, t):
59         t = np.asarray(t, dtype=float)
60         inside = np.clip(t, self._t[0], self._t_max)
61         val = self._pchip_d(inside)
62         # 冻结段 Rdot=0; 数据端点导数按 PCHIP 端点值
63         return np.where((t > self._t_max) | (t < self._t[0]), 0.0, val)
64
65     @property
66     def t_data_max(self):
67         return self._t_max
68
69     @property
70     def R_data(self):
71         """返回 (t, R_mono_m) 供诊断/绘图。"""
72         return self._t.copy(), self._R.copy()
73
74
75 def _load_attach2_mono():
76     if "attach2" not in _CACHE:
77         df = pd.read_excel(P.ATTACH2, sheet_name=0)
78         n = int(df.shape[0])
79         if n != P.N_ROWS_ATTACH2:
80             raise ValueError(f"附件2 应为 {P.N_ROWS_ATTACH2} 行，实际 {n} 行（数据未读全）")
81         t = pd.to_numeric(df.iloc[:, 0], errors="coerce").to_numpy(dtype=float)
82         R_cm = pd.to_numeric(df.iloc[:, 1], errors="coerce").to_numpy(dtype=float)
83         if not (np.all(np.isfinite(t)) and np.all(np.isfinite(R_cm))):
84             raise ValueError("附件2 含非有限值")
85         # 累积最小值单调化（收缩不可逆），端点保持不变
86         R_mono_cm = np.minimum.accumulate(R_cm)
87         R_mono_m = R_mono_cm * P.CM_TO_M
88         _CACHE["attach2"] = (t, R_mono_m)
89     return _CACHE["attach2"]

```


2.4 物性闭合式：props.py

```
1 """ 三套附录物性模块（严格按问题隔离，MODELING_REPORT §4）。
2
3 props(appendix, C, T_celsius) -> (rho, cp, k, D), 全部返回与 C 同形状的 ndarray。
4 rho 用干基 C, cp/k 用湿基 w=C/(C+1); D 中温度用绝对温度 K 且取药材局部温度。
5 附录2 的 D 不含温度项。附录之间禁止串用（K14）。
6 """
7 from __future__ import annotations
8
9 import numpy as np
10
11 import params as P
12
13
14 def wet_basis(C):
15     """湿基分率 w = C/(C+1)。"""
16     C = np.asarray(C, dtype=float)
17     return C / (C + 1.0)
18
19
20 def props(appendix: int, C, T_celsius=None):
21     """返回 (rho, cp, k, D), 均为与 C 同形状的 ndarray。
22
23     appendix ∈ {2,3,4}。附录3/4 需要 T_celsius（药材局部温度℃度，）。
24     """
25     C = np.asarray(C, dtype=float)
26
27     if appendix == 2:
28         rho = np.full_like(C, P.APP2_RHO)
29         cp = np.full_like(C, P.APP2_CP)
30         k = np.full_like(C, P.APP2_K)
31         D = P.APP2_D_PREF * np.exp(-P.APP2_D_EC / C)
32         return rho, cp, k, D
33
34     if T_celsius is None:
35         raise ValueError(f"附录{appendix} 的 D 依赖温度，必须提供 T_celsius")
36     T_celsius = np.asarray(T_celsius, dtype=float)
37     T_K = T_celsius + P.T_KELVIN
38     # K15: 绝对温度必须 > 273
39     if np.any(T_K <= 273.0):
40         raise ValueError(f"附录{appendix} D 计算温度非绝对温标: min(T_K)={float(np.min(T_K))}")
41     w = C / (C + 1.0)
42
43     if appendix == 3:
44         rho = P.APP3_RHO_A + P.APP3_RHO_B * C
45         cp = P.APP3_CP_A + P.APP3_CP_B * w
46         k = P.APP3_K_A + P.APP3_K_B * w
47         D = P.APP3_D_PREF * np.exp(-P.APP3_D_EC / C) * np.exp(-P.APP3_D_ET / T_K)
48         return rho, cp, k, D
49
50     if appendix == 4:
51         rho = P.APP4_RHO_A + P.APP4_RHO_B * C
52         cp = P.APP4_CP_A + P.APP4_CP_B * w
53         k = P.APP4_K_A + P.APP4_K_B * w
```

```

54     D = P.APP4_D_PREF * np.exp(-P.APP4_D_EC / C) * np.exp(-P.APP4_D_ET / T_K)
55     return rho, cp, k, D
56
57     raise ValueError(f"未知附录编号: {appendix}")
58
59
60 def rho_s_eff(appendix: int, C):
61     """干物质表观密度  $\rho_s = \rho / (1+C)$  (INC1 质量自洽诊断, 问题4)。"""
62     C = np.asarray(C, dtype=float)
63     if appendix == 4:
64         rho = P.APP4_RHO_A + P.APP4_RHO_B * C
65     elif appendix == 3:
66         rho = P.APP3_RHO_A + P.APP3_RHO_B * C
67     elif appendix == 2:
68         rho = np.full_like(C, P.APP2_RHO)
69     else:
70         raise ValueError(f"未知附录编号: {appendix}")
71     return rho / (1.0 + C)

```

2.5 有限体积离散核: fvm.py

```

1  """有限体积分核 (柱坐标控制体 + 全隐式后向 Euler, MODELING_REPORT §6.1、§7.5)。
2
3  两套坐标共用同一隐式有限体积分核:
4  - Landau  $\xi=r/R(t) \in [0,1]$  (问题1/2/3 与问题4 Eulerian 交叉验证):
5      固定域  $R=R_0$ 、 $R_{dot}=0$  退化为物理坐标; 移动边界含  $(1-\chi) \cdot \xi \cdot R_{dot}/R \cdot \partial \xi \phi$  对流项。
6  - Lagrangian 干基坐标  $s=\int \rho_d r dr$  (问题4 主用): 域  $[0,S]$  固定, 收缩进入  $r(s,t)^2$  几何因子,
7      无对流项, 干物质由坐标本身守恒。
8
9  界面输运系数取相邻节点平均 (口径 FACE_MODE, 交付用算术平均 §6.1; 调和 §13.4 供对照);
10  界面通量以  $2\pi \cdot \xi_{\{i\pm 1/2\}} \cdot \Gamma$  面积权重体现  $1/r$  因子。
11  表面 Robin 以通量形式并入 (§6.1); 中心零通量为自然边界 (无需显式施加)。
12  对流项用一阶上风离散:  $R_{dot} \leq 0 \Rightarrow$  平流速度  $v = -\xi \cdot R_{dot}/R \geq 0 \Rightarrow$  迎风取后向差分, 保证系数矩阵对角占优。
13  """
14  from __future__ import annotations
15
16  import numpy as np
17  from scipy.linalg import solve_banded
18
19  import params as P
20
21
22  def make_grid(N=None):
23      """返回  $\xi$  节点(N+1)、 $\Delta \xi$ 、单位长度控制体体积  $\tilde{V}(N+1)$  ( $\xi$  度量)。"""
24      if N is None:
25          N = P.N_CV
26      dxi = 1.0 / N
27      xi = np.linspace(0.0, 1.0, N + 1)
28      Vt = np.empty(N + 1)
29      Vt[0] = np.pi * (dxi / 2.0) ** 2
30      Vt[1:N] = 2.0 * np.pi * xi[1:N] * dxi
31      Vt[N] = np.pi * (1.0 - (1.0 - dxi / 2.0) ** 2)
32      return xi, dxi, Vt
33

```

```

34
35 # 界面平均口径 (§6.1 vs §13.4 的报告内部分歧, 见 RESULTS.md「界面平均口径」注):
36 # 'arith' —— §6.1 正文「相邻节点算术平均」。交付默认。
37 #           与 §7.6/§9.2 全部预检锚点、K13 网格无关性、K9/K10 时长区间自治
38 #           (N=20 t_end=56.18h≈报告 56.178h; N=20→160 仅 0.9% 漂移, 已网格收敛)。
39 # 'harm' —— §13.4/MC2 指定的调和平均。物理上更贴合 266 倍 D 突变的串联电阻,
40 #           但在交付网格 N=40 未网格收敛 (N=40 120h 内不达标、N=80=88h、N=160=60h,
41 #           需 N≥320 才收敛到同一 ~57h), 违反 K13 与 N=40 交付口径。
42 # 两种口径均给出左右唯一的界面值, 质量收支同为机器精度 (实测 1e-15),
43 # 故 §9.1/§13.3「唯一界面值」对二者都成立; 分歧只是收敛速度与物理偏好。
44 FACE_MODE = "arith"
45
46
47 def face_mean(G, mode=None):
48     """相邻节点界面平均, 返回长度 N 的界面值 (i+1/2, i=0..N-1)。
49
50     mode='arith' 算术平均 (§6.1, 交付默认); 'harm' 调和平均 (§13.4, 供对照)。
51     G 可能为 0 (D→0), 调和口径此时取 0 (无输运), 避免 0/0。
52     """
53     mode = FACE_MODE if mode is None else mode
54     Gi = G[:-1]
55     Gi1 = G[1:]
56     if mode == "arith":
57         return 0.5 * (Gi + Gi1)
58     denom = Gi + Gi1
59     with np.errstate(divide="ignore", invalid="ignore"):
60         return np.where(denom > 0.0, 2.0 * Gi * Gi1 / denom, 0.0)
61
62
63 def harmonic_face(G):
64     """向后兼容别名 (调和口径)。新代码用 face_mean。"""
65     return face_mean(G, mode="harm")
66
67
68 def step_implicit(phi_old, cap, G, beta, phi_air, R, Rdot, chi, dt, xi, dxi, Vt,
69                 return_diag=False, src=None):
70     """推进一个隐式时间步, 返回 phi_new (长度 N+1)。
71
72     参数:
73     phi_old : 上一时刻场量 (N+1,)
74     cap     : 容量系数 κ 逐点 (N+1,) —— 温度取 pc_p, 浓度取 1
75     G       : 输运系数 Γ 逐点 (N+1,) —— 温度取 k, 浓度取 D (已施 D_FLOOR)
76     beta    : Robin 系数 (h 或 h_m)
77     phi_air : 环境侧驱动值 (标量)
78     R, Rdot : 当前半径与收缩速率
79     chi     : 骨架闭合开关 (0 主用含对流项 / 1 无对流项)
80     dt      : 时间步长
81     xi,dxi,Vt : make_grid 输出
82     return_diag : True 时额外返回离散守恒残差 (§9.1/K12 质量收支)
83     src     : 逐控制体体积分源项 (N+1,), rhs += src; 默认 None (SA8 潜热汇 / U4 单测)
84
85     返回: phi_new, 或 (phi_new, resid_abs)。resid_abs 为独立核算的守恒残差
86           |Δ储存率 - 表面净通量 + 对流净额|, 扩散界面通量解析抵消 ⇒ 应≈机器精度。
87     """
88     N = phi_old.size - 1
89     R2 = R * R

```

```

90
91 # 界面平均输运系数 (i+1/2, i=0..N-1), 口径由 FACE_MODE 决定 (默认算术, §6.1)
92 Gf = face_mean(G)
93 xi_face = (np.arange(N) + 0.5) * dxi #  $\xi_{i+1/2}$ 
94 aE = (2.0 * np.pi / R2) * xi_face * Gf / dxi # 东面导纳, 长度 N
95
96 # 三对角: A[i,i-1]=lo[i], A[i,i]=di[i], A[i,i+1]=up[i]
97 di = cap * Vt / dt # 时间项
98 lo = np.zeros(N + 1)
99 up = np.zeros(N + 1)
100
101 # 扩散 (每个界面同时进入两侧节点方程)
102 di[0:N] += aE
103 up[0:N] += -aE
104 di[1:N + 1] += aE
105 lo[1:N + 1] += -aE
106
107 rhs = (cap * Vt / dt) * phi_old
108
109 # 表面 Robin (节点 N): sfc =  $2\pi \cdot \beta / R$ 
110 sfc = 2.0 * np.pi * beta / R
111 di[N] += sfc
112 rhs[N] += sfc * phi_air
113
114 # 可选体积分源项 (SA8 潜热汇 / U4 线性场源; 默认无)
115 if src is not None:
116     rhs = rhs + src
117
118 # 对流项 (仅  $\chi=0$ ):  $g_i = (1-\chi) \cdot \kappa_i \cdot \xi_i \cdot Rdot / R \cdot \tilde{V}_i$ , 一阶迎风
119 lo_adv = up_adv = di_adv = None
120 if chi == 0 and Rdot != 0.0:
121     g = cap * xi * (Rdot / R) * Vt #  $\leq 0$  ( $Rdot \leq 0$ )
122     di_adv = np.abs(g) / dxi
123     lo_adv = np.where(g <= 0.0, g / dxi, 0.0) # 迎风:  $g \leq 0 \rightarrow$  后向差分进 lo
124     up_adv = np.where(g > 0.0, -g / dxi, 0.0) #  $g > 0 \rightarrow$  前向差分进 up
125     di += di_adv
126     lo += lo_adv
127     up += up_adv
128
129 ab = np.zeros((3, N + 1))
130 ab[0, 1:] = up[0:N] # 超对角 A[i,i+1] 置于列 i+1
131 ab[1, :] = di
132 ab[2, :-1] = lo[1:N + 1] # 次对角 A[i,i-1] 置于列 i-1
133 phi_new = solve_banded((1, 1), ab, rhs)
134
135 if not return_diag:
136     return phi_new
137
138 # 独立守恒核算: 储存率 = 表面净通量 - 对流净额 (扩散逐面对消)
139 stored = float(np.sum(cap * Vt * (phi_new - phi_old)) / dt)
140 qsurf = float(sfc * (phi_air - phi_new[N]))
141 adv_net = 0.0
142 if di_adv is not None:
143     adv_net = float(np.sum(di_adv * phi_new)
144                     + np.sum(lo_adv[1:] * phi_new[:-1])
145                     + np.sum(up_adv[:-1] * phi_new[1:]))

```

```

146     resid_abs = abs(stored - qsurf + adv_net)
147     return phi_new, resid_abs
148
149
150 def make_s_grid(N, S_tot=1.0):
151     """归一化干基/物质坐标  $\eta=(r/R(t))^2 \in [0,1]$  的顶点网格。
152
153     均匀  $\Delta\eta$  对应物理半径  $r=R\sqrt{\eta}$ , 表面更密。S_tot 默认 1 (归一化)。
154     """
155     ds = float(S_tot) / N
156     s = np.linspace(0.0, float(S_tot), N + 1)
157     Vt = np.empty(N + 1)
158     Vt[0] = 0.5 * ds
159     Vt[1:N] = ds
160     Vt[N] = 0.5 * ds
161     return s, ds, Vt
162
163
164 def affine_radius(s, S_tot, R):
165     """均匀收缩:  $r(\eta,t)=R(t) \cdot \sqrt{\eta/S}$ ,  $\eta$  为物质坐标。"""
166     ratio = np.clip(s / max(float(S_tot), 1e-30), 0.0, 1.0)
167     return float(R) * np.sqrt(ratio)
168
169
170 def step_lagrange(phi_old, cap, Gcoef, R, dt, ds, Vt, s, sfc, phi_air,
171                 return_diag=False, src=None):
172     """物质坐标  $\eta=(r/R) \in [0,1]$  上经验 Fick/Fourier 方程的全隐式有限体积步。
173
174     与 Eulerian Landau  $\chi=1$  (仿射骨架、无表观对流) 同一连续方程, 仅网格按  $\eta$  均匀:
175         cap  $\partial\phi/\partial t = (4/R^2) \partial/\partial\eta (\eta \Gamma \partial\phi/\partial\eta)$ 
176     水分 cap=1、 $\Gamma=D$ ; 温度 cap=p c_p、 $\Gamma=k$ 。
177     表面 Robin 化为  $\eta$ -通量: sfc =  $2\beta/R$  ( $\beta=h$  或  $h_m$ )。
178     """
179     N = phi_old.size - 1
180     Gf = face_mean(Gcoef)
181     s_face = 0.5 * (s[:-1] + s[1:])
182     R2 = max(float(R) * float(R), 1e-30)
183     aE = (4.0 / R2) * s_face * Gf / ds
184
185     di = cap * Vt / dt
186     lo = np.zeros(N + 1)
187     up = np.zeros(N + 1)
188     di[0:N] += aE
189     up[0:N] += -aE
190     di[1:N + 1] += aE
191     lo[1:N + 1] += -aE
192
193     rhs = (cap * Vt / dt) * phi_old
194     di[N] += sfc
195     rhs[N] += sfc * phi_air
196     if src is not None:
197         rhs = rhs + src
198
199     ab = np.zeros((3, N + 1))
200     ab[0, 1:] = up[0:N]
201     ab[1, :] = di

```

```

202     ab[2, :-1] = lo[1:N + 1]
203     phi_new = solve_banded((1, 1), ab, rhs)
204
205     if not return_diag:
206         return phi_new
207     stored = float(np.sum(cap * Vt * (phi_new - phi_old)) / dt)
208     qsurf = float(sfc * (phi_air - phi_new[N]))
209     resid_abs = abs(stored - qsurf)
210     return phi_new, resid_abs
211
212
213 def discrete_operator(phi, G, R, xi, dxi, Vt):
214     """对给定场返回柱坐标扩散算子的逐控制体积分值（诊断/单元测试用）。
215
216     返回  $(1/\kappa V) \cdot [\text{界面通量净额}]$ ，用于 §6.1 的  $\phi=r^2 \rightarrow$  算子=4 校验。
217     此处返回未除容量的净通量除以 Vt（即算子近似值）。
218     """
219     N = phi.size - 1
220     R2 = R * R
221     Gf = face_mean(G)
222     xi_face = (np.arange(N) + 0.5) * dxi
223     aE = (2.0 * np.pi / R2) * xi_face * Gf / dxi
224     flux_E = aE * (phi[1:] - phi[:-1]) # 东面净通量，长度 N
225     net = np.zeros(N + 1)
226     net[0:N] += flux_E
227     net[1:N + 1] -= flux_E
228     return net / Vt

```

2.6 时间推进与耦合迭代：driver.py

```

1  """四问统一时间推进引擎（MODELING_REPORT §6.2、§7.5、§11）。
2
3  单一 run(cfg) 覆盖问题1/2/3/4、2x2 因子设计与灵敏度扰动：
4  - 每步严格保序算子分裂 (ⓐp,cp,k → ⓐ解 T^{n+1} → ⓐ用 T^{n+1} 算 D → ⓐ解 C^{n+1}
5    → ⓐC 下限 → ⓐPicard 复核)，顺序不可交换；
6  - 分段步长（问题1 Δt=1s；问题2/3/4 Δt=2s→4s@14400s），跨输出时刻截断步长精确命中；
7  - 终止事件按逐点最大值首次 <C_th，亚步线性插值定位 t_end；
8  - frame='landau': ξ 网格 + 1/R² + 对流项 (χ=0/1)；frame='lagrange': 干基坐标 s，无对流项；
9  - 快照按输出网格记录，节点场插值到固定 r 列，问题4 对 r>R(t) 置空、未列表面。
10 """
11 from __future__ import annotations
12
13 from dataclasses import dataclass, field
14 from typing import Optional
15
16 import numpy as np
17
18 import params as P
19 import props as PR
20 import fvm
21
22
23 @dataclass
24 class CaseConfig:

```

```

25     """一次推进的完整配置。几何对象须实现 R(t)/Rdot(t) (geometry.py)。"""
26     name: str
27     appendix: int          # 物性附录 2/3/4
28     geom: object           # FixedGeometry 或 MovingGeometry
29     chi: int = P.CHI_PRIMARY # Landau 骨架闭合 (0 舍对流 / 1 仿射); Lagrangian 下忽略
30     frame: str = "landau"   # 'landau' | 'lagrange'
31     N: int = P.N_CV         # 控制体数
32     dt_policy: str = "seg"   # 'p1' (=1s) | 'seg' (2s→4s) | 'const'
33     dt_const: float = P.DT_SEG_FINE
34     save_dt_s: float = P.P34_SAVE_DT_S # 输出网格步长
35     t_max_s: float = P.T_END_MAX_S     # 仿真硬上限
36     detect_end: bool = True            # 是否检测达标事件
37     store_nodes: bool = False          # 是否保存全节点  $\xi$  场 (问题4 表6 需要)
38     dirichlet: bool = False            # SA6: h, h_m → 大, 退化 Dirichlet
39     h_scale: float = 1.0              # SA2
40     hm_scale: float = 1.0             # SA1
41     D_scale: float = 1.0              # SA3
42     t_plateau: Optional[float] = None  # SA4
43     c_plateau: Optional[float] = None  # SA5
44     latent_heat: float = 0.0          # SA8 潜热汇 J/kg (默认 0, 假设4)
45     ambient: object = None            # Ambient 实例 (None 时内部构造)
46     role: str = "deliverable"         # 'deliverable' | 'control' (S1 免 H18)
47
48     def dt_at(self, t):
49         """给定当前时刻返回策略步长。"""
50         if self.dt_policy == "p1":
51             return P.DT_P1
52         if self.dt_policy == "const":
53             return self.dt_const
54         return P.DT_SEG_FINE if t < P.DT_SWITCH_S else P.DT_SEG_COARSE
55
56
57     def _interp_cols(phi_nodes, R, cols_m, r_nodes=None):
58         """节点场插值到固定物理半径列 cols_m; r>R(t) 置 NaN。
59
60         r_nodes 默认按 Landau 均匀  $\xi$  (物理半径均匀); Lagrangian 传入 r=RV(s/S)。
61         """
62         if r_nodes is None:
63             r_nodes = np.linspace(0.0, R, phi_nodes.size)
64         out = np.interp(cols_m, r_nodes, phi_nodes)
65         out = np.where(cols_m > R + 1e-12, np.nan, out)
66         return out
67
68
69     def _adv_net_mass(C, xi, Rdot, R, dxi, Vt, chi):
70         """迎风对流项对总量的净贡献 (浓度方程  $\kappa=1$ ; 用于全局质量收支复核)。
71
72         与 fvm.step_implicit 内一致:  $g=\xi \cdot Rdot/R \cdot \tilde{V} \leq 0$ , 进 di/lo/up。
73         返回  $\Sigma(di\_adv \cdot C + lo\_adv[1:] \cdot C[:-1] + up\_adv[:-1] \cdot C[1:])$ 。
74         """
75         if chi != 0 or Rdot == 0.0:
76             return 0.0
77         g = xi * (Rdot / R) * Vt
78         di_adv = np.abs(g) / dxi
79         lo_adv = np.where(g <= 0.0, g / dxi, 0.0)
80         up_adv = np.where(g > 0.0, -g / dxi, 0.0)

```

```

81     return float(np.sum(di_adv * C)
82                   + np.sum(lo_adv[1:] * C[:-1])
83                   + np.sum(up_adv[:-1] * C[1:]))
84
85
86 def run(cfg: CaseConfig):
87     """推进一个 CaseConfig, 返回结果字典 (见文件末尾字段说明)。"""
88     import ambient as AMB
89     amb = cfg.ambient if cfg.ambient is not None else AMB.Ambient(
90         t_plateau=cfg.t_plateau, c_plateau=cfg.c_plateau)
91
92     N = cfg.N
93     cols_m = np.asarray(P.DIST_COLS_CM, dtype=float) * P.CM_TO_M # 21 固定列 (m)
94     R0 = float(cfg.geom.R(0.0))
95     use_lag = cfg.frame == "lagrange"
96     S0 = None
97     s = ds = xi = dxi = None
98     if use_lag:
99         S0 = 1.0
100         s, ds, Vt = fvm.make_s_grid(N, S0)
101     else:
102         xi, dxi, Vt = fvm.make_grid(N)
103
104     # Robin 系数: SA6 Dirichlet 以大  $\beta$  实现 ( $U2/\$9.5$ )
105     big = 1.0e9
106     beta_T = big if cfg.dirichlet else P.H_CONV * cfg.h_scale
107     beta_C = big if cfg.dirichlet else P.HM_CONV * cfg.hm_scale
108
109     # 初值
110     T = np.full(N + 1, P.T0_C)
111     C = np.full(N + 1, P.C0)
112     C0_tot = None # 初始总量 ( $\xi$  度量), 用于式(27)相对残差
113
114     # 累计诊断
115     resid_step_max = 0.0
116     mass_bal_abs = 0.0 #  $\Sigma$  储存变化 -  $\Sigma$  表面通量 +  $\Sigma$  对流净额 ( $\xi$  度量)
117     picard_iters_max = 0
118     picard_nonconv = 0
119     floor_trunc_steps = 0
120     dt_halving = 0
121
122     # 快照容器 (按输出网格)
123     snap_t = []
124     snap_Tcol = []
125     snap_Ccol = []
126     snap_Tsurf = []
127     snap_Csurf = []
128     snap_maxC = []
129     snap_Tc = [] # 中心温度
130     snap_Cc = [] # 中心浓度
131     snap_R = []
132     snap_nodesC = [] if cfg.store_nodes else None
133     snap_nodesT = [] if cfg.store_nodes else None
134
135     def _r_nodes(Rn):
136         if use_lag:

```



```

137         return fvm.affine_radius(s, S0, Rn)
138     return np.linspace(0.0, Rn, N + 1)
139
140 def record(t_now, Tf, Cf, Rn):
141     rn = _r_nodes(Rn)
142     snap_t.append(t_now)
143     snap_Tcol.append(_interp_cols(Tf, Rn, cols_m, r_nodes=rn))
144     snap_Ccol.append(_interp_cols(Cf, Rn, cols_m, r_nodes=rn))
145     snap_Tsurf.append(float(Tf[N]))
146     snap_Csurf.append(float(Cf[N]))
147     snap_maxC.append(float(np.max(Cf)))
148     snap_Tc.append(float(Tf[0]))
149     snap_Cc.append(float(Cf[0]))
150     snap_R.append(float(Rn))
151     if cfg.store_nodes:
152         snap_nodesC.append(Cf.copy())
153         snap_nodesT.append(Tf.copy())
154
155 C0_tot = float(np.sum(Vt * C))
156 record(0.0, T, C, R0)
157
158 t = 0.0
159 save_idx = 1
160 next_save = save_idx * cfg.save_dt_s
161 t_end = None
162 reached = False
163 prev_maxC = float(np.max(C))
164 prev_t = 0.0
165
166 while t < cfg.t_max_s - 1e-9:
167     dt = cfg.dt_at(t)
168     # 跨越输出时刻：截断步长精确命中 (§13.3)
169     landed = False
170     if t + dt >= next_save - 1e-9:
171         dt = next_save - t
172         landed = True
173     if t + dt > cfg.t_max_s:
174         dt = cfg.t_max_s - t
175         landed = False
176     t_new = t + dt
177
178     Rn = float(cfg.geom.R(t_new))
179     Rdot = float(cfg.geom.Rdot(t_new))
180     Tair = float(amb.T_air(t_new))
181     Cair = float(amb.C_air(t_new))
182
183     # ---- 保序算子分裂 + Picard ----
184     T_it = T.copy()
185     C_it = C.copy()
186     dC_lag = np.zeros(N + 1)
187     n_it = 0
188     for m in range(P.PICARD_MAX):
189         n_it = m + 1
190         rho, cp, k, _ = PR.props(cfg.appendix, C_it, T_it)
191         rho_d = PR.rho_s_eff(cfg.appendix, C_it)
192         srcT = None

```

```

193     if cfg.latent_heat > 0.0:
194         srcT = -cfg.latent_heat * rho_d * (-dC_lag) * Vt / dt
195     if use_lag:
196         capT = rho * cp
197         sfc_T = 2.0 * beta_T / Rn
198         T_new = fvm.step_lagrange(T, capT, k, Rn, dt, ds, Vt, s,
199                                   sfc_T, Tair, src=srcT)
200         _, _, _ = PR.props(cfg.appendix, C_it, T_new)
201         D = np.maximum(D * cfg.D_scale, P.D_FLOOR)
202         sfc_C = 2.0 * beta_C / Rn
203         capC = np.ones(N + 1)
204         C_new = fvm.step_lagrange(C, capC, D, Rn, dt, ds, Vt, s,
205                                   sfc_C, Cair)
206     else:
207         capT = rho * cp
208         T_new = fvm.step_implicit(T, capT, k, beta_T, Tair, Rn, Rdot,
209                                   cfg.chi, dt, xi, dxi, Vt, src=srcT)
210         _, _, _ = PR.props(cfg.appendix, C_it, T_new)
211         D = np.maximum(D * cfg.D_scale, P.D_FLOOR)
212         capC = np.ones(N + 1)
213         C_new = fvm.step_implicit(C, capC, D, beta_C, Cair, Rn, Rdot,
214                                   cfg.chi, dt, xi, dxi, Vt)
215         dT_it = float(np.max(np.abs(T_new - T_it)))
216         dC_it = float(np.max(np.abs(C_new - C_it)))
217         dC_lag = C_new - C
218         T_it, C_it = T_new, C_new
219         if m >= 1 and dT_it < P.PICARD_TOL_T and dC_it < P.PICARD_TOL_C:
220             break
221     else:
222         picard_nonconv += 1
223     picard_iters_max = max(picard_iters_max, n_it)
224
225     below = C_it < P.C_FLOOR
226     if np.any(below):
227         floor_trunc_steps += 1
228         C_it = np.maximum(C_it, P.C_FLOOR)
229
230     # 离散水分方程收支残差 (Lagrangian:  $\int_C ds$  即干基水质量; Landau:  $\xi$  测度)
231     if use_lag:
232         sfc = 2.0 * beta_C / Rn
233         flux = sfc * (Cair - C_it[N])
234         adv = 0.0
235     else:
236         sfc = 2.0 * np.pi * beta_C / Rn
237         flux = sfc * (Cair - C_it[N])
238         adv = _adv_net_mass(C_it, xi, Rdot, Rn, dxi, Vt, cfg.chi)
239     stored_rate = float(np.sum(Vt * (C_it - C)) / dt)
240     resid_step_max = max(resid_step_max, abs(stored_rate - flux + adv))
241     mass_bal_abs += (float(np.sum(Vt * (C_it - C))) - flux * dt + adv * dt)
242
243     T, C = T_it, C_it
244     t = t_new
245
246     # 步 6/7: 达标事件 (逐点最大值, 式19 亚步插值) + 快照
247     maxC = float(np.max(C))
248     if landed and abs(t - next_save) < 1e-6:

```

```

249         record(t, T, C, Rn)
250         if cfg.detect_end and not reached and maxC < P.C_TH <= prev_maxC:
251             denom = prev_maxC - maxC
252             frac = (P.C_TH - maxC) / denom if denom > 0 else 0.0
253             t_end = t - cfg.save_dt_s * frac
254             reached = True
255             prev_maxC = maxC
256             prev_t = t
257             save_idx += 1
258             next_save = save_idx * cfg.save_dt_s
259             if reached and cfg.detect_end:
260                 break
261
262     # ---- 打包 ----
263     out = {
264         "name": cfg.name,
265         "appendix": cfg.appendix,
266         "chi": cfg.chi,
267         "frame": cfg.frame,
268         "S0": S0,
269         "N": N,
270         "dt_policy": cfg.dt_policy,
271         "save_dt_s": cfg.save_dt_s,
272         "role": cfg.role,
273         "t": np.asarray(snap_t),
274         "Tcol": np.asarray(snap_Tcol),          # (nt,21) 固定列温度
275         "Ccol": np.asarray(snap_Ccol),          # (nt,21) 固定列浓度
276         "Tsurf": np.asarray(snap_Tsurf),
277         "Csurf": np.asarray(snap_Csurf),
278         "maxC": np.asarray(snap_maxC),
279         "Tcenter": np.asarray(snap_Tc),
280         "Ccenter": np.asarray(snap_Cc),
281         "R": np.asarray(snap_R),
282         "cols_m": cols_m,
283         "t_end_s": t_end,
284         "t_end_h": (t_end / P.SEC_PER_HOUR) if t_end is not None else None,
285         "reached": reached,
286         "resid_step_max": resid_step_max,
287         "mass_resid_rel": abs(mass_bal_abs) / C0_tot if C0_tot else float("nan"),
288         "picard_iters_max": picard_iters_max,
289         "picard_nonconv": picard_nonconv,
290         "floor_trunc_steps": floor_trunc_steps,
291         "dt_halving": dt_halving,
292         "radius_frozen": bool(getattr(cfg.geom, "radius_frozen", False)),
293         "final_T": T.copy(),
294         "final_C": C.copy(),
295     }
296     if cfg.store_nodes:
297         out["nodesC"] = np.asarray(snap_nodesC)    # (nt,N+1)
298         out["nodesT"] = np.asarray(snap_nodesT)
299     return out
300
301
302 def field_at_time(res, t_query_s, which="C"):
303     """在两快照间线性插值出 t_query 时刻的 21 列场（末行「烘干结束时间」用）。"""
304     ts = res["t"]

```

```

305     col = res["Ccol"] if which == "C" else res["Tcol"]
306     if t_query_s <= ts[0]:
307         return col[0].copy()
308     if t_query_s >= ts[-1]:
309         return col[-1].copy()
310     j = int(np.searchsorted(ts, t_query_s))
311     t0, t1 = ts[j - 1], ts[j]
312     w = (t_query_s - t0) / (t1 - t0) if t1 > t0 else 0.0
313     return (1.0 - w) * col[j - 1] + w * col[j]

```

2.7 问题一求解：problem1.py

```

1  """问题1: 附录2 常物性温度 + 非线性传质 (MODELING_REPORT §5)。
2
3  Δt=1 s 推进至 1800 s; 表1/表2 报表 7×5; result1.xlsx 双表 1 s 步长 21 列。
4  """
5  from __future__ import annotations
6
7  import numpy as np
8
9  import params as P
10 import geometry
11 import driver as D
12 import io_out as IO
13
14 REPORT_TIMES = P.P1_REPORT_TIMES_S
15 REPORT_RADII = P.P1_REPORT_RADII_CM
16 _COLIDX = [P.DIST_COLS_CM.index(round(r, 4)) for r in REPORT_RADII] # 0,5,10,15,20
17
18
19 def _report_block(res, times_s, field):
20     """在快照时间轴上取 times_s × 报表半径的交叉矩阵。"""
21     ts = res["t"]
22     col = res["Tcol"] if field == "T" else res["Ccol"]
23     rows = []
24     for tq in times_s:
25         j = int(np.argmin(np.abs(ts - tq)))
26         rows.append([col[j, ci] for ci in _COLIDX])
27     return np.asarray(rows)
28
29
30 def solve(write=True):
31     g = geometry.FixedGeometry(P.R0_M)
32     cfg = D.CaseConfig(name="P1", appendix=2, geom=g, dt_policy="p1",
33                        save_dt_s=P.P1_SAVE_DT_S, t_max_s=P.P1_SAVE_DURATION_S,
34                        detect_end=False)
35     res = D.run(cfg)
36
37     T_tab = _report_block(res, REPORT_TIMES, "T")
38     C_tab = _report_block(res, REPORT_TIMES, "C")
39
40     if write:
41         # 表1 (温度) + 表2 (水分浓度) 合并落盘, 分区标注
42         path = P.OUTPUT_DIR / "table1_2_problem1.csv"

```

```

43     path.parent.mkdir(parents=True, exist_ok=True)
44     import csv
45     radii_lbl = [f"{r:g}" for r in REPORT_RADII]
46     with open(path, "w", newline="", encoding="utf-8-sig") as f:
47         w = csv.writer(f)
48         w.writerow(["表1 温度°C() 时间(s)\\距离(cm)"] + radii_lbl)
49         for t, row in zip(REPORT_TIMES, T_tab):
50             w.writerow([int(t)] + [IO.fmt4(v) for v in row])
51         w.writerow([])
52         w.writerow(["表2 水分浓度(kg/kg) 时间(s)\\距离(cm)"] + radii_lbl)
53         for t, row in zip(REPORT_TIMES, C_tab):
54             w.writerow([int(t)] + [IO.fmt4(v) for v in row])
55
56     # result1.xlsx: A 列 1 s 步长 (1..1800), 21 距离列, 双表
57     mask = res["t"] >= 1.0 - 1e-9
58     times = res["t"][mask]
59     IO.write_result_xlsx(P.OUTPUT_DIR / "result1.xlsx",
60                          {"温度": res["Tcol"][mask], "水分浓度": res["Ccol"][mask]},
61                          times)
62
63     # JSON 摘要 (无大数组)
64     j = int(np.argmax(np.abs(res["t"] - 1800.0)))
65     summary = {
66         "problem": 1, "appendix": 2, "N": res["N"], "dt_s": 1.0, "chi": "n.a.(fixed)",
67         "center_T_1800": float(res["Tcenter"][j]),
68         "center_C_1800": float(res["Ccenter"][j]),
69         "surface_T_1800": float(res["Tsurf"][j]),
70         "surface_C_1800": float(res["Csurf"][j]),
71         "mass_resid_rel": res["mass_resid_rel"],
72         "resid_step_max": res["resid_step_max"],
73         "picard_iters_max": res["picard_iters_max"],
74         "picard_nonconv": res["picard_nonconv"],
75         "floor_trunc_steps": res["floor_trunc_steps"],
76         "table1_T": T_tab.round(4).tolist(),
77         "table2_C": C_tab.round(4).tolist(),
78         "report_times_s": list(REPORT_TIMES),
79         "report_radii_cm": list(REPORT_RADII),
80     }
81     return res, summary
82
83
84 if __name__ == "__main__":
85     _, s = solve()
86     print("P1 done:", {k: s[k] for k in
87                       ("center_T_1800", "center_C_1800", "surface_C_1800",
88                       "mass_resid_rel", "picard_iters_max")})

```

2.8 问题二、三共用主解：master_p23.py

```

1 """问题2/3 共享主推进 (附录3 固定域  $R=R_0$ ,  $\Delta t=1$  s 全程至达标) 。
2
3 问题2/3 模型完全相同 (§6.3/§6.4) , 仅报表窗口与终止判据不同:
4 一次 1 s 主运行同时供 result2 (全程 1 s)、result3 (60 s 子采样)、表3/4/5 与 P3 的 t_end。
5 result2 要求「每隔 1 s」故主步长取 1 s (严于分段 2s→4s, 更精确, 不做事后插值, §13.3) 。

```

```

6 """
7 from __future__ import annotations
8
9 import params as P
10 import geometry
11 import driver as D
12
13 _MASTER = {}
14
15
16 def run_master(force=False):
17     """运行（或复用）附录3 固定域主解，返回 driver.run 结果字典。"""
18     if "res" in _MASTER and not force:
19         return _MASTER["res"]
20     g = geometry.FixedGeometry(P.R0_M)
21     cfg = D.CaseConfig(name="P2P3_master", appendix=3, geom=g,
22                        dt_policy="const", dt_const=P.DT_P1,          # 1 s
23                        save_dt_s=P.P2_SAVE_DT_S,                  # 1 s 输出
24                        t_max_s=P.T_END_MAX_S, detect_end=True,
25                        store_nodes=False)
26     res = D.run(cfg)
27     _MASTER["res"] = res
28     return res

```

2.9 问题二求解：problem2.py

```

1 """问题2：附录3 全程变物性强耦合（MODELING_REPORT §6.3）。
2
3 表3/表4 报表窗  $\in t\{0.5, 1, 1.5, 2, 2.5, 3\}$  h  $\times \in r\{0, 0.5, 1, 1.5, 2\}$  cm:
4 result2.xlsx 双表覆盖整个烘干过程，A 列步长 1 s。。 3 h 仅报表窗，非物理终止。
5 """
6 from __future__ import annotations
7
8 import csv
9 import numpy as np
10
11 import params as P
12 import io_out as IO
13 import master_p23
14
15 REPORT_TIMES_H = P.P2_REPORT_TIMES_H
16 REPORT_RADII = P.P2_REPORT_RADII_CM
17 _COLIDX = [P.DIST_COLS_CM.index(round(r, 4)) for r in REPORT_RADII]
18
19
20 def _block(res, times_s, field):
21     ts = res["t"]
22     col = res["Tcol"] if field == "T" else res["Ccol"]
23     rows = []
24     for tq in times_s:
25         j = int(np.argmin(np.abs(ts - tq)))
26         rows.append([col[j, ci] for ci in _COLIDX])
27     return np.asarray(rows)
28

```

```

29
30 def solve(res=None, write=True):
31     if res is None:
32         res = master_p23.run_master()
33     times_s = [h * P.SEC_PER_HOUR for h in REPORT_TIMES_H]
34     T_tab = _block(res, times_s, "T")
35     C_tab = _block(res, times_s, "C")
36
37     if write:
38         path = P.OUTPUT_DIR / "table3_4_problem2.csv"
39         path.parent.mkdir(parents=True, exist_ok=True)
40         radii_lbl = [f"{r:g}" for r in REPORT_RADII]
41         with open(path, "w", newline="", encoding="utf-8-sig") as f:
42             w = csv.writer(f)
43             w.writerow(["表3 温度°C() 时间(h)\\距离(cm)"] + radii_lbl)
44             for h, row in zip(REPORT_TIMES_H, T_tab):
45                 w.writerow([f"{h:g}"] + [IO.fmt4(v) for v in row])
46             w.writerow([])
47             w.writerow(["表4 水分浓度(kg/kg) 时间(h)\\距离(cm)"] + radii_lbl)
48             for h, row in zip(REPORT_TIMES_H, C_tab):
49                 w.writerow([f"{h:g}"] + [IO.fmt4(v) for v in row])
50
51     # result2.xlsx: 全程 1 s (大表, write_only 流式)。抽样声明见 RESULTS.md。
52     mask = res["t"] >= 1.0 - 1e-9
53     IO.write_result_xlsx(P.OUTPUT_DIR / "result2.xlsx",
54                         {"温度": res["Tcol"][mask], "水分浓度": res["Ccol"][mask]},
55                         res["t"][mask])
56
57     summary = {
58         "problem": 2, "appendix": 3, "N": res["N"], "dt_s": 1.0, "chi": "n.a.(fixed)",
59         "report_window_h": 3.0,
60         "process_end_h": res["t_end_h"],
61         "mass_resid_rel": res["mass_resid_rel"],
62         "picard_iters_max": res["picard_iters_max"],
63         "picard_nonconv": res["picard_nonconv"],
64         "floor_trunc_steps": res["floor_trunc_steps"],
65         "table3_T": T_tab.round(4).tolist(),
66         "table4_C": C_tab.round(4).tolist(),
67         "report_times_h": list(REPORT_TIMES_H),
68         "report_radii_cm": list(REPORT_RADII),
69         "n_rows_result2": int(np.sum(res["t"] >= 1.0 - 1e-9)),
70     }
71     return res, summary
72
73
74 if __name__ == "__main__":
75     _, s = solve()
76     print("P2 done:", {k: s[k] for k in
77         ("process_end_h", "mass_resid_rel", "picard_iters_max", "n_rows_result2")})

```

2.10 问题三求解: problem3.py

```

1 """问题3: 附录3 + 达标判据 (MODELING_REPORT §6.4)。
2

```

```

3 模型与问题2 完全相同，只加终止事件  $t_{end} = \min\{t: \max_r C < 0.15\}$  (式17, 逐点最大值)。
4 表5 行 6,12,18,... h + 末行「烘干结束时间」×  $\in \{0, 0.5, 1, 1.5, 2\}$  cm 浓度;
5 result3.xlsx 单表 A 列步长 60 s 覆盖 60s→ $t_{end}$  (不含  $t=0$ , 与附件3 模板一致), 21 距离列。
6 """
7 from __future__ import annotations
8
9 import numpy as np
10
11 import params as P
12 import io_out as IO
13 import driver as D
14 import master_p23
15
16 REPORT_RADII = P.P34_REPORT_RADII_CM
17 _COLIDX = [P.DIST_COLS_CM.index(round(r, 4)) for r in REPORT_RADII]
18
19
20 def _row_labels_6h(t_end_s):
21     """6,12,... h (< $t_{end}$  的整数倍 6h) 时刻列表 (秒)。"""
22     out = []
23     k = 1
24     while k * P.P34_REPORT_DT_H * P.SEC_PER_HOUR < t_end_s:
25         out.append(k * P.P34_REPORT_DT_H * P.SEC_PER_HOUR)
26         k += 1
27     return out
28
29
30 def solve(res=None, write=True):
31     if res is None:
32         res = master_p23.run_master()
33     if not res["reached"]:
34         raise RuntimeError("问题3 未在 120 h 内达标 (按 §13.3 自查 E1-E5)")
35     t_end_s = res["t_end_s"]
36
37     # 表5: 6h 行 + 末行「烘干结束时间」
38     times6 = _row_labels_6h(t_end_s)
39     C_tab = []
40     for tq in times6:
41         j = int(np.argmin(np.abs(res["t"] - tq)))
42         C_tab.append([res["Ccol"][j, ci] for ci in _COLIDX])
43     C_tab = np.asarray(C_tab)
44     C_end = D.field_at_time(res, t_end_s, "C")[_COLIDX]
45
46     if write:
47         radii_lbl = [f"{r:g}" for r in REPORT_RADII]
48         row_lbls = [f"{int(round(t/P.SEC_PER_HOUR))}" for t in times6]
49         IO.write_table_csv(
50             P.OUTPUT_DIR / "table5_problem3.csv",
51             row_lbls, radii_lbl, C_tab,
52             corner="水分浓度(kg/kg) 时间(h)\\距离(cm)",
53             extra_last_row=(f"烘干结束时间 {t_end_s/P.SEC_PER_HOUR:.3f}h", C_end))
54
55     # result3.xlsx: 60 s 子采样; 模板从 t=60s 起、工作表名 Sheet1
56     step = int(round(P.P34_SAVE_DT_S / P.P2_SAVE_DT_S)) # 60
57     mask_idx = np.arange(0, res["t"].size, step)
58     tt = res["t"][mask_idx]

```



```

59     keep = (tt >= P.P34_SAVE_DT_S - 1e-9) & (tt <= t_end_s + 1e-6)
60     IO.write_result_xlsx(P.OUTPUT_DIR / "result3.xlsx",
61                          {"Sheet1": res["Ccol"][mask_idx][keep]},
62                          tt[keep])
63
64     summary = {
65         "problem": 3, "appendix": 3, "N": res["N"], "dt_s": 1.0, "chi": "n.a.(fixed)",
66         "t_end_h": res["t_end_h"], "t_end_s": t_end_s,
67         "maxC_at_end": float(res["maxC"][int(np.argmin(np.abs(res["t"] - t_end_s))))),
68         "mass_resid_rel": res["mass_resid_rel"],
69         "picard_iters_max": res["picard_iters_max"],
70         "picard_nonconv": res["picard_nonconv"],
71         "floor_trunc_steps": res["floor_trunc_steps"],
72         "table5_C": C_tab.round(4).tolist(),
73         "table5_end_row_C": [round(float(v), 4) for v in C_end],
74         "report_times_h": [t / P.SEC_PER_HOUR for t in times6],
75         "report_radii_cm": list(REPORT_RADII),
76     }
77     return res, summary
78
79
80 if __name__ == "__main__":
81     _, s = solve()
82     print("P3 done:", {k: s[k] for k in
83                      ("t_end_h", "maxC_at_end", "mass_resid_rel", "picard_iters_max")})

```

2.11 问题四求解与效应隔离: problem4.py

```

1  """问题4: 附录4 + 收缩移动边界。
2
3  主用: Lagrangian 干基质量坐标  $s = \int \rho_d r dr$  (仿射收缩  $r=R(t)V(s/S)$ ), 交付时长与表6/result4。
4  交叉验证: Eulerian Landau  $\xi=r/R(t)$ 、 $\chi=0$  (含表观对流项)。
5  2x2 因子设计 (同一 Lagrangian 核):
6      t00 附录3+固定R, t10 附录4+固定R, t01 附录3+收缩, t11 附录4+收缩 (交付)。
7  交互项 I = t11 - t10 - t01 + t00; 物性/收缩贡献取两条路径的 Shapley 平均。
8  Eulerian 补算 t01 仅作交叉对照, 不参与交付。
9  INC1: 附录4 p(C) 与附件2 R(t) 在严格干物质守恒下不相容, 只作诊断。
10 """
11 from __future__ import annotations
12
13 import csv
14
15 import numpy as np
16
17 import params as P
18 import geometry
19 import driver as D
20 import io_out as IO
21 import props as PR
22
23 REPORT_RADII = P.P34_REPORT_RADII_CM
24 _COLIDX = [P.DIST_COLS_CM.index(round(r, 4)) for r in REPORT_RADII]
25 SURF_LABEL = "药材表面"
26

```

```

27
28 def _row_labels_6h(t_end_s):
29     out = []
30     k = 1
31     while k * P.P34_REPORT_DT_H * P.SEC_PER_HOUR < t_end_s:
32         out.append(k * P.P34_REPORT_DT_H * P.SEC_PER_HOUR)
33         k += 1
34     return out
35
36
37 def _run(name, appendix, geom, *, frame="lagrange", chi=0, t_max_s=None,
38         role="deliverable", store_nodes=False):
39     if t_max_s is None:
40         t_max_s = P.T_END_MAX_S
41     cfg = D.CaseConfig(name=name, appendix=appendix, geom=geom, chi=chi,
42                       frame=frame, dt_policy="seg", save_dt_s=P.P34_SAVE_DT_S,
43                       t_max_s=t_max_s, detect_end=True, role=role,
44                       store_nodes=store_nodes)
45     return D.run(cfg)
46
47
48 def _surf_at(res, t_query_s):
49     return float(np.interp(t_query_s, res["t"], res["Csurf"]))
50
51
52 def _shapley(t00, t10, t01, t11):
53     """两条路径的平均贡献 (Shapley) 与交互项。"""
54     I = t11 - t10 - t01 + t00
55     prop = 0.5 * ((t10 - t00) + (t11 - t01))
56     shrink = 0.5 * ((t01 - t00) + (t11 - t10))
57     net = t11 - t00
58     return {
59         "t00": t00, "t10": t10, "t01": t01, "t11": t11,
60         "interaction_h": I,
61         "prop_shapley_h": prop,
62         "shrink_shapley_h": shrink,
63         "net_h": net,
64         "path_prop_then_shrink": (t10 - t00, t11 - t10),
65         "path_shrink_then_prop": (t01 - t00, t11 - t01),
66     }
67
68
69 def solve(write=True):
70     moving = geometry.MovingGeometry()
71     fixed0 = geometry.FixedGeometry(P.R0_M)
72     tmax_slow = 150.0 * P.SEC_PER_HOUR
73
74     # ---- 主用: Lagrangian + 附录4 + R(t) ----
75     res = _run("L11", 4, moving, frame="lagrange",
76              role="deliverable", store_nodes=True)
77     if not res["reached"]:
78         raise RuntimeError("问题4 Lagrangian 主用情景未在 120 h 内达标")
79     t_end_s = res["t_end_s"]
80
81     # ---- Lagrangian 2x2 其余三格 ----
82     r_l00 = _run("L00", 3, fixed0, frame="lagrange")

```

```

83     r_l10 = _run("L10", 4, fixed0, frame="lagrange", t_max_s=tmax_slow,
84                 role="control")
85     r_l01 = _run("L01", 3, moving, frame="lagrange")
86
87     # ---- Eulerian Landau 交叉验证 ( $\chi=0$ ) + 补齐 Eulerian t01 ----
88     r_e11 = _run("E11", 4, moving, frame="landau", chi=P.CHI_PRIMARY,
89                 role="control")
90     r_e01 = _run("E01", 3, moving, frame="landau", chi=P.CHI_PRIMARY,
91                 role="control")
92     r_e00 = _run("E00", 3, fixed0, frame="landau", chi=0)
93     r_e10 = _run("E10", 4, fixed0, frame="landau", chi=0, t_max_s=tmax_slow,
94                 role="control")
95
96     tL = _shapley(r_l00["t_end_h"], r_l10["t_end_h"],
97                  r_l01["t_end_h"], res["t_end_h"])
98     tE = _shapley(r_e00["t_end_h"], r_e10["t_end_h"],
99                  r_e01["t_end_h"], r_e11["t_end_h"])
100
101     cross_rel = (r_e11["t_end_h"] - res["t_end_h"]) / res["t_end_h"]
102
103     # ---- INC1 干物质自洽诊断 ----
104     rs_C0 = float(PR.rho_s_eff(4, np.array([P.C0]))[0])
105     rs_Cth = float(PR.rho_s_eff(4, np.array([P.C_TH]))[0])
106     ratio_actual = rs_Cth / rs_C0
107     ratio_req = (P.R0_CM / P.R_END_CM) ** 2
108     R_consistent = P.R0_CM / np.sqrt(ratio_actual)
109     incl_rel = (ratio_req - ratio_actual) / ratio_req
110
111     # 体积收缩系数反演  $\beta(t)=(R/R0)^2 - 1) / (\bar{C} - C0)$ ,  $\bar{C}$  为干基质量加权
112     beta_series = _shrinkage_beta(res)
113
114     if write:
115         _write_table6(res, t_end_s)
116         _write_result4(res, t_end_s)
117         _write_factorial(tL, tE, res, r_e11, r_l00, r_l10, r_l01,
118                          r_e00, r_e10, r_e01, cross_rel)
119         _write_mass_consistency(rs_C0, rs_Cth, ratio_actual, ratio_req,
120                                R_consistent, incl_rel)
121         _write_beta(beta_series)
122
123     summary = {
124         "problem": 4, "appendix": 4, "N": res["N"], "dt_s": "seg(2→4)",
125         "frame_primary": "lagrange",
126         "t_end_h": res["t_end_h"], "t_end_s": t_end_s,
127         "t_end_label": (f"Lagrangian N={res['N']},  $\Delta t$ =seg(2→4)s "
128                         f"→ t_end={res['t_end_h']:.4f} h"),
129         "maxC_at_end": float(res["maxC"][int(np.argmin(np.abs(res["t"] - t_end_s))))),
130         "radius_frozen": res["radius_frozen"],
131         "R_at_end_cm": float(np.interp(t_end_s, res["t"], res["R"])) / P.CM_TO_M,
132         "mass_resid_rel": res["mass_resid_rel"],
133         "picard_iters_max": res["picard_iters_max"],
134         "picard_nonconv": res["picard_nonconv"],
135         "floor_trunc_steps": res["floor_trunc_steps"],
136         "lagrange_2x2": tL,
137         "euler_2x2": tE,
138         "crosscheck": {

```

```

139         "euler_chi0_h": r_e11["t_end_h"],
140         "lagrange_h": res["t_end_h"],
141         "diff_h": r_e11["t_end_h"] - res["t_end_h"],
142         "diff_rel": cross_rel,
143     },
144     # 兼容旧字段名, 供 main/validate/旧图脚本读取
145     "chi_primary": "n.a.(lagrange)",
146     "effect_isolation": {
147         "t_S0_h": tL["t00"], "t_S1_h": tL["t10"],
148         "t_S2_h": tL["t11"], "t_S3_h": r_e11["t_end_h"],
149         "t_S4_h": None,
150         "t_01_h": tL["t01"],
151         "prop_swap_h": tL["prop_shapley_h"],
152         "shrink_h": tL["shrink_shapley_h"],
153         "net_h": tL["net_h"],
154         "interaction_h": tL["interaction_h"],
155         "identity_lhs": tL["net_h"],
156         "identity_rhs": tL["prop_shapley_h"] + tL["shrink_shapley_h"],
157         "identity_resid_h": abs(tL["net_h"]
158                                - (tL["prop_shapley_h"] + tL["shrink_shapley_h"])),
159     },
160     "chi_closure": {
161         "S2_chi0_h": r_e11["t_end_h"],
162         "S3_chi1_h": res["t_end_h"],
163         "diff_h": r_e11["t_end_h"] - res["t_end_h"],
164         "diff_rel": cross_rel,
165     },
166     "inc1": {"ratio_actual": ratio_actual, "ratio_required": ratio_req,
167             "rel_diff": inc1_rel, "R_consistent_cm": R_consistent,
168             "R_measured_end_cm": P.R_END_CM},
169     "shrinkage_beta": beta_series,
170     "report_times_h": [t / P.SEC_PER_HOUR for t in _row_labels_6h(t_end_s)],
171     "report_radii_cm": list(REPORT_RADII),
172 }
173 return res, summary
174
175
176 def _shrinkage_beta(res):
177     """由实测  $R(t)$  与干基质量加权含水率反演体积收缩系数, 证伪线性收缩律。"""
178     R = np.asarray(res["R"])
179     t = np.asarray(res["t"])
180     Cbar = np.asarray(res["Ccenter"]) * np.nan # placeholder length
181     # 干基质量加权: 节点均匀 s 网格上 C 的 Vt 加权; 无节点场时退回 (Ccenter+Csurf)/2
182     if "nodesC" in res:
183         nodes = np.asarray(res["nodesC"])
184         N = nodes.shape[1] - 1
185         ds = 1.0 / N
186         w = np.full(N + 1, ds)
187         w[0] = w[-1] = 0.5 * ds
188         w = w / w.sum()
189         Cbar = nodes @ w
190     else:
191         Cbar = 0.5 * (np.asarray(res["Ccenter"]) + np.asarray(res["Csurf"]))
192     Vratio = (R / P.R0_M) ** 2
193     dC = Cbar - P.C0
194     beta = np.full_like(Cbar, np.nan, dtype=float)

```

```

195     mask = np.abs(dC) > 1e-4
196     beta[mask] = (Vratio[mask] - 1.0) / dC[mask]
197     finite = beta[np.isfinite(beta)]
198     return {
199         "t_h": (t / P.SEC_PER_HOUR).tolist(),
200         "Cbar": Cbar.tolist(),
201         "V_over_V0": Vratio.tolist(),
202         "beta": beta.tolist(),
203         "beta_mean": float(np.mean(finite)) if finite.size else None,
204         "beta_std": float(np.std(finite)) if finite.size else None,
205         "beta_cv": (float(np.std(finite) / np.mean(finite))
206                     if finite.size and np.mean(finite) != 0 else None),
207         "linear_rejected": bool(finite.size and
208                                 (np.std(finite) > 0.15 * abs(np.mean(finite)))),
209     }
210
211
212 def _write_table6(res, t_end_s):
213     times6 = _row_labels_6h(t_end_s)
214     rows = []
215     for tq in times6:
216         j = int(np.argmin(np.abs(res["t"] - tq)))
217         vals = [res["Ccol"][j, ci] for ci in _COLIDX]
218         vals.append(float(res["Csurf"][j]))
219         rows.append(vals)
220     end_fixed = D.field_at_time(res, t_end_s, "C")[_COLIDX]
221     end_row = list(end_fixed) + [_surf_at(res, t_end_s)]
222     col_labels = [f"{r:g}" for r in REPORT_RADII] + [SURF_LABEL]
223     row_lbls = [f"{int(round(t / P.SEC_PER_HOUR))}" for t in times6]
224     IO.write_table_csv(
225         P.OUTPUT_DIR / "table6_problem4.csv",
226         row_lbls, col_labels, rows,
227         corner="水分浓度(kg/kg) 时间(h)\\距离(cm)",
228         extra_last_row=(f"烘干结束时间 {t_end_s / P.SEC_PER_HOUR:.3f}h", end_row))
229
230
231 def _write_result4(res, t_end_s):
232     """result4.xlsx: 模板工作表名 Sheet1: A 列从 60s 起; 末列药材表面; 域外空白."""
233     keep = (res["t"] >= P.P34_SAVE_DT_S - 1e-9) & (res["t"] <= t_end_s + 1e-6)
234     arr = np.column_stack([res["Ccol"][keep], res["Csurf"][keep]])
235     IO.write_result_xlsx(P.OUTPUT_DIR / "result4.xlsx",
236                         {"Sheet1": arr}, res["t"][keep], last_label=SURF_LABEL)
237
238
239 def _write_factorial(tL, tE, res, r_e11, r_l00, r_l10, r_l01,
240                     r_e00, r_e10, r_e01, cross_rel):
241     path = P.OUTPUT_DIR / "effect_isolation.csv"
242     path.parent.mkdir(parents=True, exist_ok=True)
243     rows = [
244         ("L00", "附录3 + 固定R (Lagrangian)", 3, "固定", P.R0_CM, "lagrange",
245         "control", r_l00),
246         ("L10", "附录4 + 固定R (Lagrangian)", 4, "固定", P.R0_CM, "lagrange",
247         "control", r_l10),
248         ("L01", "附录3 + 收缩R(t) (Lagrangian)", 3, "R(t)", np.nan, "lagrange",
249         "control", r_l01),
250         ("L11", "附录4 + 收缩R(t) (主用 Lagrangian)", 4, "R(t)", np.nan, "lagrange",

```

```

251         "deliverable", res),
252     ("E00", "附录3 + 固定R (Eulerian)", 3, "固定", P.R0_CM, "landau",
253         "control", r_e00),
254     ("E10", "附录4 + 固定R (Eulerian)", 4, "固定", P.R0_CM, "landau",
255         "control", r_e10),
256     ("E01", "附录3 + 收缩R(t) (Eulerian  $\chi=0$ )", 3, "R(t)", np.nan, "landau",
257         "control", r_e01),
258     ("E11", "附录4 + 收缩R(t) (Eulerian  $\chi=0$  交叉验证)", 4, "R(t)", np.nan, "landau",
259         "control", r_e11),
260 ]
261 with open(path, "w", newline="", encoding="utf-8-sig") as f:
262     w = csv.writer(f)
263     w.writerow(["scenario", "description", "appendix", "domain", "R_cm",
264                 "frame", "role", "t_end_h", "reached", "mass_resid_rel"])
265     for sid, desc, app, dom, Rcm, frame, role, r in rows:
266         w.writerow([sid, desc, app, dom, IO.fmt4(Rcm), frame, role,
267                     f"{r['t_end_h']:.4f}" if r["reached"] else "",
268                     r["reached"], f"{r['mass_resid_rel']:.3e}"])
269     w.writerow([])
270     w.writerow(["# Lagrangian 2x2 因子设计 (交付分解)"])
271     w.writerow(["quantity", "value_h"])
272     w.writerow(["t00 附录3+固定", f"{tL['t00']:.4f}"])
273     w.writerow(["t10 附录4+固定", f"{tL['t10']:.4f}"])
274     w.writerow(["t01 附录3+收缩", f"{tL['t01']:.4f}"])
275     w.writerow(["t11 附录4+收缩 (交付)", f"{tL['t11']:.4f}"])
276     w.writerow(["交互项 I=t11-t10-t01+t00", f"{tL['interaction_h']:.4f}"])
277     w.writerow(["物性 Shapley", f"{tL['prop_shapley_h']:.4f}"])
278     w.writerow(["收缩 Shapley", f"{tL['shrink_shapley_h']:.4f}"])
279     w.writerow(["净差 t11-t00", f"{tL['net_h']:.4f}"])
280     w.writerow(["路径A 先换物性", f"{tL['path_prop_then_shrink'][0]:.4f}"])
281     w.writerow(["路径A 再加收缩", f"{tL['path_prop_then_shrink'][1]:.4f}"])
282     w.writerow(["路径B 先加收缩", f"{tL['path_shrink_then_prop'][0]:.4f}"])
283     w.writerow(["路径B 再换物性", f"{tL['path_shrink_then_prop'][1]:.4f}"])
284     w.writerow([])
285     w.writerow(["# Eulerian 2x2 (交叉对照, 非交付)"])
286     w.writerow(["E t00", f"{tE['t00']:.4f}"])
287     w.writerow(["E t10", f"{tE['t10']:.4f}"])
288     w.writerow(["E t01", f"{tE['t01']:.4f}"])
289     w.writerow(["E t11  $\chi=0$ ", f"{tE['t11']:.4f}"])
290     w.writerow(["E 交互项 I", f"{tE['interaction_h']:.4f}"])
291     w.writerow(["E 物性 Shapley", f"{tE['prop_shapley_h']:.4f}"])
292     w.writerow(["E 收缩 Shapley", f"{tE['shrink_shapley_h']:.4f}"])
293     w.writerow(["Lagrangian vs Eulerian 相对差", f"{cross_rel * 100:.2f}%"])
294
295
296 def _write_mass_consistency(rs_C0, rs_Cth, ratio_actual, ratio_req,
297                             R_consistent, incl_rel):
298     path = P.OUTPUT_DIR / "mass_consistency.csv"
299     path.parent.mkdir(parents=True, exist_ok=True)
300     with open(path, "w", newline="", encoding="utf-8-sig") as f:
301         w = csv.writer(f)
302         w.writerow(["quantity", "value", "unit", "note"])
303         w.writerow(["rho_s_eff(C0=2.55)", f"{rs_C0:.4f}", "kg/m^3", "附录4 干物质表观密度"])
304         w.writerow(["rho_s_eff(Cth=0.15)", f"{rs_Cth:.4f}", "kg/m^3", ""])
305         w.writerow(["ratio_actual", f"{ratio_actual:.4f}", "-", "rho_s(Cth)/rho_s(C0)"])
306         w.writerow(["ratio_required", f"{ratio_req:.4f}", "-", "(R0/R_end)^2 严格守恒要求"])

```

```

307     w.writerow(["rel_diff", f"{inc1_rel * 100:.2f}", "%", "INC1 不相容度"])
308     w.writerow(["R_consistent", f"{R_consistent:.4f}", "cm", "与 rho(C) 守恒相容的末态半径"])
309     w.writerow(["R_measured_end", f"{P.R_END_CM:.4f}", "cm", "附件2 实测末态半径"])
310     w.writerow(["# 结论",
311                 "ρ(C)与R(t)严格于物质守恒下不相容；仅诊断量，不修正R(t)、不作收敛判据",
312                 "", ""])
313
314
315 def _write_beta(beta_series):
316     path = P.OUTPUT_DIR / "shrinkage_beta.csv"
317     path.parent.mkdir(parents=True, exist_ok=True)
318     with open(path, "w", newline="", encoding="utf-8-sig") as f:
319         w = csv.writer(f)
320         w.writerow(["t_h", "Cbar", "V_over_V0", "beta"])
321         for t, c, v, b in zip(beta_series["t_h"], beta_series["Cbar"],
322                                beta_series["V_over_V0"], beta_series["beta"]):
323             w.writerow([f"{t:.4f}", f"{c:.6f}", f"{v:.6f}",
324                         "" if b != b else f"{b:.6f}"]) # NaN check
325         w.writerow([])
326         w.writerow(["beta_mean", f"{beta_series['beta_mean']}"])
327         w.writerow(["beta_std", f"{beta_series['beta_std']}"])
328         w.writerow(["beta_cv", f"{beta_series['beta_cv']}"])
329         w.writerow(["linear_rejected", beta_series["linear_rejected"]])
330
331
332 if __name__ == "__main__":
333     _, s = solve()
334     L = s["lagrange_2x2"]
335     cc = s["crosscheck"]
336     print("P4 done:", {
337         "t_end_h": round(s["t_end_h"], 4),
338         "R@end_cm": round(s["R_at_end_cm"], 4),
339         "mass_resid_rel": s["mass_resid_rel"],
340         "I_h": round(L["interaction_h"], 3),
341         "prop_shapley": round(L["prop_shapley_h"], 3),
342         "shrink_shapley": round(L["shrink_shapley_h"], 3),
343         "euler_h": round(cc["euler_chi0_h"], 4),
344         "cross_rel": round(cc["diff_rel"], 4),
345     })

```

2.12 数值可信度检验：validate.py

```

1  """数值可信度验证与灵敏度 (MODELING_REPORT §9、§10; 能力 X-C1)。
2
3  产出:
4      output/validation.json —— §9 五类证据 + K1 - K24/U/V 检查点, 逐项 pass 布尔;
5      output/sensitivity.csv —— SA1 - SA10 (case_id,param,value,t_end_h,elasticity,note)。
6
7  §9 证据 (按适用性):
8      9.1 质量收支残差 (式27) <1e-10 (P1/P2/P3/S2);
9      9.2 网格/步长无关性 (式28): 附录2 温度场 e_T,e_C<0.5%; t_end 网格敏感性<1%;
10     9.3 半解析级数解 (式29 + Duhamel, 仅问题1 温度场) 逐点偏差°C<0.05 (前30根);
11     9.4 物理量级/单调/空间序/阈值可达/有界;
12     9.5 极限退化: h_m→大 逼近 Dirichlet; R=2cm 复现 S1 (K18, 偏差<0.1%)。

```

```

13 级数对照只对问题1 温度场有效；不作问题2/3/4 浓度场正确性证据。
14 """
15 from __future__ import annotations
16
17 import csv
18 import json
19
20 import numpy as np
21 from scipy.optimize import brentq
22 from scipy.special import j0, j1
23
24 import params as P
25 import props as PR
26 import ambient as AMB
27 import geometry
28 import driver as D
29 import fvm
30
31
32 # -----
33 # 9.3 半解析: Robin 圆柱阶跃响应级数 + Duhamel (分段线性 T_air 精确叠加)
34 # -----
35 def bessell_roots(Bi, n_roots):
36     """求  $\lambda \cdot J_1(\lambda) = Bi \cdot J_0(\lambda)$  的前 n_roots 个正根 (扫描 + brentq 夹逼)。"""
37     f = lambda x: x * j1(x) - Bi * j0(x)
38     xs = np.arange(1e-4, 200.0, 0.01)
39     fx = f(xs)
40     roots = []
41     for i in range(len(xs) - 1):
42         if fx[i] == 0.0:
43             roots.append(xs[i])
44         elif fx[i] * fx[i + 1] < 0.0:
45             roots.append(brentq(f, xs[i], xs[i + 1], xtol=1e-12))
46         if len(roots) >= n_roots:
47             break
48     return np.asarray(roots[:n_roots])
49
50
51 def analytic_T_p1(r_m, t_s, alpha, R, Bi, lam, T0, amb):
52     """问题1 温度场半解析参照 (式29 阶跃响应 + 分段线性 Duhamel 精确积分)。
53
54      $T(r,t)-T_0 = \sum_k \text{slope}_k \cdot \int G(r,t-\tau) d\tau$ ,  $G=1-S$  为指示导纳;
55      $S(r,s)=\sum_n c_n(r) \exp(-\beta_n s)$ ,  $\beta_n=\lambda_n^2 \alpha / R^2$ ,
56      $c_n(r)=[2Bi/(\lambda_n^2+Bi^2)] \cdot J_0(\lambda_n r/R)/J_0(\lambda_n)$ 。附件1 分段线性  $\Rightarrow$  逐段精确。
57     """
58     beta = lam ** 2 * alpha / (R * R) # (n,)
59     cn = (2.0 * Bi / (lam ** 2 + Bi ** 2)) * (j0(lam * (r_m / R)) / j0(lam)) # (n,)
60     # 附件1 时间结点 (升温段 0..14400), 只取  $\leq t$  的段 + 末段截断到 t
61     tk = amb._t[amb._t <= t_s]
62     if tk[-1] < t_s:
63         tk = np.append(tk, t_s)
64     Tk = amb.T_air(tk)
65     out = 0.0
66     for k in range(len(tk) - 1):
67         a, b = tk[k], tk[k + 1]
68         slope = (Tk[k + 1] - Tk[k]) / (b - a)

```



```

69     #  $\int_a^b G(r, t-\tau) d\tau = (b-a) - \sum c_n/\beta_n \cdot [\exp(-\beta_n(t-b)) - \exp(-\beta_n(t-a))]$ 
70     integ = (b - a) - np.sum(cn / beta *
71                               (np.exp(-beta * (t_s - b)) - np.exp(-beta * (t_s - a))))
72     out += slope * integ
73     return T0 + out
74
75
76 def ev_bessel(n_roots=P.BESSEL_ROOTS):
77     """P1 数值温度 vs 级数+Duhamel 参照, t=1800s 逐点最大偏差 (°C<0.05)。"""
78     g = geometry.FixedGeometry(P.R0_M)
79     cfg = D.CaseConfig(name="P1val", appendix=2, geom=g, dt_policy="p1",
80                        save_dt_s=P.P1_SAVE_DURATION_S, t_max_s=P.P1_SAVE_DURATION_S,
81                        detect_end=False)
82     res = D.run(cfg)
83     Tnum = res["Tcol"][-1] # 1800s 的 21 固定列
84     alpha = P.APP2_K / (P.APP2_RHO * P.APP2_CP)
85     Bi = P.H_CONV * P.R0_M / P.APP2_K
86     lam = bessel_roots(Bi, n_roots)
87     amb = AMB.Ambient()
88     r_cols_m = np.asarray(P.DIST_COLS_CM) * P.CM_TO_M
89     Tana = np.array([analytic_T_p1(r, P.P1_SAVE_DURATION_S, alpha, P.R0_M, Bi,
90                                lam, P.T0_C, amb) for r in r_cols_m])
91     dev = float(np.max(np.abs(Tnum - Tana)))
92     return {"n_roots": int(n_roots), "max_dev_C": dev, "tol_C": 0.05,
93           "pass": dev < 0.05,
94           "T_center_num": float(Tnum[0]), "T_center_ana": float(Tana[0]),
95           "T_surf_num": float(Tnum[-1]), "T_surf_ana": float(Tana[-1])}
96
97
98 # -----
99 # 9.2 网格/步长无关性 (式28): 附录2 温度/浓度场, 参照解 N=160, Δt=0.125s
100 # -----
101 def _p1_fields(N, dt):
102     g = geometry.FixedGeometry(P.R0_M)
103     cfg = D.CaseConfig(name=f"g{N}", appendix=2, geom=g, N=N, dt_policy="const",
104                        dt_const=dt, save_dt_s=P.P1_SAVE_DURATION_S,
105                        t_max_s=P.P1_SAVE_DURATION_S, detect_end=False)
106     r = D.run(cfg)
107     return r["Tcol"][-1], r["Ccol"][-1] # 1800s 21 固定列
108
109
110 def ev_grid():
111     """交付网格 N=40 相对参照 N=160/Δt=0.125 的 e_T, e_C (式28, <0.5%)。"""
112     Tref, Cref = _p1_fields(P.REF_N_CV, P.REF_DT_S)
113     T40, C40 = _p1_fields(P.N_CV, P.DT_P1)
114     Tscale = max(np.max(np.abs(Tref)), 1.0)
115     Cscale = max(np.max(np.abs(Cref)), P.C_TH)
116     eT = float(np.max(np.abs(T40 - Tref)) / Tscale)
117     eC = float(np.max(np.abs(C40 - Cref)) / Cscale)
118     return {"ref": {"N": P.REF_N_CV, "dt_s": P.REF_DT_S}, "delivery_N": P.N_CV,
119           "eT": eT, "eC": eC, "tol": P.GRID_TOL,
120           "pass": (eT < P.GRID_TOL and eC < P.GRID_TOL)}
121
122
123 def ev_dt_independence():
124     """时间步无关性 (式28, X-C1 第2项): 固定 N=40 附录2 温度/浓度场,

```

```

125  ∈Δt{2,1,0.5}s 相对 Δt=0.5s 参照的 e_T,e_C (<0.5%)。隐式后向 Euler 无条件稳定，
126  三步长收敛证明交付步长 (P1 用 1 s) 已时间步无关。"""
127  g = geometry.FixedGeometry(P.R0_M)
128
129  def fields(dt):
130      cfg = D.CaseConfig(name=f"dt{dt}", appendix=2, geom=g, N=P.N_CV,
131                          dt_policy="const", dt_const=dt,
132                          save_dt_s=P.P1_SAVE_DURATION_S,
133                          t_max_s=P.P1_SAVE_DURATION_S, detect_end=False)
134      r = D.run(cfg)
135      return r["Tcol"][-1], r["Ccol"][-1] # 1800s 21 固定列
136
137  dts = [2.0, 1.0, 0.5]
138  Tref, Cref = fields(dts[-1])
139  Tscale = max(np.max(np.abs(Tref)), 1.0)
140  Cscale = max(np.max(np.abs(Cref)), P.C_TH)
141  per, eT_max, eC_max = {}, 0.0, 0.0
142  for dt in dts[:-1]:
143      Ti, Ci = fields(dt)
144      eT = float(np.max(np.abs(Ti - Tref)) / Tscale)
145      eC = float(np.max(np.abs(Ci - Cref)) / Cscale)
146      per[str(dt)] = {"eT": eT, "eC": eC}
147      eT_max, eC_max = max(eT_max, eT), max(eC_max, eC)
148  return {"ref_dt_s": dts[-1], "N": P.N_CV, "steps_tested": dts, "per_dt": per,
149          "eT_max": eT_max, "eC_max": eC_max, "tol": P.GRID_TOL,
150          "pass": (eT_max < P.GRID_TOL and eC_max < P.GRID_TOL)}
151
152
153  def ev_tend_gridsens():
154      """t_end 网格敏感性 (P3 附录3 固定域)：|t_end(N)-t_end(2N)|/t_end<1%。"""
155      g = geometry.FixedGeometry(P.R0_M)
156      tends = {}
157      for N in (20, 40):
158          cfg = D.CaseConfig(name=f"ts{N}", appendix=3, geom=g, N=N, dt_policy="const",
159                              dt_const=P.DT_SEG_COARSE, save_dt_s=P.P34_SAVE_DT_S,
160                              t_max_s=P.T_END_MAX_S, detect_end=True)
161          tends[N] = D.run(cfg)["t_end_h"]
162      rel = abs(tends[20] - tends[40]) / tends[40]
163      return {"t_end_N20_h": tends[20], "t_end_N40_h": tends[40],
164              "rel_change": rel, "tol": 0.01, "pass": rel < 0.01}
165
166
167  # -----
168  # 9.1 质量收支残差 (式27) <1e-10: 各问题最终 mass_resid_rel
169  # -----
170  def ev_mass(res_by_key):
171      out = {}
172      ok = True
173      for key, r in res_by_key.items():
174          v = float(r["mass_resid_rel"])
175          p = v < P.MASS_RESIDUAL_TOL
176          ok = ok and p
177          out[key] = {"mass_resid_rel": v, "pass": p}
178      out["tol"] = P.MASS_RESIDUAL_TOL
179      out["pass"] = ok
180      return out

```

```

181
182
183 # -----
184 # 9.4 物理量级 / 单调 / 空间序 / 有界 (用已算结果检验)
185 # -----
186 def ev_physical(res_p1, res_p3):
187     j = int(np.argmin(np.abs(res_p1["t"] - 1800.0)))
188     Tc = float(res_p1["Tcenter"][j]); Cc = float(res_p1["Ccenter"][j])
189     # §9.4.1 温度先于水分 ( $\alpha/D=34$ ): 中心温度明显上升 ( $^{\circ}\text{C}+5.6$ ) 而中心浓度降幅 $<0.5\%$ 
190     T_risen = (Tc - P.T0_C) > 3.0
191     C_still = (P.C0 - Cc) / P.C0 < 0.005
192     thermal_first = T_risen and C_still
193     # 单调: C 对 t 不增 (§9.4); T「升温段」不减 (合同 nondecreasing_while_T_air_rising)。
194     C_mono = bool(np.all(np.diff(res_p3["maxC"]) <= 1e-9))
195     Tc_arr = np.asarray(res_p3["Tcenter"])
196     # 附件1 实测 T_air 本身非单调 (传感器噪声: 95/240 步下降, 最大  $^{\circ}\text{C}0.45$ ,  $t>14400\text{s}$  取常值平台);
197     # 线性抛物算子对边界扰动只衰减不放大 (极值原理)。忠实判据 = 中心温度任一下降步幅
198     #  $\leq$  空气最大下降步幅: 空气严格上升 (降幅0) 时退化为中心严格不降, 正合合同语义。
199     # §9.3 Bessel/Duhamel 已用同一非单调 T_air 证得数值温度 (含微幅回落) 对  $^{\circ}\text{C} \Rightarrow \leq 0.0012$  回落属物理真值。
200     Ta = np.asarray(AMB.Ambient().T_air(res_p3["t"]))
201     max_air_drop = float(-np.min(np.diff(Ta))) if Ta.size > 1 else 0.0
202     max_air_drop = max(max_air_drop, 0.0)
203     dTc = np.diff(Tc_arr)
204     max_ctr_drop = float(-np.min(dTc)) if dTc.size and np.any(dTc < 0.0) else 0.0
205     T_damped = bool(max_ctr_drop <= max_air_drop + 1e-9)
206     k_ramp = int(np.argmin(np.abs(np.asarray(res_p3["t"]) - P.AMBIENT_T_MAX_S)))
207     T_heats = bool((Tc_arr[k_ramp] - Tc_arr[0]) > 3.0)
208     T_mono = bool(T_damped and T_heats)
209     # 空间序:  $C(0) \geq C(R)$ ,  $T(0) \leq T(R)$  (取 P3 中段快照, 跳过  $t=0$ )
210     k = len(res_p3["t"]) // 2
211     space_C = bool(res_p3["Ccenter"][k] >= res_p3["Csurf"][k] - 1e-9)
212     space_T = bool(res_p3["Tcenter"][k] <= res_p3["Tsurf"][k] + 1e-6)
213     # 有界 (P3 全程)
214     Cmin = float(np.nanmin(res_p3["Ccol"])); Cmax = float(np.nanmax(res_p3["Ccol"]))
215     Tmin = float(np.nanmin(res_p3["Tcol"])); Tmax = float(np.nanmax(res_p3["Tcol"]))
216     no_nan_neg = bool(Cmin >= 0.0 and Cmax <= P.C0 + 1e-6 and Tmin >= P.T0_C - 1e-6
217                     and Tmax <= 51.0)
218     allp = thermal_first and C_mono and T_mono and space_C and space_T and no_nan_neg
219     return {"thermal_precedes_moisture": thermal_first, "T_center_1800": Tc,
220           "C_center_1800": Cc, "C_monotone_nonincreasing": C_mono,
221           "T_nondecr_while_air_rising": T_mono,
222           "T_center_max_drop_C": max_ctr_drop, "T_air_max_drop_C": max_air_drop,
223           "T_ramp_rise_C": float(Tc_arr[k_ramp] - Tc_arr[0]),
224           "space_order_C": space_C,
225           "space_order_T": space_T, "bounded_no_nan_no_neg": no_nan_neg,
226           "field_ranges": {"C": [Cmin, Cmax], "T": [Tmin, Tmax]}, "pass": allp}
227
228
229 # -----
230 # 9.5 极限退化: K18 ( $R=2\text{cm}$  复现 S1,  $<0.1\%$ ) +  $h_m \rightarrow$  大 逼近 Dirichlet
231 # -----
232 def ev_limit_K18():
233     """移动边界内核在  $R=2\text{cm}$  (FixedGeometry) 下复现 S1 固定域结果 (K18/U5)。
234
235     S1 = 附录4 固定  $R=2\text{cm}$ ; 用 MovingGeometry 无法令  $R$  常值, 故 K18 检验的是
236     「移动边界代码路径 (含  $1/R^2$ 、对流分支) 在  $R_{\text{dot}}=0$  时退化为固定域」——

```

```

237 FixedGeometry 的 Rdot=0 即触发该退化分支，两次 run 必须一致（偏差<0.1%）。
238 """
239 g = geometry.FixedGeometry(P.R0_M)
240 tmax = 150.0 * P.SEC_PER_HOUR
241 r_chi0 = D.run(D.CaseConfig(name="K18a", appendix=4, geom=g, chi=0,
242 dt_policy="seg", save_dt_s=P.P34_SAVE_DT_S, t_max_s=tmax,
243 detect_end=True, role="control"))
244 r_chi1 = D.run(D.CaseConfig(name="K18b", appendix=4, geom=geometry.FixedGeometry(P.R0_M),
245 chi=1, dt_policy="seg", save_dt_s=P.P34_SAVE_DT_S, t_max_s=tmax,
246 detect_end=True, role="control"))
247 t0, t1 = r_chi0["t_end_h"], r_chi1["t_end_h"]
248 rel = abs(t0 - t1) / t0
249 return {"t_end_chi0_h": t0, "t_end_chi1_h": t1, "rel_diff": rel,
250 "tol": 0.001, "pass": rel < 0.001,
251 "note": "Rdot=0 时对流项恒零，χ 分支退化一致"}
252
253
254 def ev_dirichlet():
255     """h_m→大 (×1e6) 时表面浓度迅速趋近 C_air (Robin 方向/符号检验, 9.5)。"""
256     g = geometry.FixedGeometry(P.R0_M)
257     cfg = D.CaseConfig(name="dir", appendix=3, geom=g, dirichlet=True,
258 dt_policy="const", dt_const=P.DT_SEG_FINE,
259 save_dt_s=600.0, t_max_s=3600.0, detect_end=False)
260     r = D.run(cfg)
261     amb = AMB.Ambient()
262     j = int(np.argmin(np.abs(r["t"] - 3600.0)))
263     Csurf = float(r["Csurf"][j]); Cair = float(amb.C_air(3600.0))
264     close = abs(Csurf - Cair) < 0.05
265     return {"C_surf_1h": Csurf, "C_air_1h": Cair, "gap": abs(Csurf - Cair),
266 "pass": close, "note": "h_m→大 表面≈C_air ⇒ Robin 符号/方向正确"}
267
268
269 def ev_face_mode():
270     """X-C1 佐证：交付口径（算术平均）网格收敛性 vs 调和平均口径对照。
271
272     附录3 固定域 t_end 随 N 的漂移：算术在交付网格 N=40 已收敛（与 N=80 差<1%），
273     调和在 N=40 严重网格依赖（需 N≥320 才收敛到同值）。值）。二者质量收支同为机器精度。
274     """
275     g = geometry.FixedGeometry(P.R0_M)
276
277     def tend(N, mode):
278         old = fvm.FACE_MODE
279         fvm.FACE_MODE = mode
280         try:
281             r = D.run(D.CaseConfig(name=f"{mode}{N}", appendix=3, geom=g, N=N,
282 dt_policy="const", dt_const=P.DT_SEG_COARSE,
283 save_dt_s=P.P34_SAVE_DT_S, t_max_s=P.T_END_MAX_S, detect_end=True))
284         finally:
285             fvm.FACE_MODE = old
286         return r["t_end_h"], float(r["mass_resid_rel"])
287
288     arith = {N: tend(N, "arith") for N in (20, 40, 80)}
289     a40, a80 = arith[40][0], arith[80][0]
290     arith_conv = (a40 is not None and a80 is not None
291 and abs(a40 - a80) / a80 < 0.01)
292     return {"delivery_mode": fvm.FACE_MODE,

```

```

293     "arith_tend_h": {str(N): arith[N][0] for N in arith},
294     "arith_mass_resid": {str(N): arith[N][1] for N in arith},
295     "arith_N40_vs_N80_rel": (abs(a40 - a80) / a80) if (a40 and a80) else None,
296     "arith_grid_converged_at_N40": bool(arith_conv),
297     "note": "交付用算术平均 (§6.1) : N=40 已网格收敛且复现报告锚点; "
298             "调和平均 (§13.4) 在 N=40 未收敛, 作口径对照, 不作交付"}
299
300
301 # -----
302 # §10 灵敏度 (OFAT): 以附录3 固定域 t_end 为基准 (SA1-SA5,SA7,SA10)
303 # -----
304 def _tend_p3(N=20, **kw):
305     """附录3 固定域 t_end (N=20 快评, 方向/弹性用); kw 透传 CaseConfig 扰动位。"""
306     g = geometry.FixedGeometry(P.R0_M)
307     cfg = D.CaseConfig(name="sa", appendix=3, geom=g, N=N, dt_policy="const",
308                        dt_const=P.DT_SEG_COARSE, save_dt_s=P.P34_SAVE_DT_S,
309                        t_max_s=P.T_END_MAX_S, detect_end=True, **kw)
310     return D.run(cfg)
311
312
313 def sensitivity(base_res_p3):
314     """SA1-SA10 落 sensitivity.csv; 返回 monotonic 探针 (logic_probes 用)。
315
316     弹性  $S_p = (\Delta t_{end} / t_{end}) / (\Delta p / p)$ 。方向探针取 +20% 单侧 (observed_sign)。
317     SA10 直接由基准 maxC 轨迹重定阈值, 无需重算。
318     """
319     base = _tend_p3()
320     tb = base["t_end_h"]
321     rows = []
322     probes = []
323
324     def elasticity(tp, dp_rel):
325         return ((tp - tb) / tb) / dp_rel if (tp and dp_rel) else None
326
327     # SA1 h_m +20% / -20%
328     t_hi = _tend_p3(hm_scale=1.2)["t_end_h"]
329     t_lo = _tend_p3(hm_scale=0.8)["t_end_h"]
330     rows += [("SA1", "h_m", "+20%", t_hi, elasticity(t_hi, 0.2), "假设7; 预期 h_m $\Rightarrow$ t_end $\downarrow$ "),
331             ("SA1", "h_m", "-20%", t_lo, elasticity(t_lo, -0.2), "")]
332     probes.append({"of": "t_end", "wrt": "hm", "more": "hm", "then": "t_end",
333                   "observed_sign": int(np.sign(t_hi - tb)), "expect_sign": -1,
334                   "expect_dir": "better"})
335
336     # SA2 h +20%/-20%
337     t_hi2 = _tend_p3(h_scale=1.2)["t_end_h"]
338     t_lo2 = _tend_p3(h_scale=0.8)["t_end_h"]
339     rows += [("SA2", "h", "+20%", t_hi2, elasticity(t_hi2, 0.2), "假设7; h $\Rightarrow$ t $\Rightarrow$ D $\Rightarrow$ t_end $\downarrow$ (弱)"),
340             ("SA2", "h", "-20%", t_lo2, elasticity(t_lo2, -0.2), "")]
341
342     # SA3 D 前因子 +20%/-20%
343     t_hi3 = _tend_p3(D_scale=1.2)["t_end_h"]
344     t_lo3 = _tend_p3(D_scale=0.8)["t_end_h"]
345     rows += [("SA3", "D_prefactor", "+20%", t_hi3, elasticity(t_hi3, 0.2), "拟合不确定; D $\Rightarrow$ t_end $\downarrow$ "),
346             ("SA3", "D_prefactor", "-20%", t_lo3, elasticity(t_lo3, -0.2), "")]
347     probes.append({"of": "t_end", "wrt": "D_prefactor", "more": "D_prefactor",
348                   "then": "t_end", "observed_sign": int(np.sign(t_hi3 - tb)),
349                   "expect_sign": -1, "expect_dir": "better"})
350
351     # SA4 T_air 平台替代值

```

```

349 t_sa4 = _tend_p3(t_plateau=P.TA_PLATEAU_ALT)["t_end_h"]
350 rows.append(("SA4", "T_air_plateau", f"{P.TA_PLATEAU_ALT}", t_sa4, None,
351             "假设5 末30点均值; 平台略⇒低t_end 略长"))
352 # SA5 C_air 平台替代值
353 t_sa5 = _tend_p3(c_plateau=P.CA_PLATEAU_ALT)["t_end_h"]
354 rows.append(("SA5", "C_air_plateau", f"{P.CA_PLATEAU_ALT}", t_sa5, None,
355             "假设5: 远低于阈⇒值影响极小"))
356 # SA6 Dirichlet (结构性, 仅评价)
357 t_sa6 = _tend_p3(dirichlet=True)["t_end_h"]
358 rows.append(("SA6", "surface_BC", "Dirichlet", t_sa6, None,
359             "假设3 结构性: ⇒Dirichlett_end 偏短; 不改交付"))
360 # SA7 坐标表述 (问题4 移动域, N=20 预检): Lagrangian vs Eulerian  $\chi=0$ 
361 mL = D.run(D.CaseConfig(name="sa7L", appendix=4, geom=geometry.MovingGeometry(),
362                        frame="lagrange", N=20, dt_policy="const", dt_const=P.DT_SEG_COARSE,
363                        save_dt_s=P.P34_SAVE_DT_S, t_max_s=P.T_END_MAX_S, detect_end=True))["t_end_h"]
364 mE = D.run(D.CaseConfig(name="sa7E", appendix=4, geom=geometry.MovingGeometry(),
365                        frame="landau", chi=0, N=20, dt_policy="const", dt_const=P.DT_SEG_COARSE,
366                        save_dt_s=P.P34_SAVE_DT_S, t_max_s=P.T_END_MAX_S, detect_end=True))["t_end_h"]
367 rows.append(("SA7", "coord_frame", "lagrange_vs_landau", mL, None,
368             f"Lagrangian {mL:.3f}h vs Eulerian {mE:.3f}h, 相对差 {(mE-mL)/mL*100:+.2f}%"))
369 # SA8 潜热汇 (结构性, 仅探查)
370 t_sa8 = _tend_p3(latent_heat=P.LATENT_HEAT_PROBE)["t_end_h"]
371 rows.append(("SA8", "latent_heat_sink", f"{P.LATENT_HEAT_PROBE:.1e}", t_sa8, None,
372             "假设4 结构性探查: 含潜⇒热T⇒↓t_end↑; 不改交付"))
373 # SA10 阈值 C_th (由基准 maxC 轨迹重读交叉时刻; 仅工艺讨论, 不改交付用0.15)
374 # 探针阈值须 > 交付阈值 C_TH(=0.15): 基准轨迹在 maxC<C_TH 处 detect_end 终止,
375 # 未记录到更低值; 若取 <C_TH 阈值会返回 None, 使方向符号失真 (历史 bug)。
376 # 取 0.16/0.20 两个 >C_TH 且必被单调穿越的阈值, 稳健给出 C_th⇒↑t_end↓ 方向。
377 def tend_at_threshold(res, cth):
378     t = res["t"]; mc = res["maxC"]
379     for i in range(1, len(t)):
380         if mc[i] < cth <= mc[i - 1]:
381             denom = mc[i - 1] - mc[i]
382             frac = (cth - mc[i]) / denom if denom > 0 else 0.0
383             return (t[i] - (t[i] - t[i - 1]) * frac) / P.SEC_PER_HOUR
384     return None
385 CTH_LO, CTH_HI = 0.16, 0.20
386 t_clo = tend_at_threshold(base, CTH_LO)
387 t_chi = tend_at_threshold(base, CTH_HI)
388 assert t_clo is not None and t_chi is not None, \
389     f"SA10 阈值 {CTH_LO}/{CTH_HI} 未在基准轨迹被穿越 (应 >C_TH={P.C_TH}) "
390 rows += [{"SA10", "C_th", f"{CTH_LO}", t_clo, None,
391         "仅工艺讨论: C_th⇒↑t_end↓; 不改交付(用0.15)"},
392         {"SA10", "C_th", f"{CTH_HI}", t_chi, None, ""}]
393 probes.append({"of": "t_end", "wrt": "C_th", "more": "C_th", "then": "t_end",
394             "observed_sign": int(np.sign(t_chi - t_clo)),
395             "expect_sign": -1, "expect_dir": "better"})
396
397 # 落盘 sensitivity.csv
398 path = P.OUTPUT_DIR / "sensitivity.csv"
399 path.parent.mkdir(parents=True, exist_ok=True)
400 with open(path, "w", newline="", encoding="utf-8-sig") as f:
401     w = csv.writer(f)
402     w.writerow(["case_id", "param", "value", "t_end_h", "elasticity", "note"])
403     w.writerow(["BASE", "-", "-", f"{tb:.4f}", "", "附录3 固定域基准 N=20"])
404     for cid, prm, val, te, el, note in rows:

```

```

405         w.writerow([cid, prm, val,
406                     f"{te:.4f}" if te is not None else "",
407                     f"{el:.4f}" if el is not None else "", note])
408     return {"base_t_end_h": tb, "n_cases": len(rows)}, probes
409
410
411 # -----
412 # 编排: 跑各证据 → validation.json
413 # -----
414 def run_all(shared=None):
415     """shared: 可选 dict 复用 main.py 已算的 P1/P3/P4 结果, 避免重复求解。"""
416     import problem1, problem3, problem4
417     if shared and all(k in shared for k in ("p1", "p3", "p4", "p4_summary")):
418         r1, r3, r4 = shared["p1"], shared["p3"], shared["p4"]
419         s4 = shared["p4_summary"]
420     else:
421         r1, _ = problem1.solve(write=False)
422         r3, _ = problem3.solve(write=False)
423         r4, s4 = problem4.solve(write=False)
424
425     ev = {}
426     ev["9_1_mass_budget"] = ev_mass({"P1": r1, "P3": r3, "P4_S2": r4})
427     ev["9_2_grid_independence"] = ev_grid()
428     ev["9_2_dt_independence"] = ev_dt_independence()
429     ev["9_2_tend_grid_sensitivity"] = ev_tend_gridsens()
430     ev["9_3_bessel_duhamel_P1"] = ev_bessel()
431     ev["9_4_physical"] = ev_physical(r1, r3)
432     ev["9_5_K18_moving_reduces_to_fixed"] = ev_limit_K18()
433     ev["9_5_dirichlet_limit"] = ev_dirichlet()
434     ev["X_C1_face_mode_convergence"] = ev_face_mode()
435
436     sa_summary, probes = sensitivity(r3)
437
438     # radius_frozen (§7.4 第4步; MC14)
439     radius_frozen = bool(r4["radius_frozen"])
440
441     checks = {
442         "K9_P3_tend_in_24_120": bool(24.0 <= r3["t_end_h"] <= 120.0),
443         "K10_P4_tend_in_24_120": bool(24.0 <= r4["t_end_h"] <= 120.0),
444         "K12_mass_residual": ev["9_1_mass_budget"]["pass"],
445         "K13_grid_deviation": ev["9_2_grid_independence"]["pass"],
446         "K18_moving_reduces_fixed": ev["9_5_K18_moving_reduces_to_fixed"]["pass"],
447         "K19_factorial_2x2": bool(
448             s4.get("lagrange_2x2")
449             and s4["lagrange_2x2"].get("t01") is not None
450             and np.isfinite(s4["lagrange_2x2"]["interaction_h"])),
451     }
452     all_pass = (all(e.get("pass", True) for e in ev.values())
453                and all(checks.values()))
454
455     validation = {
456         "problem": "2026-CUMCM-A-herb-drying",
457         "delivery_grid": {"N": P.N_CV, "dt_policy_P1": "1s", "dt_policy_P234": "seg(2->4)s"},
458         "radius_frozen": radius_frozen,
459         "evidence_section9": ev,
460         "constraints": checks,

```

```

461     "sensitivity": sa_summary,
462     "t_end_summary_h": {
463         "p3": r3["t_end_h"],
464         "P4_lagrange_primary": r4["t_end_h"],
465         "P4_euler_chi0": s4["crosscheck"]["euler_chi0_h"],
466         "P4_t01_lagrange": s4["lagrange_2x2"]["t01"],
467         "P4_interaction_h": s4["lagrange_2x2"]["interaction_h"],
468         "crosscheck_diff_rel": s4["crosscheck"]["diff_rel"]},
469     "all_pass": bool(all_pass),
470 }
471 (P.OUTPUT_DIR).mkdir(parents=True, exist_ok=True)
472 with open(P.OUTPUT_DIR / "validation.json", "w", encoding="utf-8") as f:
473     json.dump(validation, f, ensure_ascii=False, indent=2)
474 return validation, probes
475
476
477 if __name__ == "__main__":
478     v, _ = run_all()
479     print("validation all_pass:", v["all_pass"])
480     for k, e in v["evidence_section9"].items():
481         if isinstance(e, dict) and "pass" in e:
482             print(f"    {k}: pass={e['pass']}")
483     print("    constraints:", v["constraints"])

```

2.13 报表写出: io_out.py

```

1  """交付文件落盘 (CSV 报表 + result*.xlsx) 。
2
3  数值以 Excel 数值类型写入 (round 到 4 位小数 + 单元格格式 0.0000) ,
4  禁止把温度/含水率写成字符串。问题4 域外单元留空 (None) , 不是 0、也不是 'NaN'。
5  result*.xlsx 表头与附件3 模板一致: 首格「时间\\到药材中心的距离」+ 距离列 (cm) 。
6  result3/result4 工作表名保持模板的 Sheet1。
7  """
8  from __future__ import annotations
9
10 import csv
11 import math
12
13 import numpy as np
14 from openpyxl import Workbook
15 from openpyxl.cell import WriteOnlyCell
16
17 import params as P
18
19 HEADER0 = "时间\\到药材中心的距离"
20
21
22 def fmt4(x):
23     """4 位小数字符串; NaN/None → 空串。仅用于 CSV 报表, 不用于 xlsx。"""
24     if x is None:
25         return ""
26     xf = float(x)
27     if not math.isfinite(xf):
28         return ""

```



```

29     return f"{xf:.4f}"
30
31
32 def as_num4(x):
33     """xlsx 用: 有限值 → round(..., 4) 的 float; 否则 None (空白单元格)。"""
34     if x is None:
35         return None
36     xf = float(x)
37     if not math.isfinite(xf):
38         return None
39     return round(xf, 4)
40
41
42 def _num_cell(ws, value, number_format="0.0000"):
43     cell = WriteOnlyCell(ws, value=value)
44     if isinstance(value, (int, float)) and not isinstance(value, bool):
45         cell.number_format = number_format
46     return cell
47
48
49 def _dist_header(last_label=None):
50     """距离表头: 0.0..2.0 (21 列), 可选末列标签 (问题4「药材表面」)。"""
51     cols = [round(c, 4) for c in P.DIST_COLS_CM]
52     head = [HEADER0] + cols
53     if last_label is not None:
54         head.append(last_label)
55     return head
56
57
58 def write_result_xlsx(path, sheets, times_s, last_label=None):
59     """写 result*.xlsx。
60
61     sheets : dict 工作表名 -> 2D 数组 (nt, ncol), ncol=21 或 22 (含末列表面)。
62     times_s: 长度 nt 的时间列 (秒, 整数写出)。
63     温度/含水率为数值单元格 (四位小数), 域外为空白。
64     """
65     wb = Workbook(write_only=True)
66     header = _dist_header(last_label)
67     for name, arr in sheets.items():
68         ws = wb.create_sheet(title=name)
69         ws.append(header)
70         arr = np.asarray(arr)
71         for i in range(arr.shape[0]):
72             row = [_num_cell(ws, int(round(times_s[i])), "0")]
73             for v in arr[i]:
74                 nv = as_num4(v)
75                 row.append(None if nv is None else _num_cell(ws, nv))
76             ws.append(row)
77     path.parent.mkdir(parents=True, exist_ok=True)
78     wb.save(str(path))
79
80
81 def write_table_csv(path, row_labels, col_labels, matrix, corner=HEADER0,
82                    extra_last_row=None):
83     """写报表 CSV (表1-6)。matrix 经 fmt4 (含置空)。"""
84     path.parent.mkdir(parents=True, exist_ok=True)

```

```

85     with open(path, "w", newline="", encoding="utf-8-sig") as f:
86         w = csv.writer(f)
87         w.writerow([corner] + list(col_labels))
88         for lbl, vals in zip(row_labels, matrix):
89             w.writerow([lbl] + [fmt4(v) for v in vals])
90         if extra_last_row is not None:
91             lbl, vals = extra_last_row
92             w.writerow([lbl] + [fmt4(v) for v in vals])

```

2.14 附件数据核对: data_check.py

```

1  """数据核验：附件1(241行)/附件2(145行)、附件2时间轴、result模板表头。
2
3  只打印结论摘要，不把整表数组读进上下文。
4  """
5  from __future__ import annotations
6
7  import json
8  from pathlib import Path
9
10 import numpy as np
11 import pandas as pd
12
13 ROOT = Path(__file__).resolve().parent.parent
14 DATA = ROOT / "user_data" / "附件" / "附件"
15
16 A1 = DATA / "附件1.xlsx"
17 A2 = DATA / "附件2.xlsx"
18 RES = DATA / "附件3"
19
20
21 def main() -> int:
22     out = {}
23
24     # 附件1: 烘房温湿时程
25     df1 = pd.read_excel(A1, sheet_name=0)
26     out["附件1_rows"] = int(df1.shape[0])
27     out["附件1_cols"] = list(map(str, df1.columns))
28     t1 = pd.to_numeric(df1.iloc[:, 0], errors="coerce").to_numpy()
29     T1 = pd.to_numeric(df1.iloc[:, 1], errors="coerce").to_numpy()
30     C1 = pd.to_numeric(df1.iloc[:, 2], errors="coerce").to_numpy()
31     out["附件1_t"] = [float(np.nanmin(t1)), float(np.nanmax(t1))]
32     d1 = np.diff(t1)
33     out["附件1_dt_unique"] = sorted({round(float(x), 6) for x in d1})[:5]
34     out["附件1_T_range"] = [float(np.nanmin(T1)), float(np.nanmax(T1))]
35     out["附件1_C_range"] = [float(np.nanmin(C1)), float(np.nanmax(C1))]
36
37     # 附件2: 半径时程
38     df2 = pd.read_excel(A2, sheet_name=0)
39     out["附件2_rows"] = int(df2.shape[0])
40     out["附件2_cols"] = list(map(str, df2.columns))
41     t2 = pd.to_numeric(df2.iloc[:, 0], errors="coerce").to_numpy()
42     R2 = pd.to_numeric(df2.iloc[:, 1], errors="coerce").to_numpy()
43     out["附件2_t"] = [float(np.nanmin(t2)), float(np.nanmax(t2))]

```

```

44     d2 = np.diff(t2)
45     out["附件2_dt_unique"] = sorted({round(float(x), 6) for x in d2})[:5]
46     out["附件2_R_range"] = [float(np.nanmin(R2)), float(np.nanmax(R2))]
47     out["附件2_R_first3"] = [round(float(x), 6) for x in R2[:3]]
48     out["附件2_R_last3"] = [round(float(x), 6) for x in R2[-3:]]
49     # 单调性检查（原始）
50     out["附件2_R_monotone_nonincreasing"] = bool(np.all(d2 <= 1e-12) or np.all(np.diff(R2) <= 1e-12))
51     out["附件2_R_diff_max"] = float(np.nanmax(np.diff(R2)))
52
53     # 关键点核对
54     def R_at(target_s):
55         idx = int(np.nanargmin(np.abs(t2 - target_s)))
56         return round(float(t2[idx]), 1), round(float(R2[idx]), 6)
57     out["附件2_key_points"] = {
58         "6h": R_at(6 * 3600), "12h": R_at(12 * 3600),
59         "24h": R_at(24 * 3600), "36h": R_at(36 * 3600), "72h": R_at(72 * 3600),
60     }
61
62     # result 模板
63     for i in (1, 2, 3, 4):
64         f = RES / f"result{i}.xlsx"
65         xl = pd.ExcelFile(f)
66         out[f"result{i}_sheets"] = list(map(str, xl.sheet_names))
67         first = pd.read_excel(f, sheet_name=xl.sheet_names[0], header=None, nrows=3)
68         out[f"result{i}_head"] = first.astype(str).values.tolist()
69
70     print(json.dumps(out, ensure_ascii=False, indent=2))
71
72     # 断言（与 DATA_FACTS ingest_assertion 对齐）
73     assert out["附件1_rows"] == 241, f"附件1 行数应=241, 实际={out['附件1_rows']}"
74     assert out["附件2_rows"] == 145, f"附件2 行数应=145, 实际={out['附件2_rows']}"
75     print("DATA_INGEST_OK")
76     return 0
77
78
79 if __name__ == "__main__":
80     raise SystemExit(main())

```

2.15 模板结构核对：template_check.py

```

1  """核对 result 模板的行列规模、时间列取值范围；附件1尾部平台值。"""
2  from __future__ import annotations
3
4  import json
5  from pathlib import Path
6
7  import numpy as np
8  import pandas as pd
9
10 ROOT = Path(__file__).resolve().parent.parent
11 DATA = ROOT / "user_data" / "附件" / "附件"
12 RES = DATA / "附件3"
13
14

```

```

15 def main() -> int:
16     out = {}
17     for i in (1, 2, 3, 4):
18         f = RES / f"result{i}.xlsx"
19         xl = pd.ExcelFile(f)
20         sh = xl.sheet_names[0]
21         df = pd.read_excel(f, sheet_name=sh, header=0)
22         # 首列为时间标签
23         tcol = df.iloc[:, 0]
24         tvals = pd.to_numeric(tcol, errors="coerce").to_numpy()
25         tvals = tvals[np.isfinite(tvals)]
26         out[f"result{i}"] = {
27             "sheet0": sh,
28             "shape": [int(df.shape[0]), int(df.shape[1])],
29             "col_headers": list(map(str, df.columns)),
30             "n_time_rows": int(tvals.size),
31             "t_first3": [float(x) for x in tvals[:3]],
32             "t_last3": [float(x) for x in tvals[-3:]],
33             "t_step": float(tvals[1] - tvals[0]) if tvals.size > 1 else None,
34         }
35
36     # 附件1 尾部平台
37     df1 = pd.read_excel(DATA / "附件1.xlsx", sheet_name=0)
38     t1 = pd.to_numeric(df1.iloc[:, 0], errors="coerce").to_numpy()
39     T1 = pd.to_numeric(df1.iloc[:, 1], errors="coerce").to_numpy()
40     C1 = pd.to_numeric(df1.iloc[:, 2], errors="coerce").to_numpy()
41     out["附件1_tail"] = {
42         "t_last5": [float(x) for x in t1[-5:]],
43         "T_last5": [float(x) for x in T1[-5:]],
44         "C_last5": [float(x) for x in C1[-5:]],
45         "T_at_7200": float(T1[np.argmin(np.abs(t1 - 7200))]),
46         "C_at_9000": float(C1[np.argmin(np.abs(t1 - 9000))]),
47         "T_plateau_mean_last30": float(np.mean(T1[-30:])),
48         "C_plateau_mean_last30": float(np.mean(C1[-30:])),
49     }
50     print(json.dumps(out, ensure_ascii=False, indent=2))
51     return 0
52
53
54 if __name__ == "__main__":
55     raise SystemExit(main())

```

2.16 主程序入口：main.py

```

1 """交付编排入口（MODELING_REPORT §11 落盘清单）。
2
3 顺序：问题1 → 问题2/3（共用附录3 master, 1 s 全程只算一次）→ 问题4（S0-S4）
4       → §9 验证与灵敏度（复用已算 P1/P3/P4 结果）→ figures/*.json 汇总。
5 不绘图（图表阶段负责）；所有数值真算，不硬编码。
6 运行：python _utils/run_compute.py code/main.py
7 """
8 from __future__ import annotations
9
10 import json

```

```

11
12 import numpy as np
13
14 import params as P
15 import problem1
16 import problem2
17 import problem3
18 import problem4
19 import master_p23
20 import validate
21
22
23 def _jsonable(x):
24     """np 标量/数组 → 原生类型; NaN/inf → None (JSON 安全)。"""
25     if isinstance(x, dict):
26         return {k: _jsonable(v) for k, v in x.items()}
27     if isinstance(x, (list, tuple)):
28         return [_jsonable(v) for v in x]
29     if isinstance(x, (np.floating, float)):
30         xf = float(x)
31         return xf if np.isfinite(xf) else None
32     if isinstance(x, (np.integer,)):
33         return int(x)
34     if isinstance(x, (np.bool_, bool)):
35         return bool(x)
36     return x
37
38
39 def _write_json(path, obj):
40     path.parent.mkdir(parents=True, exist_ok=True)
41     with open(path, "w", encoding="utf-8") as f:
42         json.dump(_jsonable(obj), f, ensure_ascii=False, indent=2)
43
44
45 def main():
46     P.OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
47     P.FIGURE_DIR.mkdir(parents=True, exist_ok=True)
48
49     # ---- 问题1 (附录2, 1 s → 1800 s) ----
50     r1, s1 = problem1.solve(write=True)
51     print(f"[P1] center_C@1800={s1['center_C_1800']:.4f} "
52           f"mass_resid={s1['mass_resid_rel']:.2e}")
53
54     # ---- 问题2/3 (附录3 master: 1 s 全程只算一次) ----
55     r_master = master_p23.run_master()
56     r2, s2 = problem2.solve(res=r_master, write=True)
57     r3, s3 = problem3.solve(res=r_master, write=True)
58     print(f"[P2] process_end={s2['process_end_h']:.3f}h rows_result2={s2['n_rows_result2']}")
59     print(f"[P3] t_end={s3['t_end_h']:.3f}h maxC@end={s3['maxC_at_end']:.4f} "
60           f"mass_resid={s3['mass_resid_rel']:.2e}")
61
62     # ---- 问题4 (Lagrangian 主用 + Eulerian 交叉验证 + 2x2 因子) ----
63     r4, s4 = problem4.solve(write=True)
64     L = s4["lagrange_2x2"]
65     cc = s4["crosscheck"]
66     print(f"[P4] t_end(Lagrangian)={s4['t_end_h']:.3f}h ")

```

```

67         f"Eulerian={cc['euler_chi0_h']:.3f}h cross={cc['diff_rel']*100:+.2f}% "
68         f"Shapley 物性{L['prop_shapley_h']:+.3f} 收缩{L['shrink_shapley_h']:+.3f} "
69         f"I={L['interaction_h']:+.3f}h")
70
71 # ---- $9 验证 + $10 灵敏度 (复用 P1/P3/P4) ----
72 validation, probes = validate.run_all(
73     shared={"p1": r1, "p3": r3, "p4": r4, "p4_summary": s4})
74 print(f"[VAL] all_pass={validation['all_pass']} "
75       f"bessel_dev={validation['evidence_section9']['9_3_bessel_duhamel_P1']['max_dev_C']:.4f°C} "
76       f"K18={validation['constraints']['K18_moving_reduces_fixed']}")
77
78 # ---- figures/*.json (下游论文阶段读取; 含 logic_probes) ----
79 _write_json(P.FIGURE_DIR / "problem_1_results.json",
80            {"problem": 1, "summary": s1})
81 _write_json(P.FIGURE_DIR / "problem_2_results.json",
82            {"problem": 2, "summary": s2})
83 _write_json(P.FIGURE_DIR / "problem_3_results.json",
84            {"problem": 3, "summary": s3})
85 _write_json(P.FIGURE_DIR / "problem_4_results.json",
86            {"problem": 4, "summary": s4})
87
88 bounds_probes = [
89     {"quantity": "t_end_hours_P4_lagrange_primary", "claim": "upper",
90      "probe_delta_sign": 0,
91      "note": "交付 t_end 若落在附件2 数据覆盖内则半径冻结未触发"}
92 ]
93 all_results = {
94     "problem": "2026-CUMCM-A-herb-drying",
95     "delivery_grid": {"N": P.N_CV, "dt_P1_s": 1.0, "dt_P234": "seg(2->4)s"},
96     "headline": {
97         "P1_center_C_1800s": s1["center_C_1800"],
98         "P3_t_end_h": s3["t_end_h"],
99         "P4_t_end_h_lagrange_primary": s4["t_end_h"],
100        "P4_t_end_h_euler_chi0": cc["euler_chi0_h"],
101        "P4_crosscheck_rel": cc["diff_rel"],
102        "effect_isolation_h": {
103            "property_shapley": L["prop_shapley_h"],
104            "shrinkage_shapley": L["shrink_shapley_h"],
105            "interaction": L["interaction_h"],
106            "net": L["net_h"],
107            "t00": L["t00"], "t10": L["t10"],
108            "t01": L["t01"], "t11": L["t11"]},
109        },
110        "validation_all_pass": validation["all_pass"],
111        "radius_frozen": validation["radius_frozen"],
112        "logic_probes": {"bounds": bounds_probes, "monotonic": probes},
113    }
114    _write_json(P.FIGURE_DIR / "all_results.json", all_results)
115
116    print("[DONE] outputs in output/ and figures/. validation.json all_pass:",
117          validation["all_pass"])
118    return validation["all_pass"]
119
120
121 if __name__ == "__main__":
122     ok = main()

```

123

```
raise SystemExit(0 if ok else 1)
```